

Web Veri Madenciliđi – Sınav Soruları ve Ayrıntılı Çözümler

Yıldız Teknik Üniversitesi – Bilgisayar Mühendisliđi Bölümü

Haziran 2026

İçindekiler

1	Birliktelik Kuralları Madenciliđi ve Apriori Algoritması – Sınav Soruları	2
2	Bilgi Erişimi ve Metin İşleme – Sınav Soruları	8
3	Web Bağlantı Analizi, PageRank ve Merkezilik Ölçütleri — Sorular	19
4	Web Crawler Mimarisi ve Bloom Filtreleri – Sınav Soruları	28
5	Web Kullanım Madenciliđi – Sınav Soruları	37
6	Tavsiye Sistemleri — Sorular	47

1 Birliktelik Kuralları Madenciliği ve Apriori Algoritması – Sınav Soruları

Soru 1

Aşağıdaki işlem veritabanı T verilmiştir. Minimum destek eşiği $\text{minsup} = 0.4$ (yani en az 2 işlem) ve minimum güven eşiği $\text{minconf} = 0.7$ olarak belirlenmiştir.

TID	İşlem (Ürünler)
T1	A, B, C, D
T2	A, B, D
T3	B, C, E
T4	A, B, C, E
T5	A, C, D

Bu veritabanı için aşağıdakileri hesaplayınız: (a) $\{A, B\}$ ürün kümesinin destek değerini, (b) $A \rightarrow B$ kuralının güven değerini, (c) $B \rightarrow A$ kuralının güven değerini, (d) $\{A, C\} \rightarrow B$ kuralının destek ve güven değerlerini, (e) Bu kurallardan hangileri verilen eşik değerlerini sağlar?

Çözüm:

Veritabanında toplam $n = 5$ işlem bulunmaktadır. Destek, ilgili ürün kümesini içeren işlem sayısının toplam işlem sayısına oranıdır. Güven ise $X \rightarrow Y$ kuralı için $\text{destek}(X \cup Y) / \text{destek}(X)$ formülüyle hesaplanır.

Adım 1: Öncelikle her bir ürün kümesinin sayımını (support count) belirleyelim.

$\{A, B\}$ kümesi T1, T2 ve T4 işlemlerinde birlikte bulunur, dolayısıyla sayımı 3'tür.

$\{A, C\}$ kümesi T1, T4 ve T5 işlemlerinde bulunur, sayımı 3'tür.

$\{A, B, C\}$ kümesi ise T1 ve T4'te bulunur, sayımı 2'dir.

Tekil ürünlerin sayımları: A ürünü 4 işlemde (T1, T2, T4, T5), B ürünü 4 işlemde (T1, T2, T3, T4), C ürünü 4 işlemde (T1, T3, T4, T5), D ürünü 3 işlemde (T1, T2, T5), E ürünü 2 işlemde (T3, T4) yer alır.

Adım 2: (a) $\{A, B\}$ ürün kümesinin desteğini hesaplayalım. $\{A, B\}$ kümesinin sayımı 3'tür.

$$\text{destek}(\{A, B\}) = \frac{3}{5} = 0.6 = \%60 \quad (1)$$

Adım 3: (b) $A \rightarrow B$ kuralının güvenini hesaplayalım.

Destek değerleri üzerinden: $\text{destek}(\{A, B\}) = 3/5$ ve $\text{destek}(\{A\}) = 4/5$.

$$\text{güven}(A \rightarrow B) = \frac{\text{destek}(\{A, B\})}{\text{destek}(\{A\})} = \frac{3/5}{4/5} = \frac{3}{4} = 0.75 = \%75 \quad (2)$$

Adım 4: (c) $B \rightarrow A$ kuralının güvenini hesaplayalım.

$\text{destek}(\{B\}) = 4/5$ olduğundan:

$$\text{güven}(B \rightarrow A) = \frac{\text{destek}(\{A, B\})}{\text{destek}(\{B\})} = \frac{3/5}{4/5} = \frac{3}{4} = 0.75 = \%75 \quad (3)$$

Adım 5: (d) $\{A, C\} \rightarrow B$ kuralının desteğini ve güvenini hesaplayalım.

$\{A, B, C\}$ kümesi T1 ve T4'te bulunduğu için sayımı 2'dir. $\text{destek}(\{A, B, C\}) = 2/5$.

$$\text{destek}(\{A, C\} \rightarrow B) = \text{destek}(\{A, B, C\}) = \frac{2}{5} = 0.4 = \%40 \quad (4)$$

$\{A, C\}$ kümesinin sayımı T1, T4, T5'ten 3'tür. $\text{destek}(\{A, C\}) = 3/5$.

$$\text{güven}(\{A, C\} \rightarrow B) = \frac{\text{destek}(\{A, B, C\})}{\text{destek}(\{A, C\})} = \frac{2/5}{3/5} = \frac{2}{3} \approx 0.667 = \%66.7 \quad (5)$$

Adım 6: (e) Eşik değerleri minsup = 0.4 ve minconf = 0.7 ile karşılaştıralım.

$A \rightarrow B$ kuralı: destek $0.6 \geq 0.4$ ve güven $0.75 \geq 0.7$ olduğundan eşikleri sağlar.

$B \rightarrow A$ kuralı: destek $0.6 \geq 0.4$ ve güven $0.75 \geq 0.7$ olduğundan eşikleri sağlar.

$\{A, C\} \rightarrow B$ kuralı: destek $0.4 \geq 0.4$ sağlanır ancak güven $0.667 < 0.7$ olduğundan minimum güven eşliğini sağlamaz ve kabul edilmez.

Sonuç olarak $A \rightarrow B$ ve $B \rightarrow A$ kuralları kabul edilirken, $\{A, C\} \rightarrow B$ reddedilir.

Soru 2

Sık 3-ürün kümeleri kümesi $F_3 = \{\{A, B, C\}, \{A, B, D\}, \{A, C, D\}, \{A, C, E\}, \{B, C, D\}\}$ olarak verilmiştir. Apriori algoritmasının candidate-gen fonksiyonunu kullanarak C_4 aday 4-ürün kümelerini oluşturunuz. Birleştirme (join) ve budama (prune) adımlarını ayrıntılı olarak gösteriniz. Sonuçta elde edilen C_4 kümesini yazınız.

Çözüm:

Apriori candidate-gen fonksiyonu iki ana adımdan oluşur: birleştirme (join) ve budama (prune). Birleştirme adımında, F_{k-1} içindeki iki küme, ilk $k - 2$ elemanları aynı ve son elemanları farklı ise birleştirilir. Budama adımında ise oluşan adayın tüm $(k - 1)$ -elemanlı alt kümelerinin F_{k-1} içinde olup olmadığı kontrol edilir.

Adım 1: F_3 kümesini ilk iki elemanına göre gruplandırılalım.

İlk iki elemanı $\{A, B\}$ olanlar: $\{A, B, C\}$ ve $\{A, B, D\}$.

İlk iki elemanı $\{A, C\}$ olanlar: $\{A, C, D\}$ ve $\{A, C, E\}$.

İlk iki elemanı $\{B, C\}$ olan: yalnızca $\{B, C, D\}$ (birleştirme için eş yok).

Adım 2 – Birleştirme (Join): $k = 4$ için ilk $k - 2 = 2$ elemanı aynı olan ikililer birleştirilir.

$\{A, B, C\}$ ile $\{A, B, D\}$ birleştirilerek $\{A, B, C, D\}$ adayı elde edilir (son elemanlar $C < D$ koşulunu sağlar).

$\{A, C, D\}$ ile $\{A, C, E\}$ birleştirilerek $\{A, C, D, E\}$ adayı elde edilir (son elemanlar $D < E$ koşulunu sağlar).

Birleştirme adımından gelen adaylar:

$$C_4^{\text{join}} = \{\{A, B, C, D\}, \{A, C, D, E\}\} \quad (6)$$

Adım 3 – Budama (Prune): $\{A, B, C, D\}$ adayının tüm 3-elemanlı alt kümelerini kontrol edelim.

$\{A, B, C, D\}$ adayının 3-elemanlı alt kümeleri: $\{A, B, C\}$, $\{A, B, D\}$, $\{A, C, D\}$ ve $\{B, C, D\}$.

Bu alt kümelerin dördü de F_3 içinde mevcuttur. Dolayısıyla bu aday kalır.

Adım 4 – Budama (Prune): $\{A, C, D, E\}$ adayının tüm 3-elemanlı alt kümelerini kontrol edelim.

$\{A, C, D, E\}$ adayının 3-elemanlı alt kümeleri: $\{A, C, D\}$, $\{A, C, E\}$, $\{A, D, E\}$ ve $\{C, D, E\}$.

$\{A, C, D\}$ ve $\{A, C, E\}$ F_3 içinde bulunurken, $\{A, D, E\}$ ve $\{C, D, E\}$ F_3 içinde yer almaz.

Apriori özelliği gereği, bir alt küme sık değilse üst küme de sık olamaz. $\{A, D, E\} \notin F_3$ olduğundan $\{A, C, D, E\}$ adayı elenir.

Adım 5 – Sonuç: Budama sonrası nihai aday kümesi:

$$C_4 = \{\{A, B, C, D\}\} \quad (7)$$

Candidate-gen fonksiyonu tek bir aday üretir: $\{A, B, C, D\}$. Bu adayın gerçekten sık olup olmadığına karar vermek için veritabanı taranarak sayımının yapılması ve minsup eşikliğiyle karşılaştırılması gerekir.

Soru 3

Bir veri madenciliği uygulamasında minsup = 0.5 ve minconf = 0.75 olarak belirlenmiştir. Apriori algoritmasının ilk aşaması tamamlanmış ve aşağıdaki sık ürün kümeleri ile destek değerleri elde edilmiştir:

Sık Ürün Kümesi	Destek
$\{X, Y, Z\}$	0.50
$\{X, Y\}$	0.60
$\{X, Z\}$	0.50
$\{Y, Z\}$	0.75
$\{X\}$	0.75
$\{Y\}$	0.80
$\{Z\}$	0.80

$\{X, Y, Z\}$ sık ürün kümesinden türetililecek tüm olası birliktelik kurallarını yazınız. Her kural için destek ve güven değerlerini hesaplayınız. Hangi kuralların verilen eşik değerlerini sağladığını belirtiniz.

Çözüm:

$\{X, Y, Z\}$ sık ürün kümesinden kural türetmek için, bu kümenin tüm boş olmayan öz alt kümelerini A olarak alır, $B = \{X, Y, Z\} - A$ olarak belirler ve $A \rightarrow B$ kuralının güvenliğini hesaplarız. $\{X, Y, Z\}$ kümesinin öz alt kümeleri $\{X, Y\}$, $\{X, Z\}$, $\{Y, Z\}$, $\{X\}$, $\{Y\}$ ve $\{Z\}$ olmak üzere altı tanedir.

Adım 1 – Kural 1: $\{X, Y\} \rightarrow Z$ kuralını hesaplayalım.

Destek, $\{X, Y, Z\}$ kümesinin desteğidir: 0.50.

$$\text{güven}(\{X, Y\} \rightarrow Z) = \frac{\text{destek}(\{X, Y, Z\})}{\text{destek}(\{X, Y\})} = \frac{0.50}{0.60} = 0.833 = \%83.3 \quad (8)$$

Adım 2 – Kural 2: $\{X, Z\} \rightarrow Y$ kuralını hesaplayalım.

Destek: 0.50.

$$\text{güven}(\{X, Z\} \rightarrow Y) = \frac{0.50}{0.50} = 1.000 = \%100 \quad (9)$$

Adım 3 – Kural 3: $\{Y, Z\} \rightarrow X$ kuralını hesaplayalım.

Destek: 0.50.

$$\text{güven}(\{Y, Z\} \rightarrow X) = \frac{0.50}{0.75} = 0.667 = \%66.7 \quad (10)$$

Adım 4 – Kural 4: $X \rightarrow \{Y, Z\}$ kuralını hesaplayalım.

Destek: 0.50.

$$\text{güven}(X \rightarrow \{Y, Z\}) = \frac{0.50}{0.75} = 0.667 = \%66.7 \quad (11)$$

Adım 5 – Kural 5: $Y \rightarrow \{X, Z\}$ kuralını hesaplayalım.

Destek: 0.50.

$$\text{güven}(Y \rightarrow \{X, Z\}) = \frac{0.50}{0.80} = 0.625 = \%62.5 \quad (12)$$

Adım 6 – Kural 6: $Z \rightarrow \{X, Y\}$ kuralını hesaplayalım.

Destek: 0.50.

$$\text{güven}(Z \rightarrow \{X, Y\}) = \frac{0.50}{0.80} = 0.625 = \%62.5 \quad (13)$$

Adım 7 – Eşik Değerlendirmesi: Eşik değerleri minsup = 0.5 ve minconf = 0.75 olarak verilmiştir.

Tüm kuralların destek değeri 0.50 olduğundan hepsi minimum destek eşiğini sağlar.

Güven eşiğini (0.75) sağlayan kurallar ise yalnızca Kural 1 ($\{X, Y\} \rightarrow Z$, güven %83.3) ve Kural 2'dir ($\{X, Z\} \rightarrow Y$, güven %100).

Kural 3, 4, 5 ve 6'nın güven değerleri 0.75'in altında kaldığından kabul edilmezler.

Bu örnek, güven değerinin simetrik olmadığını ve $A \rightarrow B$ ile $B \rightarrow A$ kurallarının farklı güven değerlerine sahip olabileceğini göstermektedir.

Soru 4

PCY (Park-Chen-Yu) algoritmasının çalışma prensibini açıklayınız ve Apriori algoritmasından farklarını belirtiniz. Aşağıdaki senaryo için PCY algoritmasının birinci ve ikinci geçişlerinde neler yapıldığını adım adım anlatınız: 4 işlemli bir veritabanı, minsup = 2 ve hash fonksiyonu $h(i, j) = (i + j) \bmod 3$ olarak verilmiştir. Hash tablosunda 3 kova (0, 1, 2 numaralı) bulunmaktadır.

TID	İşlem
100	A, B, C
200	A, C, D
300	B, C, D
400	A, B, D

Çözüm:

Adım 1 – PCY'nin Çalışma Prensibi ve Apriori'den Farkı:

PCY algoritması, Apriori'nin ilk geçişinde ana belleğin büyük kısmının atıl durumda kalmasından faydalanarak geliştirilmiş bir iyileştirmedir. Apriori'den temel farkı, birinci geçişte yalnızca tekil ürün sayımlarını değil, aynı zamanda ikili çiftlerinin hash kovalarındaki sayımlarını da yapmasıdır. Apriori'de birinci geçişte sadece tekil ürünler sayılırken, PCY'de eş zamanlı olarak hash tablosu da doldurulur. İkinci geçişte Apriori yalnızca her iki ürünü de sık olan ikilileri sayarken, PCY ayrıca hash bitmap'inden de filtreleme yapar; böylece daha az aday ikili sayılır.

Adım 2 – Birinci Geçiş, İşlem İşlem İlerleyiş:

Tüm işlemler okunur. Her ürün için sayım yapılır ve aynı zamanda sepetteki tüm ikililer hash fonksiyonundan geçirilerek ilgili kovanın sayacı artırılır. Ürünleri A=1, B=2, C=3, D=4 olarak numaralandıralım. Hash fonksiyonu $h(i, j) = (i + j) \bmod 3$ için:

İşlem 100 (A, B, C): Ürün sayımları A++, B++, C++. İkililer: (A,B) $\rightarrow h(1+2)=3 \bmod 3=0$, kova 0++; (A,C) $\rightarrow h(1+3)=4 \bmod 3=1$, kova 1++; (B,C) $\rightarrow h(2+3)=5 \bmod 3=2$, kova 2++.

İşlem 200 (A, C, D): A++, C++, D++. İkililer: (A,C) $\rightarrow$ kova 1++; (A,D) $\rightarrow h(1+4)=5 \bmod 3=2$, kova 2++; (C,D) $\rightarrow h(3+4)=7 \bmod 3=1$, kova 1++.

İşlem 300 (B, C, D): B++, C++, D++. İkililer: (B,C) $\rightarrow$ kova 2++; (B,D) $\rightarrow h(2+4)=6 \bmod 3=0$, kova 0++; (C,D) $\rightarrow$ kova 1++.

İşlem 400 (A, B, D): A++, B++, D++. İkililer: (A,B) $\rightarrow$ kova 0++; (A,D) $\rightarrow$ kova 2++; (B,D) $\rightarrow$ kova 0++.

Adım 3 – Birinci Geçiş Sonuçları:

Ürün sayımları: A:3, B:3, C:3, D:3.

Hash kova sayımları: Kova 0: 4 (A,B:2 kez, B,D:2 kez), Kova 1: 4 (A,C:2 kez, C,D:2 kez), Kova 2: 4 (B,C:2 kez, A,D:2 kez).

minsup = 2 olduğundan tüm kovalar sık kovadır. Bitmap: [1, 1, 1]. Tüm ürünler sık olduğundan $F_1 = \{A, B, C, D\}$.

Adım 4 – İkinci Geçiş:

İkinci geçişte yalnızca iki koşulu sağlayan ikililer sayılır: (1) ikilinin her iki ürünü de sık olmalıdır, (2) ikilinin hash değerine karşılık gelen bitmap biti 1 olmalıdır.

Burada tüm ürünler sık ve tüm bitmap bitleri 1 olduğundan tüm ikililer sayılır. Bununla birlikte, PCY'nin asıl gücü bitmap'te 0 olan kovalar olduğunda ortaya çıkar; bu örnekte hash tablosunun küçük olması nedeniyle tüm kovalar sık çıkmıştır. Gerçek uygulamalarda çok daha büyük hash tabloları kullanılır.

İkinci geçişte ikili sayımları üçlüler tablosu ile yapılır (çünkü üçgensel matris hash filtrelemesi ile uyumlu değildir). Sonuç sayımlar: (A,B):2, (A,C):2, (A,D):2, (B,C):2, (B,D):2, (C,D):2. Tüm ikililer minsup = 2 eşliğini sağlar, dolayısıyla F_2 tüm bu ikilileri içerir.

Adım 5 – PCY'nin Avantajlarının Değerlendirilmesi:

PCY'nin Apriori'ye göre avantajı, birinci geçişte hash tablosu sayesinde sık olmayan kovalara düşen ikilileri ikinci geçişte sayıma dahil etmeyerek aday uzayını daraltmasıdır. PCY'nin Apriori'den daha iyi performans göstermesi için hash tablosunun aday ikililerin en az $2/3$ 'ünü elmesi gerekir. Ayrıca kova sayaçlarının minsup değerini aşması gerekmez; sayaç minsup değerine ulaştığında daha fazla artırılmaz.

Soru 5

Birliktelik kuralları madenciliğinde ikili (pair) sayımları için kullanılan üçgensel matris (triangular matrix) ve üçlüler tablosu (triples) yöntemlerini karşılaştırınız. Aşağıdaki iki senaryo için her iki yöntemin bellek gereksinimini hesaplayarak hangi yöntemin daha avantajlı olduğunu belirtiniz.

Parametre	Senaryo A	Senaryo B
Toplam ürün sayısı (n)	5.000	50.000
Sık ürün sayısı (m)	200	800
Gerçekten görülen ikili sayısı (p)	8.000	2.000.000

Her bir tam sayı sayacının 4 byte yer kapladığını ve üçlüler yönteminde her üçlünün $[i, j, c]$ olarak 12 byte (3 tam sayı) tuttuğunu varsayınız. Üçgensel matris için pozisyon formülünü de veriniz. Hangi koşullar altında üçlüler yönteminin üçgensel matristen daha az bellek kullandığını genel olarak tartışınız.

Çözüm:

İkili sayımlarının bellekte tutulması için iki temel yöntem vardır: üçgensel matris ve üçlüler tablosu. Her iki yöntem de Apriori algoritmasının ikinci geçişinde yalnızca sık ürünler arasındaki ikilileri saymak için kullanılır, bu nedenle bellek hesaplamalarında toplam ürün sayısı n yerine sık ürün sayısı m kullanılır.

Adım 1 – Üçgensel Matris Yöntemi:

Sık ürünler $1, 2, \dots, m$ şeklinde yeniden numaralandırılır. Yalnızca $i < j$ olan $\{i, j\}$ çiftleri sayılır ve bu çiftler $\{1, 2\}, \{1, 3\}, \dots, \{1, m\}, \{2, 3\}, \dots, \{m-1, m\}$ sıralamasıyla tek boyutlu bir dizide saklanır.

$\{i, j\}$ çiftinin ($i < j$) bu dizideki konumu aşağıdaki formülle hesaplanır:

$$k(i, j) = (i - 1) \left(m - \frac{i}{2} \right) + j - i \quad (14)$$

Toplam ikili sayısı $m(m-1)/2$ olduğundan, her ikili için 4 byte ayrılırsa toplam bellek kullanımı yaklaşık olarak şöyledir:

$$\text{Bellek}_{\text{matris}} \approx 4 \cdot \frac{m(m-1)}{2} \approx 2m^2 \text{ byte} \quad (15)$$

Adım 2 – Üçlüler Tablosu Yöntemi:

Yalnızca gerçekten görülen (sayımı sıfırdan büyük olan) çiftler için $[i, j, c]$ üçlülere saklanır. Burada c sayım değeridir. Her üçlü 12 byte (üç adet 4 byte'lık tam sayı) yer kaplar. Ek olarak verimli erişim için bir hash tablosu gibi ek veri yapıları gerekebilir, ancak basit hesaplamada yalnızca üçlülerin kendisi dikkate alınır:

$$\text{Bellek}_{\text{üçlü}} \approx 12p \text{ byte} \quad (16)$$

Adım 3 – Senaryo A Hesaplaması:

$m = 200$ sık ürün için üçgensel matris belleği:

$$\text{Bellek}_{\text{matris}} = 2 \cdot 200^2 = 2 \cdot 40.000 = 80.000 \text{ byte} \approx 80 \text{ KB} \quad (17)$$

Üçlüler yöntemi için $p = 8.000$ gerçek ikili:

$$\text{Bellek}_{\text{üçlü}} = 12 \cdot 8.000 = 96.000 \text{ byte} = 96 \text{ KB} \quad (18)$$

Senaryo A'da üçgensel matris (80 KB), üçlüler tablosundan (96 KB) daha az bellek kullanır. Üçgensel matris tercih edilmelidir.

Adım 4 – Senaryo B Hesaplaması:

$m = 800$ sık ürün için üçgensel matris belleği:

$$\text{Bellek}_{\text{matris}} = 2 \cdot 800^2 = 2 \cdot 640.000 = 1.280.000 \text{ byte} \approx 1.28 \text{ MB} \quad (19)$$

Üçlüler yöntemi için $p = 2.000.000$ gerçek ikili:

$$\text{Bellek}_{\text{üçlü}} = 12 \cdot 2.000.000 = 24.000.000 \text{ byte} = 24 \text{ MB} \quad (20)$$

Senaryo B'de üçgensel matris (1.28 MB), üçlüler tablosundan (24 MB) çok daha az bellek kullanır. Üçgensel matris yine tercih edilmelidir.

Adım 5 – Genel Karşılaştırma ve Başabaş Noktası:

İki yöntemin bellek kullanımlarının eşit olduğu noktayı bulalım:

$$2m^2 = 12p \Rightarrow p = \frac{m^2}{6} \quad (21)$$

Bu sonuç, gerçekten görülen ikili sayısı p , olası tüm ikililerin $m(m-1)/2$ sayısının yaklaşık $1/3$ 'ünden ($p \approx m^2/6$ ile $m^2/2$ 'nin oranı yaklaşık $1/3$) az ise üçlüler yönteminin daha az bellek kullandığını gösterir. Ancak gerçek uygulamalarda üçlüler yönteminin hash tablosu gibi ek yapılar gerektirmesi nedeniyle pratik eşik değeri daha düşüktür.

Apriori algoritması sık ürünleri yeniden numaralandırdığı için m genellikle yönetilebilir boyutadır ve üçgensel matris çoğu durumda ana belleğe sığar. PCY algoritmasında ise hash bitmap ile filtreleme yapıldığı için üçgensel matris yöntemi uyumlu değildir; bu durumda üçlüler yöntemi zorunlu olarak kullanılır. Senaryo B'deki gibi büyük ölçekli verilerde bile, sık ürün sayısı m tüm ürün sayısı n 'den çok daha küçük olduğu sürece üçgensel matris yöntemi ana belleğe sığabilir ve hızlı erişim avantajı sunar.

2 Bilgi Erişimi ve Metin İşleme – Sınav Soruları

Soru 1: TF-IDF Ağırlıklandırması

Aşağıda 5 dokümandan ($N = 5$) oluşan bir koleksiyon ve $V = \{\text{veri, madencilik, algoritma, sistem, ağ}\}$ sözlüğü için ham terim frekansları (f_{ij}) verilmiştir:

Doküman	veri	madencilik	algoritma	sistem	ağ
d_1	3	2	0	1	0
d_2	1	0	0	0	0
d_3	0	5	1	2	0
d_4	2	1	0	3	1
d_5	0	0	2	0	4

Log-normalize TF ve standart IDF kullanılmaktadır. TF için $w_{ij}^{\text{TF}} = 1 + \log_{10}(f_{ij})$ (eğer $f_{ij} > 0$ ise, aksi halde 0), IDF için $\text{IDF}(t_i) = \log_{10}(N/\text{df}_i)$ formülleri geçerlidir.

a) Her bir $t_i \in V$ terimi için doküman frekansını (df_i) ve IDF değerini hesaplayınız.

b) d_1 dokümanının TF-IDF ağırlık vektörünü hesaplayınız. Her terim için TF, IDF ve nihai TF-IDF değerini ayrı ayrı gösteriniz.

c) d_1 dokümanında hangi terim en yüksek TF-IDF ağırlığına sahiptir? Bu terimin neden en yüksek ağırlığı aldığını TF ve IDF bileşenleri açısından yorumlayınız.

Çözüm

Adım 1 – Doküman Frekanslarının Belirlenmesi:

Her terimin hangi dokümanlarda geçtiğini belirleyelim.

“veri” terimi d_1, d_2, d_4 olmak üzere 3 dokümanda geçmektedir; $\text{df}_{\text{veri}} = 3$.

“madencilik” terimi d_1, d_3, d_4 olmak üzere 3 dokümanda geçmektedir; $\text{df}_{\text{madencilik}} = 3$.

“algoritma” terimi d_3, d_5 olmak üzere 2 dokümanda geçmektedir; $\text{df}_{\text{algoritma}} = 2$.

“sistem” terimi d_1, d_3, d_4 olmak üzere 3 dokümanda geçmektedir; $\text{df}_{\text{sistem}} = 3$.

“ağ” terimi d_4, d_5 olmak üzere 2 dokümanda geçmektedir; $\text{df}_{\text{ağ}} = 2$.

Adım 2 – IDF Değerlerinin Hesaplanması:

$N = 5$ için IDF değerleri:

$$\text{IDF}(\text{veri}) = \log_{10}\left(\frac{5}{3}\right) = \log_{10}(1.6667) = 0.2218 \quad (22)$$

$$\text{IDF}(\text{madencilik}) = \log_{10}\left(\frac{5}{3}\right) = 0.2218 \quad (23)$$

$$\text{IDF}(\text{algoritma}) = \log_{10}\left(\frac{5}{2}\right) = \log_{10}(2.5) = 0.3979 \quad (24)$$

$$\text{IDF}(\text{sistem}) = \log_{10}\left(\frac{5}{3}\right) = 0.2218 \quad (25)$$

$$\text{IDF}(\text{ağ}) = \log_{10}\left(\frac{5}{2}\right) = \log_{10}(2.5) = 0.3979 \quad (26)$$

“algoritma” ve “ağ” terimleri daha az dokümanda geçtikleri ($\text{df}=2$) için daha yüksek IDF değerine sahiptir; yani bu terimler daha ayırt edicidir. “veri”, “madencilik” ve “sistem” ise 5 dokümanın 3’ünde geçtikleri için daha düşük IDF alırlar.

Adım 3 – d_1 Dokümanı için TF Değerlerinin Hesaplanması:

d_1 dokümanındaki ham frekanslar: veri(3), madencilik(2), algoritma(0), sistem(1), ağ(0).
Log-normalize TF değerleri:

$$\text{TF}(\text{veri}, d_1) = 1 + \log_{10}(3) = 1 + 0.4771 = 1.4771 \quad (27)$$

$$\text{TF}(\text{madencilik}, d_1) = 1 + \log_{10}(2) = 1 + 0.3010 = 1.3010 \quad (28)$$

$$\text{TF}(\text{algoritma}, d_1) = 0 \quad (f_{ij} = 0 \text{ olduğu için}) \quad (29)$$

$$\text{TF}(\text{sistem}, d_1) = 1 + \log_{10}(1) = 1 + 0 = 1.0000 \quad (30)$$

$$\text{TF}(\text{ağ}, d_1) = 0 \quad (f_{ij} = 0 \text{ olduğu için}) \quad (31)$$

Adım 4 – d_1 Dokümanı için TF-IDF Ağırlıklarının Hesaplanması:

Nihai TF-IDF ağırlıkları ($w_{ij} = \text{TF} \times \text{IDF}$):

$$w_{\text{veri}, d_1} = 1.4771 \times 0.2218 = 0.3277 \quad (32)$$

$$w_{\text{madencilik}, d_1} = 1.3010 \times 0.2218 = 0.2886 \quad (33)$$

$$w_{\text{algoritma}, d_1} = 0 \times 0.3979 = 0.0000 \quad (34)$$

$$w_{\text{sistem}, d_1} = 1.0000 \times 0.2218 = 0.2218 \quad (35)$$

$$w_{\text{ağ}, d_1} = 0 \times 0.3979 = 0.0000 \quad (36)$$

Adım 5 – d_1 Dokümanının TF-IDF Ağırlık Vektörü:

$$d_1 = (0.3277, 0.2886, 0.0000, 0.2218, 0.0000) \quad (37)$$

Adım 6 – En Yüksek TF-IDF Ağırlığına Sahip Terimin Belirlenmesi ve Yorumlanması:

d_1 dokümanında en yüksek TF-IDF ağırlığına sahip terim 0.3277 ile “veri” dir.

Bunun nedeni, “veri” teriminin d_1 ’de 3 kez geçerek en yüksek log-normalize TF değerini (1.4771) almasıdır. “madencilik” terimi 2 kez geçerek 1.3010 TF değeri alırken, “sistem” 1 kez geçerek 1.0000 TF değeri almıştır. Her üç terimin de IDF değeri aynıdır (0.2218), çünkü üçü de 3 dokümanda geçmektedir. Bu nedenle TF-IDF sıralamasını belirleyen temel faktör, d_1 içindeki ham frekans farkları olmuştur.

Log-normalizasyon sayesinde 3 kat frekans farkı (3’e karşı 1) ağırlıkta yalnızca $1.4771/1.0000 \approx 1.48$ kat fark yaratmıştır; bu da aşırı yüksek frekanslı terimlerin baskınlığını sınırlayan log dönüşümünün etkisini göstermektedir.

Soru 2: Kosinüs Benzerliği ile Sorgu-Doküman Eşleştirme

Soru 1'deki aynı doküman koleksiyonunu kullanarak, $q = \text{"veri madencilik"}$ sorgusunu ele alalım. Sorgu vektörü, mevcut terimler için 1, olmayanlar için 0 olacak şekilde ikili (binary) ağırlıklandırılmıştır: $q = (1, 1, 0, 0, 0)$. Doküman vektörleri ise Soru 1'de hesaplanan TF-IDF ağırlıklarını kullanmaktadır. Kosinüs benzerliği şu formülle hesaplanır:

$$\cos(q, d_j) = \frac{q \cdot d_j}{\|q\| \cdot \|d_j\|} = \frac{\sum_{i=1}^{|V|} w_{iq} \cdot w_{ij}}{\sqrt{\sum_{i=1}^{|V|} w_{iq}^2} \cdot \sqrt{\sum_{i=1}^{|V|} w_{ij}^2}} \quad (38)$$

a) d_1 ve d_3 dokümanları için kosinüs benzerliklerini adım adım hesaplayınız.

b) Tüm 5 doküman için kosinüs benzerliklerini hesaplayarak dokümanları azalan benzerlik sırasına göre sıralayınız.

c) d_2 dokümanı yalnızca “veri” terimini içermesine rağmen neden d_4 'ten daha yüksek bir kosinüs benzerliği skoru almıştır? Kosinüs normalizasyonunun rolünü tartışınız.

Çözüm

Adım 1 – Tüm Dokümanların TF-IDF Vektörlerinin Hesaplanması:

Öncelikle tüm dokümanların TF-IDF vektörlerini Soru 1'deki yöntemle hesaplayalım. Her doküman için log-normalize TF ve IDF çarpımları:

$d_1 = (0.3277, 0.2886, 0.0000, 0.2218, 0.0000)$ (Soru 1'den)

d_2 : veri(1), diğerleri(0). $\text{TF}(\text{veri}) = 1 + \log_{10}(1) = 1$. $w_{\text{veri}} = 1 \times 0.2218 = 0.2218$.

$$d_2 = (0.2218, 0, 0, 0, 0) \quad (39)$$

d_3 : madencilik(5), algoritma(1), sistem(2). $\text{TF}(\text{madencilik}) = 1 + \log_{10}(5) = 1 + 0.6990 = 1.6990$, $w = 1.6990 \times 0.2218 = 0.3769$. $\text{TF}(\text{algoritma}) = 1 + \log_{10}(1) = 1$, $w = 1 \times 0.3979 = 0.3979$. $\text{TF}(\text{sistem}) = 1 + \log_{10}(2) = 1 + 0.3010 = 1.3010$, $w = 1.3010 \times 0.2218 = 0.2886$.

$$d_3 = (0, 0.3769, 0.3979, 0.2886, 0) \quad (40)$$

d_4 : veri(2), madencilik(1), sistem(3), ağ(1). $\text{TF}(\text{veri}) = 1 + \log_{10}(2) = 1.3010$, $w = 1.3010 \times 0.2218 = 0.2886$. $\text{TF}(\text{madencilik}) = 1$, $w = 0.2218$. $\text{TF}(\text{sistem}) = 1 + \log_{10}(3) = 1.4771$, $w = 0.3277$. $\text{TF}(\text{ağ}) = 1$, $w = 1 \times 0.3979 = 0.3979$.

$$d_4 = (0.2886, 0.2218, 0, 0.3277, 0.3979) \quad (41)$$

d_5 : algoritma(2), ağ(4). $\text{TF}(\text{algoritma}) = 1 + \log_{10}(2) = 1.3010$, $w = 1.3010 \times 0.3979 = 0.5177$. $\text{TF}(\text{ağ}) = 1 + \log_{10}(4) = 1 + 0.6021 = 1.6021$, $w = 1.6021 \times 0.3979 = 0.6375$.

$$d_5 = (0, 0, 0.5177, 0, 0.6375) \quad (42)$$

Adım 2 – Sorgu Vektörünün Normunun Hesaplanması:

Sorgu vektörü: $q = (1, 1, 0, 0, 0)$.

$$\|q\| = \sqrt{1^2 + 1^2 + 0^2 + 0^2 + 0^2} = \sqrt{2} \approx 1.4142 \quad (43)$$

Adım 3 – d_1 için Kosinüs Benzerliği:

Nokta çarpımı:

$$q \cdot d_1 = 1 \times 0.3277 + 1 \times 0.2886 + 0 + 0 + 0 = 0.6163 \quad (44)$$

d_1 'in normu:

$$\|d_1\| = \sqrt{0.3277^2 + 0.2886^2 + 0 + 0.2218^2 + 0} = \sqrt{0.1074 + 0.0833 + 0.0492} = \sqrt{0.2399} = 0.4898 \quad (45)$$

Kosinüs benzerliği:

$$\cos(q, d_1) = \frac{0.6163}{1.4142 \times 0.4898} = \frac{0.6163}{0.6927} = 0.8897 \quad (46)$$

Adım 4 – d_3 için Kosinüs Benzerliği:

Nokta çarpımı:

$$q \cdot d_3 = 0 + 1 \times 0.3769 + 0 + 0 + 0 = 0.3769 \quad (47)$$

d_3 'ün normu:

$$\|d_3\| = \sqrt{0 + 0.3769^2 + 0.3979^2 + 0.2886^2 + 0} = \sqrt{0.1421 + 0.1583 + 0.0833} = \sqrt{0.3837} = 0.6194 \quad (48)$$

Kosinüs benzerliği:

$$\cos(q, d_3) = \frac{0.3769}{1.4142 \times 0.6194} = \frac{0.3769}{0.8760} = 0.4303 \quad (49)$$

Adım 5 – d_2 için Kosinüs Benzerliği:

Nokta çarpımı ve norm:

$$q \cdot d_2 = 0.2218, \quad \|d_2\| = 0.2218 \quad (50)$$

$$\cos(q, d_2) = \frac{0.2218}{1.4142 \times 0.2218} = \frac{1}{1.4142} = 0.7071 \quad (51)$$

Adım 6 – d_4 için Kosinüs Benzerliği:

Nokta çarpımı:

$$q \cdot d_4 = 0.2886 + 0.2218 = 0.5104 \quad (52)$$

d_4 'ün normu:

$$\|d_4\| = \sqrt{0.2886^2 + 0.2218^2 + 0 + 0.3277^2 + 0.3979^2} = \sqrt{0.0833 + 0.0492 + 0.1074 + 0.1583} = \sqrt{0.3982} = 0.631 \quad (53)$$

Kosinüs benzerliği:

$$\cos(q, d_4) = \frac{0.5104}{1.4142 \times 0.6310} = \frac{0.5104}{0.8923} = 0.5720 \quad (54)$$

Adım 7 – d_5 için Kosinüs Benzerliği:

d_5 sorgu terimlerinin hiçbirini içermediği için:

$$q \cdot d_5 = 0, \quad \cos(q, d_5) = 0 \quad (55)$$

Adım 8 – Azalan Benzerlik Sıralaması:

$$d_1(0.890) > d_2(0.707) > d_4(0.572) > d_3(0.430) > d_5(0.000) \quad (56)$$

Adım 9 – d_2 'nin d_4 'ten Yüksek Skor Almasının Nedeni:

d_2 yalnızca “veri” terimini içerir ve vektör normu $\|d_2\| = 0.2218$ ile çok küçüktür. d_4 ise “veri” ve “madencilik” dahil 4 terim içerir ve vektör normu $\|d_4\| = 0.6310$ ile belirgin şekilde daha büyüktür.

$q \cdot d_2 = 0.2218$ iken $q \cdot d_4 = 0.5104$ yani d_4 'ün ham nokta çarpımı daha yüksektir. Ancak kosinüs benzerliği, nokta çarpımını vektör uzunluklarıyla normalize eder. d_2 'nin tek bir terim içeren “kısa” bir vektör olması, normalize edilmiş skorunu yükseltir; d_4 ise birçok terim içerdiği için normu büyür ve benzerlik skoru düşer.

Bu, kosinüs benzerliğinin en önemli özelliğidir: uzun dokümanların sadece daha fazla kelime içerdikleri için yüksek skor almasını engeller. Kosinüs, vektörler arasındaki açıyı ölçtüğü için doküman uzunluğundan bağımsız bir benzerlik değerlendirmesi sunar.

Soru 3: Boolean Modeli ve Ters İndeks Yapısı

Aşağıda 6 dokümandan oluşan bir koleksiyon verilmiştir. Dokümanlar durma kelimelerinden arındırılmış ve gövdelenmiş halleriyle gösterilmektedir:

- D_1 : bilgi madencilik sistem
- D_2 : veri taban yönetim
- D_3 : bilgi erişim veri madencilik
- D_4 : sistem yönetim algoritma
- D_5 : bilgi veri algoritma erişim
- D_6 : madencilik veri taban

a) Bu koleksiyon için ters indeks (inverted index) yapısını oluşturunuz. Her benzersiz terim için, o terimi içeren dokümanların listesini (posting list) yazınız.

b) $Q_1 = \text{“bilgi AND madencilik”}$ Boolean sorgusu için, ters indeksi kullanarak hangi dokümanların getirileceğini adım adım gösteriniz.

c) $Q_2 = \text{“(veri OR sistem) AND NOT algoritma”}$ Boolean sorgusu için, ters indeksi kullanarak hangi dokümanların getirileceğini adım adım gösteriniz.

d) Boolean modelinin bir avantajını ve bir dezavantajını, yukarıdaki Q_1 sorgusunun sonuçlarını da göz önünde bulundurarak açıklayınız.

Çözüm

Adım 1 – Benzersiz Terimlerin Belirlenmesi:

Önce koleksiyondaki tüm benzersiz terimleri belirleyelim: $V = \{\text{algoritma, bilgi, erişim, madencilik, sistem, taban, veri, yönetim}\}$

Adım 2 – Ters İndeks (Posting List) Yapısının Oluşturulması:

Her terim için hangi dokümanlarda geçtiğini tarayalım.

“algoritma” D_4 ve D_5 'te geçmektedir; gönderi listesi $\{D_4, D_5\}$.

“bilgi” D_1 , D_3 ve D_5 'te geçmektedir; gönderi listesi $\{D_1, D_3, D_5\}$.

“erişim” D_3 ve D_5 'te geçmektedir; gönderi listesi $\{D_3, D_5\}$.

“madencilik” D_1 , D_3 ve D_6 'da geçmektedir; gönderi listesi $\{D_1, D_3, D_6\}$.

“sistem” D_1 ve D_4 'te geçmektedir; gönderi listesi $\{D_1, D_4\}$.

“taban” D_2 ve D_6 'da geçmektedir; gönderi listesi $\{D_2, D_6\}$.

“veri” D_2 , D_3 , D_5 ve D_6 'da geçmektedir; gönderi listesi $\{D_2, D_3, D_5, D_6\}$.

“yönetim” D_2 ve D_4 'te geçmektedir; gönderi listesi $\{D_2, D_4\}$.

Ters indeksin avantajı, bir sorgu terimini içeren dokümanları sabit zamanda ($O(1)$) bulabilmesidir. Sözlük (vocabulary) üzerinden doğrudan ilgili gönderi listesine erişilir; tüm dokümanların taranması gerekmez. Büyük ölçekli koleksiyonlarda (milyonlarca doküman) bu yapı, sorgu işleme süresini dramatik biçimde kısaltır.

Adım 3 – Q_1 Sorgusunun İşlenmesi, Sözlük Araması:

Q_1 = “bilgi AND madencilik” sorgusu için:

“bilgi” terimi için ters indeksten gönderi listesi alınır: $\{D_1, D_3, D_5\}$.

“madencilik” terimi için gönderi listesi alınır: $\{D_1, D_3, D_6\}$.

Adım 4 – Q_1 Sorgusunun İşlenmesi, Kesişim (AND) İşlemi:

İki listenin ortak elemanları bulunur:

$$\{D_1, D_3, D_5\} \cap \{D_1, D_3, D_6\} = \{D_1, D_3\} \quad (57)$$

Sıralı listeler üzerinde kesişim algoritması, iki işaretçi ile doğrusal zamanda ($O(|L_1| + |L_2|)$) çalışır: $D_1 = D_1$ olduğu için D_1 sonuca eklenir; $D_3 = D_3$ olduğu için D_3 sonuca eklenir; $D_5 < D_6$ olduğu için ilk listenin işaretçisi ilerler ve liste sonuna gelinir.

Adım 5 – Q_1 Sorgusunun Sonucu:

Q_1 sorgusu D_1 ve D_3 dokümanlarını getirir. Gerçekten de D_1 hem “bilgi” hem “madencilik” içerir, D_3 de her ikisini içerir. D_5 “bilgi” içerir ama “madencilik” içermez; D_6 “madencilik” içerir ama “bilgi” içermez. Boolean AND işlemi yalnızca her iki koşulu birden sağlayan dokümanları döndürmüştür.

Adım 6 – Q_2 Sorgusunun İşlenmesi, Sözlük Araması:

Q_2 = “(veri OR sistem) AND NOT algoritma” sorgusu için:

“veri” için gönderi listesi $\{D_2, D_3, D_5, D_6\}$.

“sistem” için gönderi listesi $\{D_1, D_4\}$.

“algoritma” için gönderi listesi $\{D_4, D_5\}$.

Adım 7 – Q_2 Sorgusunun İşlenmesi, Birleşim (OR) İşlemi:

“veri” ve “sistem” listeleri birleştirilir:

$$\{D_2, D_3, D_5, D_6\} \cup \{D_1, D_4\} = \{D_1, D_2, D_3, D_4, D_5, D_6\} \quad (58)$$

Adım 8 – Q_2 Sorgusunun İşlenmesi, Fark (AND NOT) İşlemi:

Birleşim kümesinden “algoritma” içeren dokümanlar çıkarılır:

$$\{D_1, D_2, D_3, D_4, D_5, D_6\} \setminus \{D_4, D_5\} = \{D_1, D_2, D_3, D_6\} \quad (59)$$

Adım 9 – Q_2 Sorgusunun Sonucu:

D_4 “sistem” içermesine rağmen aynı zamanda “algoritma” içerdiği için elenmiştir. D_5 “veri” içermesine rağmen “algoritma” içerdiği için elenmiştir. Sonuç olarak D_1, D_2, D_3, D_6 dokümanları getirilir.

Adım 10 – Boolean Modelinin Avantajı:

Boolean modeli son derece basit ve hesaplama açısından verimlidir. Q_1 örneğinde görüldüğü gibi, AND, OR, NOT operatörleriyle kesin mantıksal koşullar ifade edilebilir ve ters indeks sayesinde sonuçlar çok hızlı bir şekilde (yalnızca gönderi listeleri üzerinde küme işlemleriyle) elde edilebilir. Tam eşleşme (exact match) prensibi, kullanıcıya net ve anlaşılır sonuçlar sunar.

Adım 11 – Boolean Modelinin Dezavantajı:

Boolean modeli terim frekansını dikkate almaz ve sonuçlarda sıralama yapamaz. Q_1 sorgusunda D_1 ve D_3 arasında bir ilgililik derecesi ayrımı yapılamaz; oysa D_1 ’de “bilgi” 1 kez, D_3 ’te de 1 kez geçmektedir ama D_3 aynı zamanda “erişim” ve “veri” gibi ek terimler de içermektedir – Boolean modeli bu zenginliği yakalayamaz.

Ayrıca, “bilgi” terimini içeren ama “madencilik” içermeyen D_5 gibi kısmen ilgili dokümanlar tamamen dışlanır; kullanıcıya “bu doküman sorgunuzla kısmen ilgili olabilir” bilgisi verilemez. Bu eksiklikler nedeniyle Boolean modeli büyük koleksiyonlarda tek başına yetersiz kalır ve genellikle vektör uzay modeli gibi sıralama yapabilen modellerle hibrit olarak kullanılır.

Soru 4: Okapi BM25 ile İlgililik Skoru Hesaplama

Bir arama motorunda $N = 1000$ doküman bulunmaktadır. $q = \text{“yapay zeka”}$ sorgusu için d dokümanının BM25 skoru hesaplanacaktır. İlgili parametreler ve değerler aşağıda verilmiştir:

“yapay” terimi için doküman frekansı $df = 50$, d dokümanındaki frekansı $f = 3$. “zeka” terimi için doküman frekansı $df = 100$, d dokümanındaki frekansı $f = 2$. d dokümanının uzunluğu $|d| = 200$ kelime. Koleksiyondaki ortalama doküman uzunluğu $avgdl = 150$ kelime. BM25 parametreleri: $k_1 = 1.5$, $b = 0.75$.

BM25 formülü ve IDF bileşeni aşağıdaki gibidir:

$$BM25(q, d) = \sum_{t \in q} IDF(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{avgdl}\right)} \quad (60)$$

$$IDF(t) = \log_{10} \left(\frac{N - df_t + 0.5}{df_t + 0.5} \right) \quad (61)$$

a) Her iki sorgu terimi için IDF değerlerini hesaplayınız. “yapay” teriminin IDF değeri neden “zeka” teriminden daha yüksektir?

b) d dokümanı için K normalizasyon faktörünü ve her iki terimin BM25 katkısını ayrı ayrı hesaplayarak nihai BM25 skorunu bulunuz.

c) Aynı d dokümanında “yapay” terimi $f = 3$ yerine $f = 30$ kez geçseydi, bu terimin BM25 katkısı ne olurdu? Sonucu (b) şıkkındaki değerle karşılaştırarak k_1 parametresinin TF doygunluğu (saturation) üzerindeki rolünü açıklayınız.

Çözüm

Adım 1 – “yapay” Terimi için IDF Hesabı:

$$IDF(yapay) = \log_{10} \left(\frac{1000 - 50 + 0.5}{50 + 0.5} \right) = \log_{10} \left(\frac{950.5}{50.5} \right) = \log_{10}(18.8218) = 1.2746 \quad (62)$$

Adım 2 – “zeka” Terimi için IDF Hesabı:

$$IDF(zeka) = \log_{10} \left(\frac{1000 - 100 + 0.5}{100 + 0.5} \right) = \log_{10} \left(\frac{900.5}{100.5} \right) = \log_{10}(8.9602) = 0.9523 \quad (63)$$

Adım 3 – IDF Değerlerinin Karşılaştırılması:

“yapay” teriminin IDF değeri (1.2746), “zeka” teriminin IDF değerinden (0.9523) daha yüksektir. Bunun nedeni, “yapay” kelimesinin koleksiyonda daha az dokümanda geçmesidir ($df=50$ ’ye karşı $df=100$). IDF’nin temel prensibi gereği, daha nadir terimler daha ayırt edici oldukları için daha yüksek ağırlık alırlar. $N = 1000$ dokümanlık bir koleksiyonda “yapay” yalnızca 50 dokümanda (%5) geçerken, “zeka” 100 dokümanda (%10) geçmektedir. Formüldeki $+0.5$ düzeltme terimleri, çok düşük df değerlerinde sıfıra bölme gibi sayısal sorunları önlemek için eklenmiştir.

Adım 4 – K Normalizasyon Faktörünün Hesaplanması:

$$K = k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{avgdl}\right) \quad (64)$$

$$K = 1.5 \cdot \left(1 - 0.75 + 0.75 \cdot \frac{200}{150}\right) = 1.5 \cdot (0.25 + 0.75 \cdot 1.3333) \quad (65)$$

$$K = 1.5 \cdot (0.25 + 1.0000) = 1.5 \cdot 1.25 = 1.875 \quad (66)$$

$|d| = 200 > \text{avgdl} = 150$ olduğu için d dokümanı ortalamadan uzundur ve $K = 1.875$ değeri, $b = 0$ durumundaki $K = k_1 = 1.5$ değerinden daha büyük olmuştur. Daha büyük K , daha güçlü normalizasyon ve dolayısıyla daha düşük terim katkısı anlamına gelir.

Adım 5 – “yapay” Teriminin BM25 Katkısı:

$$\text{katkı}_{\text{yapay}} = 1.2746 \cdot \frac{3 \cdot (1.5 + 1)}{3 + 1.875} = 1.2746 \cdot \frac{3 \cdot 2.5}{4.875} = 1.2746 \cdot \frac{7.5}{4.875} \quad (67)$$

$$\text{katkı}_{\text{yapay}} = 1.2746 \cdot 1.5385 = 1.9610 \quad (68)$$

Adım 6 – “zeka” Teriminin BM25 Katkısı:

$$\text{katkı}_{\text{zeka}} = 0.9523 \cdot \frac{2 \cdot 2.5}{2 + 1.875} = 0.9523 \cdot \frac{5.0}{3.875} \quad (69)$$

$$\text{katkı}_{\text{zeka}} = 0.9523 \cdot 1.2903 = 1.2290 \quad (70)$$

Adım 7 – Nihai BM25 Skoru ve Yorumu:

$$\text{BM25}(q, d) = 1.9610 + 1.2290 = 3.1900 \quad (71)$$

d dokümanı için “yapay” teriminin katkısı (1.9610), “zeka” teriminin katkısından (1.2290) daha yüksektir. Bunun iki nedeni vardır: “yapay”ın IDF değeri daha yüksektir (daha nadir bir terim) ve d dokümanında daha sık geçmektedir (3’e karşı 2). BM25, her iki faktörü de hesaba katarak dengeli bir ilgililik skoru üretir.

Adım 8 – $f = 30$ Durumu için “yapay” Teriminin BM25 Katkısı:

“yapay” terimi d ’de $f = 30$ kez geçseydi:

$$\text{katkı}_{\text{yapay}} = 1.2746 \cdot \frac{30 \cdot 2.5}{30 + 1.875} = 1.2746 \cdot \frac{75.0}{31.875} \quad (72)$$

$$\text{katkı}_{\text{yapay}} = 1.2746 \cdot 2.3529 = 2.9990 \quad (73)$$

Adım 9 – TF Doygunluğu (Saturation) Analizi:

$f = 3$ için katkı 1.9610, $f = 30$ için katkı 2.9990’dır. Terim frekansı 10 kat artmasına rağmen ($3 \rightarrow 30$), BM25 katkısı yalnızca yaklaşık 1.53 kat artmıştır ($1.9610 \rightarrow 2.9990$). Bu, k_1 parametresinin sağladığı TF doygunluğunun (saturation) doğrudan bir göstergesidir.

BM25 formülünde $\frac{f \cdot (k_1 + 1)}{f + K}$ terimi incelendiğinde: f küçükken ($f \ll K$) bu oran yaklaşık olarak $\frac{k_1 + 1}{K} \cdot f$ şeklinde doğrusal davranır; f büyüdükçe ($f \gg K$) oran $(k_1 + 1)$ değerine yakınsar ve artış durur. $k_1 = 1.5$ için üst sınır $k_1 + 1 = 2.5$ ’tir; yani bir terim dokümanda sonsuz kez geçse bile TF bileşeninin çarpma katkısı en fazla 2.5 olabilir. Bu mekanizma, bir terimin dokümanda çok fazla geçmesinin marjinal faydasını sınırlayarak, TF-IDF’nin doğrusal yapısına kıyasla çok daha gerçekçi bir modelleme sunar. Pratikte, bir kelimenin ilk birkaç geçişi dokümanın konusu hakkında çok şey söylerken, 50. veya 100. geçişi çok az ek bilgi taşır – BM25 bu sezgiyi matematiksel olarak modeller.

Soru 5: Kesinlik, Geri Çağırma, Karmaşıklık Matrisi ve Rocchio Geri Bildirimi

Bir bilgi erişim sisteminde “makine öğrenmesi” sorgusu için yapılan arama sonucunda sistem 20 doküman döndürmüştür. Kullanıcı bu dokümanları incelemiş ve 8 tanesinin gerçekten ilgili, 12 tanesinin ise ilgisiz olduğunu belirlemiştir. Koleksiyonda bu sorguyla ilgili toplam 25 doküman bulunmaktadır. Ayrıca koleksiyonun toplam büyüklüğü 200 dokümandır.

a) Bu sonuçlar için karmaşıklık matrisini (confusion matrix) oluşturunuz. TP (doğru pozitif), FP (yanlış pozitif), FN (yanlış negatif) ve TN (doğru negatif) değerlerini ayrı ayrı belirtiniz.

b) Kesinlik (precision), geri çağırma (recall) ve F1-skoru değerlerini hesaplayınız. Kesinlik ile geri çağırma arasındaki ters orantıyı bu örnek üzerinden yorumlayınız.

c) Kullanıcı ilk tur sonuçlarından memnun kalmamış ve Rocchio ilgililik geri bildirimi uygulamak istemektedir. Orijinal sorgu vektörü $q_0 = (0.5, 0.3, 0.0, 0.2)$, ilgili dokümanların centroid’i $\bar{D}_r = (0.8, 0.6, 0.4, 0.1)$ ve ilgisiz dokümanların centroid’i $\bar{D}_{ir} = (0.1, 0.2, 0.9, 0.3)$ olarak verilmiştir. Rocchio parametreleri $\alpha = 1$, $\beta = 0.75$, $\gamma = 0.15$ için genişletilmiş sorgu vektörü q_m ’i hesaplayınız.

$$q_m = \alpha \cdot q_0 + \beta \cdot \frac{1}{|\bar{D}_r|} \sum_{d_j \in D_r} d_j - \gamma \cdot \frac{1}{|\bar{D}_{ir}|} \sum_{d_j \in D_{ir}} d_j \quad (74)$$

d) Hesapladığınız q_m vektörünü q_0 ile karşılaştırarak Rocchio geri bildiriminin ikinci tur erişimi nasıl iyileştireceğini tartışınız. Özellikle orijinalde sıfır olan üçüncü boyuttaki değişime dikkat ediniz.

Çözüm

Adım 1 – TP (Doğru Pozitif) Değerinin Hesaplanması:

Verilenler: Sistemin getirdiği doküman sayısı $|S| = 20$. Bunların 8’i ilgili, 12’si ilgisiz. Koleksiyondaki toplam ilgili doküman sayısı $|R| = 25$. Koleksiyon toplam büyüklüğü = 200.

Sistemin getirdiği ve gerçekten ilgili olan dokümanlar:

$$TP = |R \cap S| = 8 \quad (75)$$

Adım 2 – FP (Yanlış Pozitif) Değerinin Hesaplanması:

Sistemin getirdiği ama ilgisiz olan dokümanlar:

$$FP = |S| - TP = 20 - 8 = 12 \quad (76)$$

Adım 3 – FN (Yanlış Negatif) Değerinin Hesaplanması:

İlgili olduğu halde sistemin getirmediği dokümanlar:

$$FN = |R| - TP = 25 - 8 = 17 \quad (77)$$

Adım 4 – TN (Doğru Negatif) Değerinin Hesaplanması:

İlgisiz olan ve sistemin getirmediği dokümanlar. Koleksiyondaki toplam ilgisiz doküman sayısı $200 - 25 = 175$ ’tir. Bunların 12’si getirilmiştir (FP), kalanı getirilmemiştir:

$$TN = 175 - 12 = 163 \quad (78)$$

Adım 5 – Karmaşıklık Matrisinin Oluşturulması:

	Getirilen (Pozitif)	Getirilmeyen (Negatif)	Toplam
İlgili	TP = 8	FN = 17	25
İlgisiz	FP = 12	TN = 163	175
Toplam	20	180	200

Adım 6 – Kesinlik (Precision) Değerinin Hesaplanması:

$$\text{Kesinlik} = \frac{TP}{TP + FP} = \frac{8}{8 + 12} = \frac{8}{20} = 0.40 \text{ (\%40)} \quad (79)$$

Adım 7 – Geri Çağırma (Recall) Değerinin Hesaplanması:

$$\text{Geri Çağırma} = \frac{TP}{TP + FN} = \frac{8}{8 + 17} = \frac{8}{25} = 0.32 \text{ (\%32)} \quad (80)$$

Adım 8 – F1-Skorunun Hesaplanması:

$$F_1 = \frac{2 \cdot \text{Kesinlik} \cdot \text{Geri Çağırma}}{\text{Kesinlik} + \text{Geri Çağırma}} = \frac{2 \cdot 0.40 \cdot 0.32}{0.40 + 0.32} = \frac{0.256}{0.72} = 0.3556 \quad (81)$$

Adım 9 – Kesinlik ve Geri Çağırma Arasındaki Ters Orantının Yorumlanması:

Kesinlik ve geri çağırma arasındaki ters orantı bu örnekte net biçimde görülmektedir. Kesinlik %40 ile görece düşüktür; sistem getirdiği her 5 dokümandan yalnızca 2'sini doğru tahmin edebilmektedir. Geri çağırma ise %32 ile daha da düşüktür; ilgili dokümanların yalnızca üçte biri bulunabilmektedir.

Sistemi daha seçici hale getirip (örneğin benzerlik eşiğini yükselterek) daha az doküman getirirsek, FP azalacağı için kesinlik artar, ancak daha az ilgili doküman getirileceği için geri çağırma düşer. Tersine, daha fazla doküman getirirsek (eşığı düşürürsek), daha çok ilgili doküman yakalanacağı için geri çağırma artar, ancak beraberinde daha çok ilgisiz doküman da getirileceği için kesinlik düşer. F1-skoru 0.3556 ile her iki metriğin de iyileştirilmesi gerektiğini göstermektedir.

Adım 10 – Rocchio Genişletilmiş Sorgu Vektörü, $\alpha \cdot q_0$ Terimi:

Rocchio formülüne göre:

$$q_m = \alpha \cdot q_0 + \beta \cdot \bar{D}_r - \gamma \cdot \bar{D}_{ir} \quad (82)$$

İlk terim:

$$\alpha \cdot q_0 = 1 \cdot (0.5, 0.3, 0.0, 0.2) = (0.5, 0.3, 0.0, 0.2) \quad (83)$$

Adım 11 – Rocchio Genişletilmiş Sorgu Vektörü, $\beta \cdot \bar{D}_r$ Terimi:

$$\beta \cdot \bar{D}_r = 0.75 \cdot (0.8, 0.6, 0.4, 0.1) = (0.6, 0.45, 0.3, 0.075) \quad (84)$$

Adım 12 – Rocchio Genişletilmiş Sorgu Vektörü, $\gamma \cdot \bar{D}_{ir}$ Terimi:

$$\gamma \cdot \bar{D}_{ir} = 0.15 \cdot (0.1, 0.2, 0.9, 0.3) = (0.015, 0.03, 0.135, 0.045) \quad (85)$$

Adım 13 – Rocchio Genişletilmiş Sorgu Vektörü q_m 'in Hesaplanması:

$$q_m = (0.5 + 0.6 - 0.015, 0.3 + 0.45 - 0.03, 0.0 + 0.3 - 0.135, 0.2 + 0.075 - 0.045) \quad (86)$$

$$= (1.085, 0.720, 0.165, 0.230) \quad (87)$$

Genişletilmiş sorgu vektörü:

$$q_m = (1.085, 0.720, 0.165, 0.230) \quad (88)$$

Adım 14 – Birinci Boyuttaki Değişimin Analizi:

q_0 ile q_m karşılaştırıldığında birinci boyutta önemli bir değişim gözlemlenir ($0.5 \rightarrow 1.085$). Orijinal sorguda orta düzeyde ağırlığa sahip olan bu terim, ilgili dokümanlarda çok yüksek (0.8), ilgisiz dokümanlarda ise çok düşük (0.1) ağırlığa sahiptir. Rocchio, $\beta \cdot 0.8 = 0.6$ ekleyip $\gamma \cdot 0.1 = 0.015$ çıkararak bu terimin sorgudaki ağırlığını yaklaşık iki katına çıkarmıştır. Bu, ilgili dokümanlarda güçlü biçimde temsil edilen bir terimin sorguda daha baskın hale gelmesini sağlar.

Adım 15 – İkinci Boyuttaki Değişimin Analizi:

İkinci boyutta ($0.3 \rightarrow 0.720$): Benzer şekilde, ilgili dokümanlarda yüksek (0.6), ilgisizlerde düşük (0.2) olan bu terimin ağırlığı da önemli ölçüde artmıştır.

Adım 16 – Üçüncü Boyuttaki Kritik Değişimin Analizi:

En kritik değişim üçüncü boyutta gerçekleşmiştir ($0.0 \rightarrow 0.165$). Orijinal sorguda hiç bulunmayan (0.0) bu terim, ilgili dokümanlarda anlamlı bir ağırlığa sahipken (0.4), ilgisiz dokümanlarda çok yüksektir (0.9). Rocchio, $\beta \cdot 0.4 = 0.3$ pozitif katkı ile $\gamma \cdot 0.9 = 0.135$ negatif katkıyı dengelemiş ve net 0.165 ağırlığıyla bu terimi sorguya eklemiştir.

Bu, Rocchio'nun en güçlü özelliğidir: kullanıcının orijinal sorgusunda ifade etmediği, ancak ilgili dokümanlarda sıkça geçen terimler otomatik olarak sorguya dahil edilir. Aynı zamanda ilgisiz dokümanlarda sık geçen terimler (üçüncü boyutun ilgisiz centroid'de 0.9 olması gibi) negatif katkıyla bastırılır.

Adım 17 – Dördüncü Boyuttaki Değişimin Analizi:

Dördüncü boyutta ($0.2 \rightarrow 0.230$): Bu terim ilgili ve ilgisiz dokümanlarda benzer düzeyde olduğu için (0.1'e karşı 0.3), Rocchio bunun ağırlığında büyük bir değişiklik yapmamıştır.

Adım 18 – Rocchio Geri Bildiriminin Genel Etkisinin Değerlendirilmesi:

q_m ile yapılacak ikinci tur erişimde, genişletilmiş sorgu hem orijinal terimleri güçlendirecek hem de yeni ayırt edici terimleri içerecek, ilgisiz dokümanlarla ilişkili terimleri ise bastıracaktır. Bu sayede ikinci turda hem kesinlik hem de geri çağırma değerlerinin artması beklenir.

$\alpha = 1$ seçimi, orijinal sorgunun tamamen korunmasını ve kullanıcının niyetinin kaybolmamasını sağlar. $\beta > \gamma$ (0.75 i 0.15) seçimi ise pozitif geri bildirimin negatif geri bildirimden daha güvenilir olduğu varsayımına dayanır; kullanıcının "ilgili" işaretlemesi "ilgisiz" işaretlemesinden genellikle daha tutarlıdır.

3 Web Bağlantı Analizi, PageRank ve Merkezilik Ölçütleri — Sorular

Soru 1: PageRank Akış Denklemleri

Aşağıda verilen üç sayfalı (A, B, C) Web grafi için PageRank değerlerini akış denklemleri yöntemiyle hesaplayınız. Grafın bağlantı yapısı şöyledir:

A sayfası B ve C'ye bağlantı vermektedir (giden bağlantı sayısı $d_A = 2$).

B sayfası yalnızca C'ye bağlantı vermektedir (giden bağlantı sayısı $d_B = 1$).

C sayfası A ve B'ye bağlantı vermektedir (giden bağlantı sayısı $d_C = 2$).

Tüm PageRank değerlerinin toplamının 1 olduğu ($r_A + r_B + r_C = 1$) kısıtını kullanarak r_A , r_B ve r_C değerlerini bulunuz. Her adımı açıkça gösteriniz.

Çözüm:

Adım 1: PageRank akış denklemi, bir sayfanın PageRank değerinin kendisine bağlantı veren tüm sayfaların PageRank değerlerinin, o sayfaların giden bağlantı sayılarına bölünmüş toplamı olduğunu söyler:

$$r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i} \quad (89)$$

Adım 2: Verilen graf için akış denklemlerini her düğüm için ayrı ayrı yazalım.

A sayfasına yalnızca C'den bağlantı gelmektedir ve C'nin 2 giden bağlantısı vardır. B sayfasına A'dan (A'nın 2 giden bağlantısından biri) ve C'den (C'nin 2 giden bağlantısından biri) bağlantı gelmektedir. C sayfasına ise A'dan (A'nın 2 giden bağlantısından biri) ve B'den (B'nin tek giden bağlantısı) bağlantı gelmektedir. Böylece üç denklem aşağıdaki gibi yazılır:

$$r_A = \frac{r_C}{2} \quad (90)$$

$$r_B = \frac{r_A}{2} + \frac{r_C}{2} \quad (91)$$

$$r_C = \frac{r_A}{2} + r_B \quad (92)$$

Adım 3: Birinci denklemden $r_A = r_C/2$ olduğu görülür. Bu ifadeyi ikinci denklemde yerine koyalım:

$$r_B = \frac{r_C/2}{2} + \frac{r_C}{2} = \frac{r_C}{4} + \frac{r_C}{2} = \frac{r_C + 2r_C}{4} = \frac{3r_C}{4} \quad (93)$$

Adım 4: Üçüncü denklemde r_A ve r_B ifadelerini yerine koyalım:

$$r_C = \frac{r_C/2}{2} + \frac{3r_C}{4} = \frac{r_C}{4} + \frac{3r_C}{4} = r_C \quad (94)$$

Bu denklem özdeş olarak sağlanmaktadır; bu durum denklem sisteminin bir ölçek faktörüne kadar çözüme sahip olduğunu gösterir.

Adım 5: Tek bir çözüm elde etmek için normalizasyon kısıtı $r_A + r_B + r_C = 1$ uygulanır. Tüm değişkenleri r_C cinsinden ifade edelim: $r_A = r_C/2$ ve $r_B = 3r_C/4$.

Toplam kısıtı:

$$\frac{r_C}{2} + \frac{3r_C}{4} + r_C = 1 \quad (95)$$

$$\frac{2r_C}{4} + \frac{3r_C}{4} + \frac{4r_C}{4} = \frac{9r_C}{4} = 1 \quad (96)$$

$$r_C = \frac{4}{9} \quad (97)$$

Buradan $r_A = r_C/2 = 2/9$ ve $r_B = 3r_C/4 = 3/9 = 1/3$ bulunur.

Adım 6: Sonuç olarak PageRank vektörü $r = [2/9, 3/9, 4/9]^T$ olur. Doğrulama: $2/9 + 3/9 + 4/9 = 9/9 = 1$.

C sayfası en yüksek PageRank değerine ($4/9$) sahiptir; bunun nedeni hem A hem de B tarafından işaret edilmesidir.

B sayfası ikinci sıradadır ($3/9$); A ve C'den bağlantı almaktadır.

A sayfası yalnızca C'den bağlantı aldığı için en düşük değere ($2/9$) sahiptir.

Soru 2: Güç İterasyonu (Power Iteration)

Aşağıda bağlantı yapısı verilen üç sayfalı (X, Y, Z) Web grafi için güç iterasyonu yöntemini kullanarak PageRank değerlerini hesaplayınız. İlk beş iterasyonu ($t = 0$ 'dan $t = 5$ 'e kadar) adım adım gösteriniz ve her iterasyondaki vektörü açıkça yazınız. Yakınsamanın hangi değere doğru olduğunu belirtiniz.

Bağlantı yapısı: X sayfası yalnızca Y'ye bağlantı vermektedir ($d_X = 1$). Y sayfası X ve Z'ye bağlantı vermektedir ($d_Y = 2$). Z sayfası yalnızca X'e bağlantı vermektedir ($d_Z = 1$). Başlangıç vektörü olarak $r^{(0)} = [1/3, 1/3, 1/3]^T$ kullanınız.

Çözüm:

Adım 1: Önce stokastik bitişiklik matrisi M 'yi oluşturalım. $M_{ji} = 1/d_i$ eğer $i \rightarrow j$ varsa, aksi halde 0:

$$M = \begin{bmatrix} 0 & 1/2 & 1 \\ 1 & 0 & 0 \\ 0 & 1/2 & 0 \end{bmatrix} \quad (98)$$

Sütun toplamlarının 1 olduğu kontrol edilebilir: birinci sütun $0 + 1 + 0 = 1$, ikinci sütun $1/2 + 0 + 1/2 = 1$, üçüncü sütun $1 + 0 + 0 = 1$.

Adım 2: Başlangıç vektörü $r^{(0)} = [1/3, 1/3, 1/3]^T$ olarak verilmiştir.

Adım 3 – İterasyon 1: $r^{(1)} = M \cdot r^{(0)}$

$$r_X^{(1)} = 0 \cdot \frac{1}{3} + \frac{1}{2} \cdot \frac{1}{3} + 1 \cdot \frac{1}{3} = 0 + \frac{1}{6} + \frac{1}{3} = \frac{1}{6} + \frac{2}{6} = \frac{3}{6} = \frac{1}{2} \quad (99)$$

$$r_Y^{(1)} = 1 \cdot \frac{1}{3} + 0 \cdot \frac{1}{3} + 0 \cdot \frac{1}{3} = \frac{1}{3} = \frac{2}{6} \quad (100)$$

$$r_Z^{(1)} = 0 \cdot \frac{1}{3} + \frac{1}{2} \cdot \frac{1}{3} + 0 \cdot \frac{1}{3} = \frac{1}{6} \quad (101)$$

$$r^{(1)} = [3/6, 2/6, 1/6]^T = [1/2, 1/3, 1/6]^T.$$

Adım 4 – İterasyon 2: $r^{(2)} = M \cdot r^{(1)}$

$$r_X^{(2)} = 0 \cdot \frac{3}{6} + \frac{1}{2} \cdot \frac{2}{6} + 1 \cdot \frac{1}{6} = 0 + \frac{2}{12} + \frac{2}{12} = \frac{4}{12} = \frac{1}{3} = \frac{2}{6} \quad (102)$$

$$r_Y^{(2)} = 1 \cdot \frac{3}{6} + 0 + 0 = \frac{3}{6} = \frac{1}{2} \quad (103)$$

$$r_Z^{(2)} = 0 + \frac{1}{2} \cdot \frac{2}{6} + 0 = \frac{2}{12} = \frac{1}{6} \quad (104)$$

$$r^{(2)} = [2/6, 3/6, 1/6]^T.$$

Adım 5 – İterasyon 3: $r^{(3)} = M \cdot r^{(2)}$

$$r_X^{(3)} = \frac{1}{2} \cdot \frac{3}{6} + 1 \cdot \frac{1}{6} = \frac{3}{12} + \frac{2}{12} = \frac{5}{12} \quad (105)$$

$$r_Y^{(3)} = 1 \cdot \frac{2}{6} = \frac{2}{6} = \frac{4}{12} \quad (106)$$

$$r_Z^{(3)} = \frac{1}{2} \cdot \frac{3}{6} = \frac{3}{12} \quad (107)$$

$$r^{(3)} = [5/12, 4/12, 3/12]^T.$$

Adım 6 – İterasyon 4: $r^{(4)} = M \cdot r^{(3)}$

$$r_X^{(4)} = \frac{1}{2} \cdot \frac{4}{12} + 1 \cdot \frac{3}{12} = \frac{4}{24} + \frac{6}{24} = \frac{10}{24} = \frac{5}{12} \quad (108)$$

$$r_Y^{(4)} = 1 \cdot \frac{5}{12} = \frac{5}{12} = \frac{10}{24} \quad (109)$$

$$r_Z^{(4)} = \frac{1}{2} \cdot \frac{4}{12} = \frac{4}{24} = \frac{2}{12} \quad (110)$$

$$r^{(4)} = [5/12, 5/12, 2/12]^T.$$

Adım 7 – İterasyon 5: $r^{(5)} = M \cdot r^{(4)}$

$$r_X^{(5)} = \frac{1}{2} \cdot \frac{5}{12} + 1 \cdot \frac{2}{12} = \frac{5}{24} + \frac{4}{24} = \frac{9}{24} = \frac{3}{8} \quad (111)$$

$$r_Y^{(5)} = 1 \cdot \frac{5}{12} = \frac{5}{12} = \frac{10}{24} \quad (112)$$

$$r_Z^{(5)} = \frac{1}{2} \cdot \frac{5}{12} = \frac{5}{24} \quad (113)$$

$$r^{(5)} = [9/24, 10/24, 5/24]^T = [3/8, 5/12, 5/24]^T.$$

Adım 8 – Yakınsama Analizi: $r^{(4)}$ ve $r^{(5)}$ arasında hâlâ bir salınım gözlenmektedir. İterasyonlara devam edildiğinde vektörün $[2/5, 2/5, 1/5]^T = [0.4, 0.4, 0.2]^T$ değerine yakınsadığı görülür. Bu değer, akış denklemlerinin çözümüdür.

Doğrulamak gerekirse, M matrisi ve $r = [2/5, 2/5, 1/5]^T$ için $M \cdot r$ çarpımı yapıldığında yine $[2/5, 2/5, 1/5]^T$ elde edildiği gösterilebilir.

Güç iterasyonu, Web ölçeğindeki graflar için PageRank hesaplamasının temel yöntemidir; $|r^{(t+1)} - r^{(t)}|_1 < \varepsilon$ (tipik olarak $\varepsilon = 10^{-6}$) koşulu sağlandığında iterasyon sonlandırılır.

Soru 3: Örümcek Tuzacı ve Google Matrisi

Aşağıda verilen üç sayfalı (y, a, m) Web grafını ele alalım:

y sayfası y ve a'ya bağlantı vermektedir ($d_y = 2$).

a sayfası y ve m'ye bağlantı vermektedir ($d_a = 2$).

m sayfası yalnızca kendisine bağlantı vermektedir ($d_m = 1$), yani bir örümcek tuzacıdır (spider trap).

Bu graf için aşağıdakileri yapınız:

a) Stokastik bitişiklik matrisi M 'yi oluşturunuz ve güç iterasyonunun ilk 3 adımını $r^{(0)} = [1/3, 1/3, 1/3]^T$ başlangıç vektörü ile hesaplayarak örümcek tuzacının PageRank değerlerini nasıl etkilediğini gösteriniz.

b) Sönümleme faktörü $\beta = 0.8$ kullanarak Google matrisi $A = \beta M + (1 - \beta) \frac{1}{n} ee^T$ 'yi oluşturunuz. Matris elemanlarını kesir olarak yazınız.

c) Google matrisi A 'yı kullanarak güç iterasyonunun ilk 2 adımını hesaplayınız ve örümcek tuzağı sorununun nasıl çözüldüğünü açıklayınız.

Çözüm:

a) Örümcek Tuzağı Etkisi

Adım 1: Stokastik bitişiklik matrisi M aşağıdaki gibidir. m sayfası yalnızca kendisine bağlantı verdiği için $M_{mm} = 1/d_m = 1$ 'dir:

$$M = \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 0 \\ 0 & 1/2 & 1 \end{bmatrix} \quad (114)$$

Adım 2: Başlangıç vektörü $r^{(0)} = [1/3, 1/3, 1/3]^T$ ile güç iterasyonuna başlayalım.

Adım 3 – İterasyon 1: $r^{(1)} = M \cdot r^{(0)}$

$$r_y^{(1)} = \frac{1}{2} \cdot \frac{1}{3} + \frac{1}{2} \cdot \frac{1}{3} + 0 = \frac{1}{6} + \frac{1}{6} = \frac{1}{3}$$

$$r_a^{(1)} = \frac{1}{2} \cdot \frac{1}{3} + 0 + 0 = \frac{1}{6}$$

$$r_m^{(1)} = 0 + \frac{1}{2} \cdot \frac{1}{3} + 1 \cdot \frac{1}{3} = \frac{1}{6} + \frac{2}{6} = \frac{1}{2}$$

$$r^{(1)} = [1/3, 1/6, 1/2]^T.$$

Adım 4 – İterasyon 2: $r^{(2)} = M \cdot r^{(1)}$

$$r_y^{(2)} = \frac{1}{2} \cdot \frac{1}{3} + \frac{1}{2} \cdot \frac{1}{6} = \frac{2}{12} + \frac{1}{12} = \frac{3}{12}$$

$$r_a^{(2)} = \frac{1}{2} \cdot \frac{1}{3} = \frac{2}{12}$$

$$r_m^{(2)} = \frac{1}{2} \cdot \frac{1}{6} + 1 \cdot \frac{1}{2} = \frac{1}{12} + \frac{6}{12} = \frac{7}{12}$$

$$r^{(2)} = [3/12, 2/12, 7/12]^T.$$

Adım 5 – İterasyon 3: $r^{(3)} = M \cdot r^{(2)}$

$$r_y^{(3)} = \frac{1}{2} \cdot \frac{3}{12} + \frac{1}{2} \cdot \frac{2}{12} = \frac{5}{24}$$

$$r_a^{(3)} = \frac{1}{2} \cdot \frac{3}{12} = \frac{3}{24}$$

$$r_m^{(3)} = \frac{1}{2} \cdot \frac{2}{12} + 1 \cdot \frac{7}{12} = \frac{1}{12} + \frac{7}{12} = \frac{16}{24}$$

$$r^{(3)} = [5/24, 3/24, 16/24]^T.$$

Adım 6 – Örümcek Tuzağı Etkisinin Yorumu: Görüldüğü gibi m sayfasının PageRank değeri her iterasyonda artmakta ($1/2 \rightarrow 7/12 \rightarrow 16/24$), y ve a sayfalarının değerleri ise azalmaktadır. İterasyonlar devam ettikçe $r_m \rightarrow 1$ ve $r_y, r_a \rightarrow 0$ olur. Örümcek tuzağı tüm PageRank skorunu kendi üzerinde biriktirir.

b) Google Matrisinin Oluşturulması

Adım 1: $n = 3$, $\beta = 0.8 = 4/5$, $(1 - \beta) = 1/5$. Her bir sayfaya eşit dağıtılan teleport katkısı $(1 - \beta)/n = 1/15$ 'tir.

$$A = \beta M + (1 - \beta) \frac{1}{n} ee^T = \frac{4}{5} M + \frac{1}{15} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad (115)$$

Adım 2: $\frac{4}{5} M$ matrisini hesaplayalım:

$$\frac{4}{5} M = \begin{bmatrix} 4/5 \cdot 1/2 & 4/5 \cdot 1/2 & 4/5 \cdot 0 \\ 4/5 \cdot 1/2 & 4/5 \cdot 0 & 4/5 \cdot 0 \\ 4/5 \cdot 0 & 4/5 \cdot 1/2 & 4/5 \cdot 1 \end{bmatrix} = \begin{bmatrix} 6/15 & 6/15 & 0 \\ 6/15 & 0 & 0 \\ 0 & 6/15 & 12/15 \end{bmatrix} \quad (116)$$

Adım 3: Her elemana $1/15$ teleport katkısı eklenir:

$$A = \begin{bmatrix} 6/15 + 1/15 & 6/15 + 1/15 & 0 + 1/15 \\ 6/15 + 1/15 & 0 + 1/15 & 0 + 1/15 \\ 0 + 1/15 & 6/15 + 1/15 & 12/15 + 1/15 \end{bmatrix} = \begin{bmatrix} 7/15 & 7/15 & 1/15 \\ 7/15 & 1/15 & 1/15 \\ 1/15 & 7/15 & 13/15 \end{bmatrix} \quad (117)$$

Sütun toplamaları kontrol edildiğinde her birinin $15/15 = 1$ olduğu görülür; A sütun-stokastiktir. Ayrıca tüm elemanlar pozitif olduğu için A indirgenemez ve periyodik değildir.

c) Google Matrisi ile Güç İterasyonu

Adım 1: $r^{(0)} = [1/3, 1/3, 1/3]^T$ ile başlayalım.

Adım 2 – İterasyon 1: $r^{(1)} = A \cdot r^{(0)}$

Her satır için A 'nın ilgili satırı ile $r^{(0)}$ 'ın iç çarpımı:

$$\begin{aligned} r_y^{(1)} &= \frac{7}{15} \cdot \frac{1}{3} + \frac{7}{15} \cdot \frac{1}{3} + \frac{1}{15} \cdot \frac{1}{3} = \frac{7+7+1}{15 \cdot 3} = \frac{15}{45} = \frac{1}{3} \\ r_a^{(1)} &= \frac{7}{15} \cdot \frac{1}{3} + \frac{1}{15} \cdot \frac{1}{3} + \frac{1}{15} \cdot \frac{1}{3} = \frac{7+1+1}{45} = \frac{9}{45} = \frac{1}{5} \\ r_m^{(1)} &= \frac{1}{15} \cdot \frac{1}{3} + \frac{7}{15} \cdot \frac{1}{3} + \frac{13}{15} \cdot \frac{1}{3} = \frac{1+7+13}{45} = \frac{21}{45} = \frac{7}{15} \\ r^{(1)} &= [1/3, 1/5, 7/15]^T. \end{aligned}$$

Adım 3 – İterasyon 2: $r^{(2)} = A \cdot r^{(1)}$

$$\begin{aligned} r_y^{(2)} &= \frac{7}{15} \cdot \frac{1}{3} + \frac{7}{15} \cdot \frac{1}{5} + \frac{1}{15} \cdot \frac{7}{15} = \frac{7}{45} + \frac{7}{75} + \frac{7}{225} \\ \text{Ortak payda 225: } &\frac{35}{225} + \frac{21}{225} + \frac{7}{225} = \frac{63}{225} = \frac{7}{25} = 0.28 \\ r_a^{(2)} &= \frac{7}{15} \cdot \frac{1}{3} + \frac{1}{15} \cdot \frac{1}{5} + \frac{1}{15} \cdot \frac{7}{15} = \frac{7}{45} + \frac{1}{75} + \frac{7}{225} = \frac{35}{225} + \frac{3}{225} + \frac{7}{225} = \frac{45}{225} = \frac{1}{5} = 0.20 \\ r_m^{(2)} &= \frac{1}{15} \cdot \frac{1}{3} + \frac{7}{15} \cdot \frac{1}{5} + \frac{13}{15} \cdot \frac{7}{15} = \frac{1}{45} + \frac{7}{75} + \frac{91}{225} = \frac{5}{225} + \frac{21}{225} + \frac{91}{225} = \frac{117}{225} = \frac{13}{25} = 0.52 \\ r^{(2)} &= [7/25, 5/25, 13/25]^T = [0.28, 0.20, 0.52]^T. \end{aligned}$$

Adım 4 – Örümcek Tuzağı Sorununun Çözümü: Teleport mekanizması sayesinde, rastgele gezgin her adımda $1 - \beta = 0.2$ olasılıkla rastgele bir sayfaya atlar. Bu sayede m sayfası tüm skoru ememez; y ve a sayfaları da teleport yoluyla sabit bir katkı almaya devam eder.

Kararlı durumda PageRank vektörü yaklaşık olarak $[7/33, 5/33, 21/33]^T \approx [0.212, 0.152, 0.636]^T$ değerine yakınsar. m hâlâ en yüksek PageRank'e sahiptir (kendine döngü ve teleportun birleşik etkisiyle), ancak y ve a 'nın skorları sıfıra gitmez.

Google matrisi A 'nın tüm elemanları pozitif olduğu için Markov zinciri indirgenemez ve periyodik değildir; dolayısıyla güç iterasyonu tek ve benzersiz bir durağan dağılıma yakınsar.

Soru 4: Derece, Arasındalık ve Yakınlık Merkeziliği

Aşağıda kenar listesi verilen 5 düğümlü (A, B, C, D, E) yönlendirilmemiş sosyal ağ grafi için her bir düğümün derece merkeziliğini (degree centrality), arasındalık merkeziliğini (betweenness centrality) ve yakınlık merkeziliğini (closeness centrality) hesaplayınız. Tüm merkezilik değerlerini normalize edilmiş olarak veriniz.

Graftaki kenarlar: $A-B, B-C, C-D, B-E, C-E$.

Çözüm:

Adım 1: Önce grafin yapısını netleştirelim. Beş düğüm ve beş kenar vardır. A yalnızca B 'ye bağlıdır. B ; A, C ve E 'ye bağlıdır. C ; B, D ve E 'ye bağlıdır. D yalnızca C 'ye bağlıdır. E ; B ve C 'ye bağlıdır.

Derece Merkeziliği

Adım 2: Yönlendirilmemiş grafta bir düğümün derecesi, doğrudan bağlı olduğu komşu sayısıdır.

A : 1 (B)

B : 3 (A, C, E)

C: 3 (B, D, E)

D: 1 (C)

E: 2 (B, C)

Adım 3: Normalleştirme için $N - 1 = 4$ ile bölünür:

$$C'_D(A) = 1/4 = 0.25$$

$$C'_D(B) = 3/4 = 0.75$$

$$C'_D(C) = 3/4 = 0.75$$

$$C'_D(D) = 1/4 = 0.25$$

$$C'_D(E) = 2/4 = 0.50$$

B ve C en yüksek derece merkeziliğine sahiptir. A ve D ise en düşük dereceye sahip olup grafın uç noktalarıdır.

Arasındalık Merkeziliği

Adım 4: Arasındalık merkeziliği, bir düğümün diğer düğüm çiftleri arasındaki en kısa yollar üzerinde bulunma sıklığını ölçer. $N = 5$ için toplam $\binom{5}{2} = 10$ düğüm çifti vardır. Her çift için en kısa yolları (shortest paths, SP) belirleyelim.

Adım 5: Tüm düğüm çiftleri için en kısa yollar:

(A,B): A-B (uzunluk 1). Ara düğüm yok.

(A,C): A-B-C (uzunluk 2). Alternatif A-B-E-C (uzunluk 3) en kısa değildir. Tek SP A-B-C'dir; B ara düğümdür.

(A,D): A-B-C-D (uzunluk 3). Tek SP; B ve C ara düğümdür.

(A,E): A-B-E (uzunluk 2). Alternatif A-B-C-E (uzunluk 3) en kısa değildir. Tek SP A-B-E'dir; B ara düğümdür.

(B,C): B-C (uzunluk 1). Ayrıca B-E-C (uzunluk 2) vardır ancak daha uzundur. Tek SP B-C'dir; ara düğüm yok.

(B,D): B-C-D (uzunluk 2). Tek SP; C ara düğümdür.

(B,E): B-E (uzunluk 1). Tek SP; ara düğüm yok.

(C,D): C-D (uzunluk 1). Tek SP; ara düğüm yok.

(C,E): C-E (uzunluk 1). Tek SP; ara düğüm yok.

(D,E): D-C-E (uzunluk 2). Tek SP; C ara düğümdür.

Adım 6: Şimdi her düğümün kaç en kısa yol üzerinde bulunduğunu sayalım (her çift için $p_{jk}(i)/p_{jk}$ oranı toplanır; tüm SP'ler tek olduğu için oran ya 1 ya da 0'dır):

A düğümü: Hiçbir çiftin en kısa yolu üzerinde değildir (tüm çiftlerde A ya uç noktadır ya da yol A'dan geçmez). $C_B(A) = 0$.

B düğümü: (A,C) için 1, (A,D) için 1, (A,E) için 1. Toplam $C_B(B) = 3$.

C düğümü: (A,D) için 1, (B,D) için 1, (D,E) için 1. Toplam $C_B(C) = 3$.

D düğümü: Hiçbir çiftin en kısa yolu üzerinde değildir. $C_B(D) = 0$.

E düğümü: Hiçbir çiftin en kısa yolu üzerinde değildir (E yalnızca uç nokta olduğu çiftlerde yer alır, ara düğüm olduğu hiçbir çift yoktur). $C_B(E) = 0$.

Adım 7: Normalleştirme: Maksimum olası arasındalık değeri $(N - 1)(N - 2)/2 = 4 \cdot 3/2 = 6$ 'dır.

$$C'_B(A) = 0/6 = 0$$

$$C'_B(B) = 3/6 = 0.50$$

$$C'_B(C) = 3/6 = 0.50$$

$$C'_B(D) = 0$$

$$C'_B(E) = 0$$

B ve C, grafın iki yakası (A ile D) arasındaki tüm trafiği kontrol etmektedir. Derece merkeziliği yüksek olan E'nin arasındalık merkeziliğinin sıfır olması dikkat çekicidir; bu, derece merkeziliğinin aracılık rolünü yakalayamadığının iyi bir örneğidir.

Yakınlık Merkeziliği

Adım 8: Her düğümün diğer tüm düğümlere olan en kısa yol uzaklıkları toplamını hesaplayalım.

A düğümü: $d(A, B) = 1, d(A, C) = 2, d(A, D) = 3, d(A, E) = 2$. Toplam = $1 + 2 + 3 + 2 = 8$.

B düğümü: $d(B, A) = 1, d(B, C) = 1, d(B, D) = 2, d(B, E) = 1$. Toplam = $1 + 1 + 2 + 1 = 5$.

C düğümü: $d(C, A) = 2, d(C, B) = 1, d(C, D) = 1, d(C, E) = 1$. Toplam = $2 + 1 + 1 + 1 = 5$.

D düğümü: $d(D, A) = 3, d(D, B) = 2, d(D, C) = 1, d(D, E) = 2$. Toplam = $3 + 2 + 1 + 2 = 8$.

E düğümü: $d(E, A) = 2, d(E, B) = 1, d(E, C) = 1, d(E, D) = 2$. Toplam = $2 + 1 + 1 + 2 = 6$.

Adım 9: Normalleştirilmemiş yakınlık merkeziliği $C_C(i) = 1 / \sum_{j \neq i} d(i, j)$:

$$C_C(A) = 1/8 = 0.125$$

$$C_C(B) = 1/5 = 0.200$$

$$C_C(C) = 1/5 = 0.200$$

$$C_C(D) = 1/8 = 0.125$$

$$C_C(E) = 1/6 \approx 0.167$$

Adım 10: Normalleştirilmiş yakınlık merkeziliği $C'_C(i) = (N-1) \cdot C_C(i)$ yani $(N-1) / \sum d(i, j)$:

$$C'_C(A) = 4/8 = 0.500$$

$$C'_C(B) = 4/5 = 0.800$$

$$C'_C(C) = 4/5 = 0.800$$

$$C'_C(D) = 4/8 = 0.500$$

$$C'_C(E) = 4/6 \approx 0.667$$

B ve C en yüksek yakınlık merkeziliğine sahiptir; grafın merkezinde yer alırlar ve diğer düğümlere ortalama en kısa mesafededirler. A ve D en düşük yakınlık merkeziliğine sahiptir; grafın uçlarındadırlar ve diğer düğümlere erişimleri daha uzun sürer. E ise orta düzeydedir.

Soru 5: HITS Algoritması ile Otorite ve Göbek Hesaplama

Aşağıda bağlantı yapısı verilen dört sayfalı (1, 2, 3, 4) yönlendirilmiş Web grafı için HITS algoritmasını kullanarak otorite (authority) ve göbek (hub) skorlarını hesaplayınız. Güç iterasyonunun ilk 3 adımını açıkça gösteriniz. Her iterasyonda vektörleri maksimum değerlerine göre normalize ediniz.

Bağlantı yapısı:

Sayfa 1; sayfa 2 ve sayfa 3'e bağlantı vermektedir.

Sayfa 2; yalnızca sayfa 3'e bağlantı vermektedir.

Sayfa 3; yalnızca sayfa 1'e bağlantı vermektedir.

Sayfa 4; sayfa 2 ve sayfa 3'e bağlantı vermektedir.

Çözüm:

Adım 1: Önce bitişiklik matrisi L 'yi oluşturalım. $L_{ij} = 1$ eğer i sayfası j sayfasına bağlantı veriyorsa:

$$L = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix} \quad (118)$$

L^T transpozu:

$$L^T = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (119)$$

Adım 2: HITS algoritmasında otorite vektörü a ve göbek vektörü h karşılıklı olarak güncellenir:

$$a^{(t+1)} = L^T \cdot h^{(t)}, \quad h^{(t+1)} = L \cdot a^{(t+1)} \quad (120)$$

Başlangıç değerleri: $h^{(0)} = [1, 1, 1, 1]^T$.

Adım 3 – İterasyon 1:

Otorite skorları: $a^{(1)} = L^T \cdot h^{(0)}$

$$a_1^{(1)} = 0 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 = 1 \quad (121)$$

$$a_2^{(1)} = 1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 = 2 \quad (122)$$

$$a_3^{(1)} = 1 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 = 3 \quad (123)$$

$$a_4^{(1)} = 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 0 \quad (124)$$

$a^{(1)} = [1, 2, 3, 0]^T$. Maksimum değer 3 ile normalize edelim: $a_{\text{norm}}^{(1)} = [1/3, 2/3, 1, 0]^T$.

Göbek skorları: $h^{(1)} = L \cdot a^{(1)}$

$$h_1^{(1)} = 0 \cdot 1 + 1 \cdot 2 + 1 \cdot 3 + 0 \cdot 0 = 0 + 2 + 3 + 0 = 5 \quad (125)$$

$$h_2^{(1)} = 0 \cdot 1 + 0 \cdot 2 + 1 \cdot 3 + 0 \cdot 0 = 3 \quad (126)$$

$$h_3^{(1)} = 1 \cdot 1 + 0 \cdot 2 + 0 \cdot 3 + 0 \cdot 0 = 1 \quad (127)$$

$$h_4^{(1)} = 0 \cdot 1 + 1 \cdot 2 + 1 \cdot 3 + 0 \cdot 0 = 5 \quad (128)$$

$h^{(1)} = [5, 3, 1, 5]^T$. Normalize (5'e böl): $h_{\text{norm}}^{(1)} = [1, 3/5, 1/5, 1]^T$.

Adım 4 – İterasyon 2:

Otorite skorları: $a^{(2)} = L^T \cdot h^{(1)}$

$$a_1^{(2)} = 0 \cdot 5 + 0 \cdot 3 + 1 \cdot 1 + 0 \cdot 5 = 1 \quad (129)$$

$$a_2^{(2)} = 1 \cdot 5 + 0 \cdot 3 + 0 \cdot 1 + 1 \cdot 5 = 10 \quad (130)$$

$$a_3^{(2)} = 1 \cdot 5 + 1 \cdot 3 + 0 \cdot 1 + 1 \cdot 5 = 13 \quad (131)$$

$$a_4^{(2)} = 0 \quad (132)$$

$a^{(2)} = [1, 10, 13, 0]^T$. Normalize (13'e böl): $a_{\text{norm}}^{(2)} = [1/13, 10/13, 1, 0]^T \approx [0.077, 0.769, 1, 0]^T$.

Göbek skorları: $h^{(2)} = L \cdot a^{(2)}$

$$h_1^{(2)} = 0 \cdot 1 + 1 \cdot 10 + 1 \cdot 13 + 0 \cdot 0 = 23 \quad (133)$$

$$h_2^{(2)} = 0 \cdot 1 + 0 \cdot 10 + 1 \cdot 13 + 0 \cdot 0 = 13 \quad (134)$$

$$h_3^{(2)} = 1 \cdot 1 + 0 \cdot 10 + 0 \cdot 13 + 0 \cdot 0 = 1 \quad (135)$$

$$h_4^{(2)} = 0 \cdot 1 + 1 \cdot 10 + 1 \cdot 13 + 0 \cdot 0 = 23 \quad (136)$$

$h^{(2)} = [23, 13, 1, 23]^T$. Normalize (23'e böl): $h_{\text{norm}}^{(2)} = [1, 13/23, 1/23, 1]^T \approx [1, 0.565, 0.043, 1]^T$.

Adım 5 – İterasyon 3:

Otorite skorları: $a^{(3)} = L^T \cdot h^{(2)}$

$$a_1^{(3)} = 1 \cdot 1 = 1 \quad (137)$$

$$a_2^{(3)} = 1 \cdot 23 + 1 \cdot 23 = 46 \quad (138)$$

$$a_3^{(3)} = 1 \cdot 23 + 1 \cdot 13 + 1 \cdot 23 = 59 \quad (139)$$

$$a_4^{(3)} = 0 \quad (140)$$

$a^{(3)} = [1, 46, 59, 0]^T$. Normalize (59'a böl): $a_{\text{norm}}^{(3)} = [1/59, 46/59, 1, 0]^T \approx [0.017, 0.780, 1, 0]^T$.

Göbek skorları: $h^{(3)} = L \cdot a^{(3)}$

$$h_1^{(3)} = 1 \cdot 46 + 1 \cdot 59 = 105 \quad (141)$$

$$h_2^{(3)} = 1 \cdot 59 = 59 \quad (142)$$

$$h_3^{(3)} = 1 \cdot 1 = 1 \quad (143)$$

$$h_4^{(3)} = 1 \cdot 46 + 1 \cdot 59 = 105 \quad (144)$$

$h^{(3)} = [105, 59, 1, 105]^T$. Normalize (105'e böl): $h_{\text{norm}}^{(3)} = [1, 59/105 \approx 0.562, 1/105 \approx 0.010, 1]^T$.

Adım 6 – Sonuçların Yorumlanması:

İterasyonlar ilerledikçe skorlar kararlı bir sıralamaya yakınsamaktadır.

Otorite skorlarına göre sıralama: Sayfa 3 en yüksek otoritedir (sayfa 1, 2 ve 4 tarafından işaret edilmektedir), ardından sayfa 2 gelir (sayfa 1 ve 4 tarafından işaret edilmektedir), sayfa 1 düşük otoriteye sahiptir (yalnızca sayfa 3 tarafından işaret edilmektedir), sayfa 4'ün otoritesi sıfırdır (hiçbir sayfa tarafından işaret edilmemektedir).

Göbek skorlarına göre sıralama: Sayfa 1 ve sayfa 4 en yüksek göbek skoruna sahiptir (her ikisi de ikişer sayfaya bağlantı vermektedir ve bu sayfalar yüksek otoriteye sahiptir), sayfa 2 orta düzeydedir, sayfa 3 ise en düşük göbek skoruna sahiptir (yalnızca sayfa 1'e bağlantı vermektedir).

Otorite ve göbek skorları arasındaki karşılıklı pekiştirme ilişkisi bu örnekte açıkça görülmektedir: İyi göbekler (sayfa 1 ve 4) iyi otoritelere bağlantı vermekte, iyi otoriteler (sayfa 3) ise iyi göbekler tarafından işaret edilmektedir. Sayfa 4'ün hiçbir gelen bağlantısı olmadığı için otoritesi sıfırdır; ancak giden bağlantıları sayesinde yüksek göbek skoruna ulaşmıştır. Bu durum, HITS'in iki rolü nasıl ayırt ettiğinin iyi bir örneğidir.

4 Web Crawler Mimarisi ve Bloom Filtreleri – Sınav Soruları

Soru 1: URL Kanonikleştirme

Bir web crawler'ın tarama sırasında aynı sayfaya işaret eden aşağıdaki URL'lerle karşılaştığını varsayalım:

1. `HTTP://WWW.EXAMPLE.COM:80/dir/page.html`
2. `http://www.example.com/dir/page.html#section2`
3. `http://www.example.com/dir/other/../page.html`
4. `http://www.example.com/dir/./page.html`
5. `http://www.example.com/dir/%73ubdir/page.html`

a) Yukarıdaki beş URL'nin her biri için, hangi kanonikleştirme dönüşümlerinin uygulanması gerektiğini adım adım açıklayınız. Dönüşümleri uygulayarak her bir URL'nin kanonik formunu yazınız.

b) URL 3 ve URL 4'teki `/../` ve `/./` yol bileşenlerinin ne anlama geldiğini ve bunların Web sunucuları tarafından nasıl yorumlandığını tartışınız.

c) Kanonikleştirmenin crawler performansı açısından neden kritik olduğunu, özellikle de aynı sayfanın birden çok kez taranmasının önlenmesi bağlamında açıklayınız. Varsayılan port numarası, fragment tanımlayıcıları ve büyük-küçük harf duyarlılığı gibi unsurların her birinin kanonikleştirmedeki rolünü belirtiniz.

Çözüm

Adım 1: Beş URL'nin kanonikleştirme dönüşümleri.

URL 1 (`HTTP://WWW.EXAMPLE.COM:80/dir/page.html`) için iki dönüşüm uygulanır. İlk olarak büyük-küçük harf normalizasyonu yapılır; protokol ismi `http` ve ana makine adı `www.example.com` olarak küçük harfe dönüştürülür. İkinci olarak `:80` varsayılan port numarası kaldırılır, çünkü HTTP protokolü için 80 portu zaten varsayılandır ve açıkça belirtilmesi anlamsal bir fark yaratmaz.

Nihai kanonik form:

$$\text{http://www.example.com/dir/page.html} \quad (145)$$

URL 2 (`http://www.example.com/dir/page.html#section2`) için fragment tanımlayıcısı olan `#section2` kısmı atılır. Fragment tanımlayıcıları sayfanın içindeki bir bölüme işaret eden istemci tarafı referanslardır ve sunucuya gönderilen HTTP isteğini etkilemezler. Dolayısıyla aynı sayfa içindeki farklı fragment'lere sahip URL'ler aynı kaynağa işaret eder.

Nihai kanonik form:

$$\text{http://www.example.com/dir/page.html} \quad (146)$$

URL 3 (`http://www.example.com/dir/other/../page.html`) için yol normalizasyonu uygulanır. `/../` yol bileşeni "üst dizine çık" anlamına gelir. `/dir/other/..` ifadesi çözümlendiğinde, `/other/` dizinine girilip hemen `/../` ile bir üst dizine (`/dir/`) geri dönülür.

Nihai kanonik form:

$$\text{http://www.example.com/dir/page.html} \quad (147)$$

URL 4 (`http://www.example.com/dir/./page.html`) için `/./` yol bileşeni temizlenir. `/./` "bulunulan dizin" anlamına gelir ve anlamsal olarak nötrdür.

Nihai kanonik form:

`http://www.example.com/dir/page.html` (148)

URL 5 (`http://www.example.com/dir/%73ubdir/page.html`) için yüzde kodlaması çözülür. %73 ASCII'deki "s" harfine karşılık gelir. Bu kodlama çözüldüğünde `/dir/subdir/page.html` yolu elde edilir.

Nihai kanonik form:

`http://www.example.com/dir/subdir/page.html` (149)

Görüldüğü gibi, URL 1 ve URL 2 tamamen aynı sayfaya (`/dir/page.html`) işaret etmekte olup kanonikleştirme sonrası aynı forma indirgenirler. URL 3 ve URL 4 de aynı hedefe ulaşır. Yalnızca URL 5 farklı bir sayfaya (`/dir/subdir/page.html`) işaret etmektedir.

Adım 2: `/../` ve `/./` yol bileşenlerinin anlamı.

`/../` bileşeni, Unix dosya sistemlerindeki "parent directory" kavramının URL yol hiyerarşisine taşınmış halidir; bulunulan dizinin bir üst seviyesine çıkmayı ifade eder.

`/./` bileşeni ise "current directory" anlamına gelir ve tamamen nötr bir bileşendir. Her ikisi de URL yol çözümlemesinde standart olarak RFC 3986'da tanımlanmıştır.

Web sunucuları bu bileşenleri istek işleme öncesinde normalize ederek çözümler. Crawler'ın bu bileşenleri kendi başına temizlemesi gerekir; aksi halde aynı sayfaya farklı URL'ler üzerinden tekrar tekrar erişmeye çalışarak bant genişliği ve zaman kaybına yol açar.

Adım 3: Kanonikleştirmenin crawler performansı açısından kritik rolü.

Web crawler'ın en önemli uygulama detaylarından biri URL'lerin doğru şekilde işlenmesidir. Kanonikleştirme olmaksızın, aynı sayfaya işaret eden çok sayıda farklı URL, crawler tarafından farklı sayfalar olarak algılanır. Bu durumda crawler aynı sayfayı tekrar tekrar indirir, parse eder ve dizinler; bu da hem bant genişliğini hem işlemci zamanını hem de depolama alanını boşa harcar. Büyük ölçekli taramalarda bu israf katlanarak büyür.

Varsayılan port numarasının kaldırılması ile ilgili olarak, :80 ekini taşıyan ve taşımayan URL'lerin aynı sayfayı göstermesi sağlanır.

Fragment tanımlayıcılarının atılması ile sayfa içi bağlantı farklılıklarının gereksiz yere farklı sayfalar olarak algılanması önlenir.

Büyük-küçük harf normalizasyonu ise özellikle Windows tabanlı sunucuların (büyük-küçük harf duyarsız dosya sistemleri) ve Unix tabanlı sunucuların (büyük-küçük harf duyarlı dosya sistemleri) farklı davranışları arasındaki tutarsızlığı gidermek için hayattır. Ana makine adlarının DNS standardı gereği zaten büyük-küçük harf duyarsız olmasına rağmen, yol kısmında aynı garanti yoktur.

Tüm bu dönüşümler sonucunda URL'ler tek bir kanonik forma indirgenir ve crawler'ın ziyaret edilmiş sayfaları takip eden veri yapısında (ziyaret edilmişler tablosu ya da Bloom filtresi) tutarlılık sağlanır.

Soru 2: Göreli URL'den Mutlak URL'ye Dönüşüm

Bir web crawler, temel (base) URL'si `http://www.universite.edu.tr/bolumler/bilgisayar/` olan bir sayfayı indirmiştir. Bu sayfanın HTML içeriğinden aşağıdaki göreli (relative) bağlantılar çıkarılmıştır:

1. `akademik_kadro.html`
2. `/duyurular/2025/sinav_takvimi.pdf`
3. `../muhendislik/lisans_programlari.html`

4. //kutuphane.universite.edu.tr/veritabanlari/

5. ?lang=tr&sayfa=ozgecmis

a) Yukarıdaki beş görel URL'nin her birini mutlak URL'ye dönüştürünüz. Her dönüşümde temel URL'nin hangi bileşenlerinin kullanıldığını ve görel URL'nin tipini (yola görel, köke görel, protokole görel vb.) belirtiniz.

b) 5 numaralı ?lang=tr&sayfa=ozgecmis görel URL'si ne tür bir referanstır? Bu URL tipi, sayfanın hangi kısmının değiştiğini ancak hangi kısmının sabit kaldığını ifade eder? Crawler'ın bu URL'yi işlerken dikkat etmesi gereken husus nedir?

c) Crawler'ın temel URL'yi belirlemek için kullanabileceği üç farklı kaynağı sıralayınız ve bu kaynaklar arasındaki öncelik sırasını (hangisinin önce kontrol edildiğini) gerekçesiyle açıklayınız.

Çözüm

Adım 1: Beş görel URL'nin mutlak URL'ye dönüşümü.

1 numaralı akademik_kadro.html yola görel (path-relative) bir URL'dir; temel URL'nin bulunduğu dizine göre çözümlenir. Temel URL'nin yol bileşeni /bolumler/bilgisayar/ olduğundan, bu görel URL temel yolun sonuna eklenir.

Mutlak URL:

http://www.universite.edu.tr/bolumler/bilgisayar/akademik_kadro.html (150)

2 numaralı /duyurular/2025/sinav_takvimi.pdf köke görel (root-relative) bir URL'dir; başındaki / karakteri yolun sunucu kök dizininden başladığını belirtir. Bu durumda temel URL'nin yalnızca protokol ve ana makine adı kısmı kullanılır, yol kısmı tamamen görel URL ile değiştirilir.

Mutlak URL:

http://www.universite.edu.tr/duyurular/2025/sinav_takvimi.pdf (151)

3 numaralı ../muhendislik/lisans_programlari.html yola görel bir URL'dir ancak ../ ile bir üst dizine çıkmayı ifade eder. Temel yol /bolumler/bilgisayar/ olduğundan, bir üst dizin /bolumler/ olur ve muhendislik/lisans_programlari.html buraya eklenir.

Mutlak URL:

http://www.universite.edu.tr/bolumler/muhendislik/lisans_programlari.html (152)

4 numaralı //kutuphane.universite.edu.tr/veritabanlari/ protokole görel (protocol-relative) bir URL'dir; // ile başlaması, protokolün temel URL'dekiyle aynı kalacağını ancak ana makine adının değişeceğini belirtir. Temel URL'den yalnızca protokol (http:) alınır.

Mutlak URL:

<http://kutuphane.universite.edu.tr/veritabanlari/> (153)

5 numaralı ?lang=tr&sayfa=ozgecmis sorgu parametrelili bir görel URL'dir; yalnızca sorgu dizgesi (query string) içerir. Yol bileşeni tamamen temel URL'den alınır ve mevcut sorgu dizgesi bu yeni sorgu dizgesiyle değiştirilir. Temel URL'nin yolu /bolumler/bilgisayar/ olduğundan:

Mutlak URL:

<http://www.universite.edu.tr/bolumler/bilgisayar/?lang=tr&sayfa=ozgecmis> (154)

Adım 2: Sorgu parametrelili URL'nin anlamı ve crawler açısından önemi.

?lang=tr&sayfa=ozgecmis ifadesi, aynı sayfanın farklı bir sorgu dizgesiyle çağrılmasını temsil eder. Bu URL tipi, sayfanın yol kısmının sabit kaldığını (/bolumler/bilgisayar/), yalnızca sorgu parametrelerinin değiştiğini ifade eder.

Crawler'ın burada dikkat etmesi gereken en kritik husus, dinamik sayfaların (sunucu tarafı betiklerle oluşturulan sayfalar) sorgu parametrelerindeki her değişikliğin gerçekten farklı bir içerik üretip üretmediğidir. Eğer sorgu parametreleri yalnızca sayfanın sunumunu değiştiriyorsa (örneğin dil seçimi, sıralama düzeni gibi), aynı içeriğin birden çok kez taranması söz konusu olabilir. Bu, örümcek tuzaklarına (spider traps) yol açma riski taşır; zira CGI betikleri sonsuz sayıda farklı sorgu parametresi kombinasyonu üretebilir. Crawler'ın bu tür dinamik URL'leri tespit edip uygun şekilde filtrelemesi gerekir.

Adım 3: Temel URL'nin belirlenmesi için üç kaynak ve öncelik sırası.

Crawler temel URL'yi sırasıyla şu kaynaklardan belirler:

İlk olarak HTML `<base>` meta etiketi kontrol edilir. Sayfanın `<head>` bölümünde yer alan `<base href="...">` etiketi, sayfadaki tüm görelî bağlantılar için açıkça bir temel URL tanımlar ve bu nedenle en yüksek önceliğe sahiptir.

İkinci olarak HTTP yanıt başlığındaki `Content-Location` veya `Location` alanı kontrol edilir. Sunucu tarafından gönderilen bu başlık, istenen kaynağın gerçek konumunu belirtir; sayfa yönlendirme sonucu gelmişse bu başlık hayati önem taşır.

Üçüncü olarak, eğer ne HTML `<base>` etiketi ne de ilgili HTTP başlığı mevcutsa, crawler sayfanın kendi URL'sini temel URL olarak kabul eder.

Bu üç kaynak bu öncelik sırasıyla kontrol edilir; çünkü `<base>` etiketi sayfa yazarının bilinçli bir tercihini yansıtır, HTTP başlığı sunucu tarafı gerçek konumu belirtir, sayfanın kendi URL'si ise bu ikisinin yokluğunda makul bir varsayılan değer sunar.

Soru 3: Bloom Filtresi ile Yanlış Pozitif Olasılığı Hesabı

Bir web crawler, daha önce ziyaret ettiği URL'leri takip etmek için bir Bloom filtresi kullanmaktadır. Filtre $N = 2^{30}$ (yaklaşık 1.07 milyar) bitlik bir bit dizisi ve $k = 6$ adet bağımsız hash fonksiyonu ile yapılandırılmıştır. Crawler şu ana kadar $m = 200$ milyon (2×10^8) adet URL'yi filtreye eklemiştir.

a) Filtreye eklenen toplam dart sayısını (d) hesaplayınız. Dart atma modelini kullanarak, tek bir bitin 0 olarak kalma olasılığını ($e^{-d/N}$ yaklaşımıyla) hesaplayınız. Dizideki 1'lerin yoğunluğunu (density) bulunuz.

b) Bu yoğunluğu kullanarak yanlış pozitif olasılığını $P(\text{FP}) = (\text{yoğunluk})^k$ formülüyle hesaplayınız. Sonucu hem ondalık sayı hem de yüzde olarak ifade ediniz.

c) Crawler'ın $m = 400$ milyon URL eklemesi durumunda (diğer parametreler aynı kalmak kaydıyla) yanlış pozitif olasılığını yeniden hesaplayınız. İki senaryo arasındaki farkı yorumlayınız. Yanlış pozitif olasılığının m ile nasıl değiştiğini matematiksel olarak açıklayınız.

d) Crawler'ın karşılaştığı bir yanlış pozitif durumunda ne gibi bir sonuç ortaya çıkar? Buna karşılık Bloom filtresinin neden yanlış negatif (false negative) üretmediğini ve bunun crawler için neden kritik bir garanti olduğunu açıklayınız. Yanlış pozitifin yarattığı maliyet ile yanlış negatifin yaratacağı maliyeti karşılaştırmamız.

Çözüm

Adım 1: Dart sayısı, 0 kalma olasılığı ve 1 yoğunluğunun hesaplanması.

Toplam dart sayısı, eklenen her URL için k hash fonksiyonu çalıştırıldığından aşağıdaki gibi hesaplanır:

$$d = k \cdot m = 6 \times 2 \times 10^8 = 1.2 \times 10^9 \quad (155)$$

Dizi boyutu $N = 2^{30} = 1.073.741.824 \approx 1.074 \times 10^9$ bittir.

Dart başına hedef oranı şu şekilde bulunur:

$$\frac{d}{N} = \frac{1.2 \times 10^9}{2^{30}} = \frac{1.2 \times 10^9}{1.0737 \times 10^9} \approx 1.1176 \quad (156)$$

Tek bir bitin d dartın hiçbirisi tarafından vurulmama, yani 0 olarak kalma olasılığı:

$$P(\text{bit 0 kalır}) = \left(1 - \frac{1}{N}\right)^d \approx e^{-d/N} = e^{-1.1176} \quad (157)$$

$$e^{-1.1176} \approx 0.3270 \quad (158)$$

Dolayısıyla dizideki bitlerin yaklaşık %32.70'i 0 olarak kalır.

Dizideki 1'lerin yoğunluğu:

$$1 \text{ yoğunluğu} = 1 - e^{-d/N} = 1 - 0.3270 = 0.6730 \quad (159)$$

Adım 2: Yanlış pozitif olasılığının hesaplanması.

Yanlış pozitif, daha önce eklenmemiş bir URL için k hash fonksiyonunun tamamının hâlihazırda 1 olan bitlere işaret etmesi durumunda gerçekleşir. Her bir hash sonucunun 1 olan bir bite denk gelme olasılığı yoğunluğa eşittir ve k bağımsız hash fonksiyonu için:

$$P(\text{FP}) = \left(1 - e^{-d/N}\right)^k = (0.6730)^6 \quad (160)$$

$$P(\text{FP}) = 0.6730^6 \approx 0.0927 \quad (161)$$

Yüzde olarak ifade edildiğinde yaklaşık %9.27 yanlış pozitif olasılığı söz konusudur. Bu, 200 milyon URL için yaklaşık 1.07 milyar bit (yaklaşık 128 MB) bellek kullanıldığında elde edilen bir orandır.

Adım 3: $m = 400$ milyon URL için yeniden hesaplama ve karşılaştırma.

Yeni dart sayısı:

$$d = 6 \times 4 \times 10^8 = 2.4 \times 10^9 \quad (162)$$

Dart başına hedef oranı:

$$\frac{d}{N} = \frac{2.4 \times 10^9}{1.0737 \times 10^9} \approx 2.2352 \quad (163)$$

0 kalma oranı:

$$e^{-d/N} = e^{-2.2352} \approx 0.1070 \quad (164)$$

1'lerin yoğunluğu:

$$1 - 0.1070 = 0.8930 \quad (165)$$

Yeni yanlış pozitif olasılığı:

$$P(\text{FP}) = (0.8930)^6 \approx 0.5076 \quad (166)$$

Yanlış pozitif olasılığı %50.76'ya fırlamıştır.

İki senaryo arasındaki dramatik fark, Bloom filtresinin doyuma ulaşma davranışını gösterir: d/N oranı büyüdükçe 1'lerin yoğunluğu 1'e yaklaşır ve $(1 - e^{-d/N})^k$ ifadesi hızla büyür. Matematiksel olarak, m arttıkça $d = k \cdot m$ doğrusal olarak artar, $e^{-d/N}$ üstel olarak azalır, $(1 - e^{-d/N})$ ise 1'e yaklaşır. Bu nedenle yanlış pozitif olasılığı m ile süper-doğrusal bir şekilde artar.

Filtrenin etkin kullanımı için d/N oranının makul bir seviyede tutulması gerekir; aksi takdirde filtre neredeyse her sorgulamada “evet” demeye başlar ve kullanışsız hale gelir.

Adım 4: Yanlış pozitif ve yanlış negatifin sonuçlarının karşılaştırılması.

Bir yanlış pozitif durumunda, Bloom filtresi daha önce ziyaret edilmemiş bir URL için “bu URL muhtemelen daha önce görüldü” der. Bunun sonucunda crawler bu URL'yi atlar ve yeni bir sayfayı kaçırmış olur. Ancak bu sayfa büyük olasılıkla başka bir bağlantı yoluyla tekrar keşfedilecektir. Ayrıca yanlış pozitifin yarattığı maliyet yalnızca bir sayfanın geçici olarak taranmamasıdır; bant genişliği ve zaman açısından büyük bir kayıp değildir.

Buna karşılık Bloom filtresi yanlış negatif (false negative) üretmez — yani bir URL daha önce eklenmişse, ilgili tüm bitler 1 olarak işaretlenmiş olduğundan filtre mutlaka onu tanır. Bu kritik bir garantidir çünkü yanlış negatif olsaydı, crawler daha önce ziyaret ettiği bir sayfayı “yeni” sanıp tekrar indirmeye kalkışacak ve bu durum örümcek tuzaklarıyla birleştiğinde crawler'ın sonsuz döngüye girmesine neden olabilecekti.

Yanlış negatifin maliyeti (aynı sayfanın defalarca taranması) yanlış pozitifin maliyetinden (bir sayfanın atlanması) çok daha yüksek olduğu için Bloom filtresinin “kesinlikle hayır, güvenilirmez evet” (no false negatives, possible false positives) tasarım tercihi crawler bağlamında son derece isabetlidir.

Soru 4: BFS ve DFS Frontier Stratejileri

Bir web crawler'ın frontier (sınır) veri yapısı, taranacak URL'lerin hangi sırayla ziyaret edileceğini belirleyen merkezi bileşendir. Frontier'ın gerçekleştirimi, Web'in bir çizge (graph) olarak taranması problemine karşılık gelir.

a) Genişlik öncelikli arama (BFS) ve derinlik öncelikli arama (DFS) stratejilerini frontier veri yapısı açısından karşılaştırınız. Her iki stratejinin hangi veri yapısıyla (FIFO/LIFO) gerçekleştirildiğini, nasıl çalıştığını ve birbirlerinden hangi açılardan ayrıldıklarını açıklayınız.

b) Web crawling bağlamında BFS'nin DFS'ye göre neden daha üstün olduğunu, başlangıç (seed) sayfalarının rolü ve “siber uzayda kaybolma” (lost in cyberspace) kavramı üzerinden tartışınız. DFS stratejisinin Web ortamında yarattığı riskleri somut örneklerle açıklayınız.

c) Büyük ölçekli bir evrensel crawler'da frontier'ın yalnızca basit bir kuyruk olarak gerçekleştirilmesi neden yetersiz kalır? Frontier'ın ölçeklenebilir bir mimaride nasıl gerçekleştirilmesi gerektiğini, paralel kuyruklar ve önceliklendirme mekanizmaları bağlamında kısaca açıklayınız.

Çözüm

Adım 1: BFS ve DFS'nin veri yapısı ve çalışma prensipleri açısından karşılaştırılması.

BFS, bir kuyruk (queue) veri yapısıyla, yani FIFO (First-In First-Out) prensibiyle gerçekleştirilir. Algoritma, önce başlangıç (seed) URL'lerini kuyruğa ekler; ardından sırasıyla her URL'yi kuyruktan çıkarır, sayfayı indirir, sayfadaki bağlantıları çıkarır ve yeni bağlantıları kuyruğun sonuna ekler. Bu sayede başlangıç sayfalarına en kısa yoldan (en az tıklama mesafesinde) ulaşılabilen sayfalar öncelikli olarak ziyaret edilir.

DFS ise bir yığın (stack) veri yapısıyla, yani LIFO (Last-In First-Out) prensibiyle gerçekleştirilir. DFS'de bir sayfadan çıkarılan bağlantılar yığının en üstüne eklenir, böylece crawler sürekli olarak en son keşfedilen bağlantıyı takip eder.

Bu iki strateji arasındaki temel fark, BFS'nin başlangıç noktasına olan uzaklığı (mesafeyi) kademeli olarak artırması, DFS'nin ise başlangıçtan hızla uzaklaşarak keyfi derinlikteki yollara

dalmasıdır. BFS'nin bellek kullanımı genellikle daha yüksektir çünkü frontier'da aynı anda daha fazla URL tutulur; buna karşılık DFS'nin bellek ayak izi daha küçüktür.

Adım 2: Web crawling'de BFS'nin üstünlüğü ve DFS'nin riskleri.

İyi seçilmiş başlangıç sayfalarından yola çıkıldığında BFS, crawler'ı kaliteli sayfalara yakın tutar. Seed sayfalar genellikle insan tarafından seçilmiş, güvenilir ve konuyla ilgili başlangıç noktalarıdır. BFS, bu sayfalara yakın olan (kısa bağlantı mesafesindeki) sayfaları önceliklendirerek taramanın kalitesini yüksek tutar.

DFS ise crawler'ın başlangıç sayfalarından hızla uzaklaşıp "siber uzayda kaybolmasına" yol açar. Örneğin, bir üniversitenin ana sayfasından başlayan bir DFS crawler'ı, ilk bağlantıyı takip edip hiç geri dönmeden yüzlerce seviye derine inerek başlangıçta hedeflenen akademik içerik yerine tamamen alakasız sayfalara ulaşabilir. Web'in ölçeği ve bağlantı yapısının karmaşıklığı düşünüldüğünde, DFS'nin bu kontrolsüz derinleşme eğilimi ciddi bir risktir.

Ayrıca örümcek tuzakları DFS'de çok daha yıkıcıdır; crawler bir tuzığa düştüğünde yığının en tepesindeki URL'leri sürekli işleyerek tuzığın içinde sonsuza kadar kalabilir. BFS'de ise tuzaktan gelen URL'ler kuyruğun sonuna ekleneceğinden diğer sayfaların da taranma şansı olur. Tüm bu nedenlerle web crawling'de BFS stratejisi standart olarak tercih edilir.

Adım 3: Büyük ölçekli frontier gerçekleştirimi için gereksinimler.

Milyarlarca sayfanın tarandığı evrensel crawler'larda basit bir kuyruk yapısı iki temel nedenden ötürü yetersiz kalır. İlk olarak, tüm URL'leri tek bir makinenin belleğinde tutmak mümkün değildir; frontier'ın disk tabanlı ve dağıtık olması gerekir. İkinci olarak, tüm URL'ler eşit öneme sahip değildir; bazı sayfalar daha sık değişir, bazıları daha yüksek PageRank değerine sahiptir, bazıları ise daha tazedir.

Ölçeklenebilir bir mimaride frontier, birden çok paralel kuyruk olarak gerçekleştirilir. Bu kuyruklar, yükü farklı sunuculara yaymak ve bağlantıları canlı tutmak amacıyla ana makine adına (hostname) göre bölümlenir; böylece aynı sunucuya yapılan istekler aynı kuyrukta toplanarak HTTP bağlantılarının yeniden kullanımı (keep-alive) sağlanır.

Ayrıca frontier, bir önceliklendirme mekanizması içermelidir: tazelik politikasına göre değişme olasılığı yüksek sayfalar, PageRank gibi önem ölçütlerine göre yüksek prestijli sayfalar ya da konusal crawler'larda benzerlik puanı yüksek sayfalar öncelikli olarak taranmalıdır. Bu önceliklendirme, frontier'ın bir öncelik kuyruğu (priority queue) olarak gerçekleştirilmesini gerektirir. Büyük ölçekli sistemlerde bu yapı, dağıtık dosya sistemleri ve senkronizasyon mekanizmalarıyla desteklenir.

Soru 5: Crawler Mimarisi, Bileşenleri ve Tazelik Politikası

Bir web crawler'ın temel sıralı (sequential) mimarisi; seed, frontier, fetch, parse/extract ve repository olmak üzere beş ana bileşenden oluşur. Ayrıca evrensel crawler'lar hiçbir zaman "tamamlanmaz"; Web'deki sayfalar sürekli değiştiği için etkin bir tazelik (freshness) politikası zorunludur.

a) Sıralı bir crawler'ın beş ana bileşenini sırasıyla açıklayınız. Her bir bileşenin görevini, girdisini ve çıktısını belirterek bileşenler arasındaki veri akışını tanımlayınız. Bu mimaride frontier bileşeninin neden kritik bir rol oynadığını vurgulayınız.

b) Evrensel crawler'lar için tazelik politikasının gerekliliğini açıklayınız. HTTP başlıklarındaki Last-Modified ve Expires alanlarının neden sayfa değişimlerini izlemek için güvenilir olmadığını tartışınız.

c) Ntoulas, Cho ve Olston (2004) tarafından yapılan çalışmada, değişim sıklığının (change frequency) değil, değişim derecesinin (degree of change) daha iyi bir tahminleyici olduğu gösterilmiştir. Bu bulguyu bir örnek üzerinden açıklayınız: Her saat başı küçük bir zaman damgası güncellemesi alan bir sayfa ile haftada bir kez tüm içeriği yeniden yazılan bir sayfayı karşılaştırınız.

Hangi sayfanın daha sık taranması gerektiğini ve neden değişim derecesinin daha iyi bir metrik olduğunu tartışınız.

d) Page repository (sayfa deposu) bileşeninde, her sayfanın ayrı bir dosya olarak saklanması yaklaşımının neden verimsiz olduğunu ve daha iyi bir alternatifin nasıl çalıştığını açıklayınız. URL'den benzersiz dosya adı üretmek için MD5 gibi bir özetleme fonksiyonunun nasıl kullanılabilirliğini belirtiniz.

Çözüm

Adım 1: Sıralı crawler mimarisinin beş bileşeninin açıklanması.

Seed (başlangıç) bileşeni, crawler'ın taramaya başlayacağı başlangıç URL'lerinin listesini içerir. Bu URL'ler genellikle insan tarafından seçilmiş, güvenilir ve konuyla ilgili sayfalardır. Seed kümesinin kalitesi, tüm taramanın kalitesini doğrudan etkiler.

Frontier (sınır) bileşeni, şu ana kadar keşfedilmiş ancak henüz ziyaret edilmemiş URL'leri tutan ve crawler'ın sayfaları hangi sırayla ziyaret edeceğini belirleyen veri yapısıdır. Girdi olarak seed URL'lerini ve parse aşamasından gelen yeni keşfedilmiş URL'leri alır; çıktı olarak sıradaki ziyaret edilecek URL'yi fetch bileşenine iletir. Frontier, Web çizgesinin taranma sırasını belirlediği için mimarinin en kritik bileşenidir.

Fetch (getirme) bileşeni, frontier'dan gelen URL'yi kullanarak ilgili sayfayı HTTP protokolü üzerinden indirir. Girdisi bir URL, çıktısı ise sayfanın ham HTML içeriğidir. Bu bileşenin sağlam (robust) olması gerekir; zaman aşımı, sunucu yanıt vermemesi, dosya bulunamaması gibi hata durumlarını yönetebilmelidir. Ayrıca her sayfanın yalnızca ilk 10–100 KB'lık kısmının indirilmesi gibi optimizasyonlar uygulanabilir.

Parse/Extract (çözümleme/çıkarma) bileşeni, indirilen sayfanın HTML içeriğini çözümleyerek iki şey üretir: sayfanın metin içeriği (dizinleme için) ve sayfadaki tüm köprü bağlantıları (hyperlink). Girdisi ham HTML, çıktıları ise işlenmiş metin ve görel URL'lerin listesidir. Bu aşamada görel URL'ler mutlak URL'lere dönüştürülür, kanonikleştirme uygulanır ve durdurma sözcükleri gibi metin ön işleme adımları gerçekleştirilir.

Repository (depo) bileşeni, indirilen ve işlenen sayfaların kalıcı olarak saklandığı depolama birimidir. Girdisi parse edilmiş sayfa içeriği, çıktısı ise dizinleme motorunun kullanabileceği organize edilmiş doküman koleksiyonudur.

Veri akışı: Seed URL'leri frontier'a yüklenir. Frontier'dan sıradaki URL çıkarılarak fetch bileşenine gönderilir. Fetch, sayfayı indirip parse bileşenine iletir. Parse, metni ve bağlantıları çıkarır. Metin repository'ye kaydedilir; yeni bağlantılar işlenip frontier'a eklenir. Bu döngü, belirlenen durdurma kriteri (maksimum sayfa sayısı, maksimum süre vb.) sağlanana kadar devam eder.

Adım 2: Tazelik politikasının gerekliliği ve HTTP başlıklarının güvenilirliği.

Evrensel crawler'lar hiçbir zaman "tamamlanmaz" çünkü Web dinamik bir ortamdır; sayfalar sürekli değişir, yeni sayfalar eklenir ve mevcut sayfalar silinir. Bir crawler tüm Web'i taramayı başarsa bile, tarama tamamlandığı anda bazı sayfalar çoktan güncelliğini yitirmiş olacaktır. Bu nedenle crawler'ın sürekli olarak sayfaları yeniden ziyaret etmesi ve indeksini güncel tutması gerekir. Ancak sonsuz sayıda sayfanın sürekli taranması mümkün olmadığından, hangi sayfaların ne sıklıkla yeniden taranacağına karar veren bir tazelik politikası zorunludur.

HTTP başlıklarındaki **Last-Modified** ve **Expires** alanları bu amaç için güvenilir değildir. **Last-Modified** başlığı, sunucunun bildirdiği son değişiklik tarihini içerir; ancak birçok sunucu bu alanı güncellemez ya da yanıltıcı değerler döndürür. Dinamik olarak oluşturulan sayfalarda bu alan genellikle isteğin yapıldığı anı gösterir, bu da sayfanın sürekli değişiyormuş gibi algılanmasına yol açar.

Expires başlığı ise sayfanın ne zaman güncelliğini yitireceğini belirtir; ancak bu tamamen sunucu yöneticisinin takdirindedir ve çoğu zaman gerçekçi olmayan değerler içerir. Bu güvenilirlik nedeniyle crawler'lar, HTTP başlıklarına güvenmek yerine sayfaların geçmiş değişim

davranışına dayanan istatistiksel tahmin yöntemleri geliştirmiştir.

Adım 3: Değişim sıklığı ve değişim derecesi karşılaştırması.

Her saat başı yalnızca küçük bir zaman damgası güncellemesi alan bir A sayfası ile haftada bir kez tüm içeriği yeniden yazılan bir B sayfasını ele alalım. Değişim sıklığı (change frequency) açısından bakıldığında A sayfası çok daha sık değişmektedir ve bu metriğe göre daha sık taranması gerekir. Oysa A sayfasındaki her değişiklik, sayfanın anlamsal içeriğini neredeyse hiç etkilemeyen önemsiz bir güncellemedir. B sayfası ise haftada bir kez değişmesine rağmen her değişiklikte sayfanın büyük bir kısmı yeniden yazılmakta ve anlamsal içeriği önemli ölçüde farklılaşmaktadır.

Ntoulas, Cho ve Olston (2004) tarafından gösterildiği üzere, yakın geçmişin geleceği tahmin edeceği varsayımı altında değişim derecesi (degree of change), değişim sıklığından daha iyi bir tahminleyicidir. Değişim derecesi, bir sayfanın iki ardışık ziyareti arasındaki içerik farkının büyüklüğünü ölçer; örneğin sayfadaki terimlerin yüzde kaçının değiştiği, eklenen veya silinen metin miktarı gibi.

Bu örnekte B sayfasının değişim derecesi yüksek olduğu için daha sık taranması, A sayfasının ise değişim derecesi düşük olduğu için daha seyrek taranması gerekir. Bu yaklaşım, crawler'ın sınırlı bant genişliğini ve tarama kapasitesini anlamsal olarak anlamlı değişikliklerin olduğu sayfalara yönlendirmesini sağlar.

Adım 4: Sayfa deposu gerçekleştirimi ve verimli alternatifler.

Her sayfanın ayrı bir dosya olarak saklanması naif bir yaklaşımdır. URL'den benzersiz bir dosya adı üretmek için MD5 gibi bir kriptografik özetleme fonksiyonu kullanılabilir; MD5, herhangi bir uzunluktaki URL'yi 128 bitlik (32 karakter hexadecimal) sabit uzunlukta bir özete dönüştürür. Örneğin <http://www.example.com/page.html> URL'si için MD5 ile 5c2e...a4f1 gibi benzersiz bir dosya adı üretilebilir.

Ancak bu yöntem, çok sayıda küçük dosya oluşturduğundan dosya sistemi açısından verimsizdir. Dosya sistemleri milyonlarca küçük dosyayı yönetirken ciddi performans sorunları yaşar; her dosya için ayrı inode, ayrı metadata ve ayrı disk arama (seek) işlemi gerekir.

Daha iyi bir yaklaşım, birçok sayfanın XML işaretleme ile ayrılarak tek bir büyük dosyada birleştirilmesidir. Bu durumda her sayfa için {dosya adı, sayfa kimliği} eşlemesi tutulur. Sayfalar büyük bir birleşik dosyada (örneğin her biri 1 GB'lık dosyalar) saklanır; her sayfa XML etiketleriyle (<page id="...">...</page>) sarılarak birbirinden ayrılır. Bu yaklaşım, disk arama sayısını azaltır, sıralı okuma performansını artırır ve dosya sistemi metadata yükünü minimize eder.

Depolama için ilişkisel veritabanı yönetim sistemleri (RDBMS) büyük ek yük (overhead) getirdiğinden, Berkeley DB gibi hafif ve gömülü veritabanları tercih edilir. Berkeley DB, anahtar-değer (key-value) tabanlı basit bir yapı sunar; URL anahtar, sayfa içeriği değer olarak saklanır ve bu sayede hem hızlı erişim hem de düşük ek yük sağlanır.

5 Web Kullanım Madenciliği – Sınav Soruları

Soru 1: Kullanıcı Tanımlama Yöntemlerinin Karşılaştırılması

Bir e-ticaret şirketi, web sitesi için kullanıcı tanımlama yöntemini seçmek üzere bir analiz yapmaktadır. Şirketin gereksinimleri şunlardır: (i) kullanıcılar siteye kayıt olmadan da takip edilebilmelidir, (ii) aynı kullanıcı farklı cihazlardan (örneğin ev bilgisayarı, iş bilgisayarı ve cep telefonu) geldiğinde de tutarlı şekilde tanımlanabilmelidir, (iii) veri kapsamı (coverage) mümkün olduğunca yüksek olmalıdır, yani ziyaretçilerin büyük çoğunluğu tanımlanabilmelidir. Bu üç gereksinim ışığında, web kullanım madenciliğinde kullanılan başlıca beş kullanıcı tanımlama yöntemini — IP+User Agent, Gömülü Oturum Kimliği (Embedded Session ID), Kullanıcı Kaydı (Registration), Çerez (Cookie) ve Yazılım Aracısı (Software Agent) — değerlendiriniz.

a) Her bir yöntemin çalışma prensibini birer cümleyle açıklayınız ve yukarıdaki üç gereksinim (kayıtsız takip, çapraz cihaz tutarlılığı, veri kapsamı) açısından avantajlarını ve dezavantajlarını belirtiniz.

b) Şirketin gereksinimleri göz önüne alındığında, en uygun yöntemin hangisi olduğunu gerekçeleriyle açıklayınız. Neden diğer yöntemlerin yetersiz kaldığını da belirtiniz.

Çözüm

Adım 1: Beş kullanıcı tanımlama yönteminin değerlendirilmesi.

IP adresi ve User Agent kombinasyonuna dayalı yöntem, sunucu günlüklerindeki IP adresi ve tarayıcı bilgisini eşleştirerek kullanıcı tanımlaması yapar. Kayıtsız takip açısından avantajlıdır çünkü kullanıcıdan hiçbir işlem talep etmez; veri kapsamı teoride %100'dür çünkü her HTTP isteği bu bilgileri otomatik olarak taşır. Buna karşılık çapraz cihaz tutarlılığı açısından tamamen başarısızdır: aynı kullanıcı farklı cihazlardan farklı IP'lerle ve farklı tarayıcılarla bağlandığında ayrı kullanıcılar gibi görünür. Ayrıca NAT arkasındaki birden fazla kullanıcı aynı IP'yi paylaştığında tek kullanıcı gibi algılanır.

Gömülü oturum kimliği (Embedded Session ID), sunucunun her oturum başlangıcında ürettiği benzersiz bir kimliğin tüm sayfa URL'lerine dinamik olarak eklenmesiyle çalışır. Kayıtsız takip sağlar ve aynı oturum içinde IP değişse bile kullanıcıyı takip edebilir. Ancak çapraz cihaz tutarlılığı yoktur: farklı bir cihazdan ya da yeni bir tarayıcı oturumundan gelen aynı kullanıcıya yeni bir oturum kimliği atanır. Veri kapsamı yüksektir, ancak kullanıcının URL'leri yer imlerine eklemesi veya paylaşması durumunda eski oturum kimliklerinin geçerliliğini yitirmesi sorun yaratır.

Kullanıcı kaydı (Registration) yöntemi, kullanıcının siteye kullanıcı adı ve parola ile giriş yapmasına dayanır. Çapraz cihaz tutarlılığı açısından en başarılı yöntemdir: aynı kullanıcı hangi cihazdan giriş yaparsa yapsın doğru şekilde tanınır. Buna karşılık kayıtsız takip gereksinimini doğrudan ihlal eder, çünkü kullanıcının aktif olarak kayıt olması ve giriş yapması gerekir. Veri kapsamı ise en düşük olan yöntemdir: ziyaretçilerin büyük çoğunluğu kayıt olmadan siteyi terk eder; dolayısıyla yalnızca küçük bir kullanıcı alt kümesi tanımlanabilir.

Çerez (Cookie) tabanlı yöntem, kullanıcının tarayıcısına benzersiz bir kimlik içeren küçük bir metin dosyası yerleştirir. Kayıtsız takip sağlar ve kullanıcıdan onay dışında herhangi bir işlem talep etmez. Veri kapsamı yüksektir; çerezleri devre dışı bırakan veya silen kullanıcılar dışında tüm ziyaretçiler tanımlanabilir. Ancak çapraz cihaz tutarlılığı yoktur: her tarayıcı kendi çerez deposuna sahip olduğundan, aynı kullanıcı farklı cihazlarda veya farklı tarayıcılarda farklı çerez kimlikleriyle görünür. Ayrıca paylaşımlı bilgisayarlarda birden fazla kullanıcı aynı çerezi kullanabilir.

Yazılım aracısı (Software Agent), kullanıcının cihazına yüklenen özel bir yazılımın tüm tarama aktivitelerini kaydetmesiyle çalışır. Kayıtsız takip mümkündür; hatta en zengin ve en

doğru veriyi sağlar. Ancak çapraz cihaz tutarlılığı yalnızca yazılımın tüm cihazlara yüklenmesi koşuluyla sağlanabilir, bu da pratikte neredeyse imkansızdır. Veri kapsamı en düşük olan yöntemdir: kullanıcıların yazılımı gönüllü olarak yüklemesi gerektiğinden, örneklem son derece küçük ve yanlıdır.

Adım 2: En uygun yöntemin belirlenmesi ve diğerlerinin neden yetersiz kaldığı.

Verilen gereksinimler ışığında hiçbir yöntem tek başına tüm gereksinimleri mükemmel şekilde karşılayamaz. Kayıtsız takip ve yüksek veri kapsamı isteği, kullanıcı kaydı ve yazılım aracısı yöntemlerini doğrudan eler; her ikisi de kullanıcının aktif katılımını gerektirdiğinden veri kapsamı çok düşüktür. Çapraz cihaz tutarlılığı gereksinimi ise gömülü oturum kimliği ve IP+User Agent yöntemlerini eler; her ikisi de yalnızca tek bir oturum veya tek bir IP bağlamında çalışır, farklı cihazlardaki aynı kullanıcıyı ilişkilendiremez.

Bu durumda pratikte en uygun strateji, çerez tabanlı yöntemin birincil mekanizma olarak kullanılması ve kullanıcı giriş yaptığında kayıt tabanlı yöntemle birleştirilmesidir. Bu hibrit yaklaşımda, siteye gelen her ziyaretçiye bir çerez kimliği atanır (kayıtsız takip, yüksek kapsam); kullanıcı gönüllü olarak giriş yaptığında, çerez kimliği kullanıcı hesabıyla ilişkilendirilir. Aynı kullanıcı farklı bir cihazdan geldiğinde, yeni bir çerez kimliği alır ancak giriş yaptığı anda bu yeni çerez de aynı hesaba bağlanır. Böylece hem kayıtsız kullanıcılar izlenebilir hem de kayıtlı kullanıcılar için çapraz cihaz tutarlılığı sağlanmış olur. Bu yaklaşım günümüzde büyük e-ticaret sitelerinin standart uygulamasıdır.

Soru 2: Oturumlara Ayırma (Sessionization) — Zaman Odaklı Sezgiseller

Bir web sitesinin sunucu günlüklerinden, belirli bir kullanıcıya (IP: 192.168.1.10, User Agent: Mozilla/5.0) ait aşağıdaki istek dizisi elde edilmiştir. Zaman damgaları (timestamp) “dd-mm-yyyy ss:dd:ss” formatında verilmiştir. Oturumlara ayırma işleminde iki zaman odaklı sezgisel birlikte kullanılacaktır: h1 sezgiseli, bir oturumdaki ilk ve son istek arasındaki toplam sürenin 30 dakikayı aşması durumunda oturumu sonlandırır; h2 sezgiseli, ardışık iki istek arasındaki sayfada kalma süresinin 10 dakikayı aşması durumunda yeni bir oturum başlatır.

Sıra	Zaman Damgası	İstenen Sayfa
1	01-06-2026 08:00:00	/index.html
2	01-06-2026 08:02:15	/products/category1.html
3	01-06-2026 08:05:30	/products/itemA.html
4	01-06-2026 08:18:45	/products/itemB.html
5	01-06-2026 08:25:10	/cart.html
6	01-06-2026 08:38:00	/index.html
7	01-06-2026 08:40:20	/products/category2.html
8	01-06-2026 08:44:55	/products/itemC.html
9	01-06-2026 09:20:30	/products/itemD.html
10	01-06-2026 09:22:00	/cart.html
11	01-06-2026 09:28:15	/checkout.html
12	01-06-2026 09:55:40	/index.html

a) Yukarıdaki istek dizisine h2 sezgiselini (ardışık istekler arası 10 dakika sınırı) uygulayarak, hangi istekler arasında oturum sınırı oluştuğunu hesaplayınız. Her ardışık istek çifti için sayfada kalma süresini dakika cinsinden gösteriniz.

b) h2 sezgiseli ile belirlenen oturum sınırlarını dikkate alarak, her bir oturum grubuna h1 sezgiselini (toplam süre 30 dakika sınırı) uygulayınız. Her oturumun başlangıç ve bitiş zamanlarını, toplam süresini ve içerdiği istekleri belirtiniz.

c) Her iki sezgiselin birlikte uygulanması sonucunda oluşan nihai oturum yapısını yazınız ve 9 numaralı isteğin neden yeni bir oturum başlattığını açıklayınız.

Çözüm

Adım 1: h2 sezgiselinin uygulanması ve oturum sınırlarının belirlenmesi.

Ardışık istekler arasındaki süre farkları hesaplanır.

İstek 1 (08:00:00) ile istek 2 (08:02:15) arasında 2.25 dakika vardır; bu değer 10 dakikanın altında olduğu için oturum sınırı oluşmaz.

İstek 2 (08:02:15) ile istek 3 (08:05:30) arasında 3.25 dakika; sınır yok.

İstek 3 (08:05:30) ile istek 4 (08:18:45) arasında 13.25 dakika vardır; bu süre 10 dakikayı aştığı için bir oturum sınırı oluşur.

İstek 4 (08:18:45) ile istek 5 (08:25:10) arasında 6.42 dakika; sınır yok.

İstek 5 (08:25:10) ile istek 6 (08:38:00) arasında 12.83 dakika; 10 dakikayı aştığı için oturum sınırı oluşur.

İstek 6 (08:38:00) ile istek 7 (08:40:20) arasında 2.33 dakika; sınır yok.

İstek 7 (08:40:20) ile istek 8 (08:44:55) arasında 4.58 dakika; sınır yok.

İstek 8 (08:44:55) ile istek 9 (09:20:30) arasında 35.58 dakika; 10 dakikayı aştığı için oturum sınırı oluşur.

İstek 9 (09:20:30) ile istek 10 (09:22:00) arasında 1.50 dakika; sınır yok.

İstek 10 (09:22:00) ile istek 11 (09:28:15) arasında 6.25 dakika; sınır yok.

İstek 11 (09:28:15) ile istek 12 (09:55:40) arasında 27.42 dakika; 10 dakikayı aştığı için oturum sınırı oluşur.

Buna göre h2 sezgiseli dört oturum sınırı belirler: istek 3–4 arası, istek 5–6 arası, istek 8–9 arası ve istek 11–12 arası. Bu sınırlar, istekleri şu gruplara ayırır:

$$\text{Grup A} = \{1, 2, 3\}, \quad \text{Grup B} = \{4, 5\}, \quad \text{Grup C} = \{6, 7, 8\}, \quad \text{Grup D} = \{9, 10, 11\}, \quad \text{Grup E} = \{12\} \quad (167)$$

Adım 2: h1 sezgiselinin her gruba uygulanması.

Şimdi her gruba h1 sezgiseli uygulanarak toplam sürenin 30 dakikayı aşıp aşmadığı kontrol edilir.

Grup A: İlk istek 08:00:00, son istek 08:05:30, toplam süre 5.5 dakika. 30 dakikanın altında olduğu için tek oturum olarak kalır.

$$\text{Oturum-1} = \{1, 2, 3\}, \quad \text{başlangıç: } 08:00:00, \quad \text{bitiş: } 08:05:30, \quad \text{süre: } 5.5 \text{ dakika} \quad (168)$$

Grup B: İlk istek 08:18:45, son istek 08:25:10, toplam süre 6.42 dakika. 30 dakikanın altında, tek oturum.

$$\text{Oturum-2} = \{4, 5\}, \quad \text{başlangıç: } 08:18:45, \quad \text{bitiş: } 08:25:10, \quad \text{süre: } 6.42 \text{ dakika} \quad (169)$$

Grup C: İlk istek 08:38:00, son istek 08:44:55, toplam süre 6.92 dakika. 30 dakikanın altında, tek oturum.

$$\text{Oturum-3} = \{6, 7, 8\}, \quad \text{başlangıç: } 08:38:00, \quad \text{bitiş: } 08:44:55, \quad \text{süre: } 6.92 \text{ dakika} \quad (170)$$

Grup D: İlk istek 09:20:30, son istek 09:28:15, toplam süre 7.75 dakika. 30 dakikanın altında, tek oturum.

$$\text{Oturum-4} = \{9, 10, 11\}, \quad \text{başlangıç: } 09:20:30, \quad \text{bitiş: } 09:28:15, \quad \text{süre: } 7.75 \text{ dakika} \quad (171)$$

Grup E: İlk istek 09:55:40, son istek 09:55:40, toplam süre 0 dakika. Tek istek olduğu için h1 sınırına takılmaz.

$$\text{Oturum-5} = \{12\} \quad (172)$$

Bu örnekte h1 sezgiseli hiçbir grubu bölmemiştir, çünkü her grubun toplam süresi zaten 30 dakikanın oldukça altındadır.

Adım 3: Nihai oturum yapısı ve 9 numaralı isteğin analizi.

Nihai oturum yapısı şu şekildedir:

$$\begin{aligned} \text{Oturum-1} &= \{\text{/index.html, /products/category1.html, /products/itemA.html}\} \\ \text{Oturum-2} &= \{\text{/products/itemB.html, /cart.html}\} \\ \text{Oturum-3} &= \{\text{/index.html, /products/category2.html, /products/itemC.html}\} \\ \text{Oturum-4} &= \{\text{/products/itemD.html, /cart.html, /checkout.html}\} \\ \text{Oturum-5} &= \{\text{/index.html}\} \end{aligned} \quad (173)$$

9 numaralı istek (/products/itemD.html, 09:20:30), bir önceki istekten (istek 8, /products/itemC.html, 08:44:55) tam 35.58 dakika sonra gelmiştir. Bu süre, h2 sezgiselinin 10 dakikalık sayfada kalma sınırının çok üzerinde olduğu için yeni bir oturum sınırı oluşmuştur.

Pratik yorumu şudur: kullanıcı büyük olasılıkla 08:44:55'te itemC sayfasını inceledikten sonra siteyi terk etmiş, yaklaşık 35 dakika boyunca başka bir işle meşgul olmuş ve 09:20:30'da siteye geri dönerek itemD sayfasını görüntülemiştir. Bu 35 dakikalık boşluk, h2 sezgiseli tarafından bir oturum sonu olarak değerlendirilmiştir.

h1 sezgiselinin (30 dakika toplam süre sınırı) burada devreye girmemesinin nedeni, h2'nin zaten daha önce bir sınır çizmiş olmasıdır; h2 sınırı oluştuktan sonra, yeni oturumun toplam süresi (9-10-11 istekleri için sadece 7.75 dakika) h1 sınırının altında kalmıştır.

Soru 3: Yol Tamamlama (Path Completion)

Bir web sitesine ait sayfalar ve aralarındaki bağlantılar (link yapısı) aşağıdaki yönlü graf ile verilmiştir. Her ok, bir sayfadan diğerine doğrudan bir HTML bağlantısı olduğunu göstermektedir:

Sayfalar: A (Ana Sayfa), B (Ürün Kategorileri), C (Ürün X Detay), D (Ürün Y Detay), E (Sepet), F (Arama Sonuçları).

Bağlantılar: $A \rightarrow B$, $A \rightarrow F$, $B \rightarrow C$, $B \rightarrow D$, $C \rightarrow E$, $D \rightarrow E$, $F \rightarrow C$, $F \rightarrow D$, $E \rightarrow B$.

Sunucu günlüklerinde, belirli bir kullanıcının bir oturumu için aşağıdaki sayfa erişim dizisi kaydedilmiştir:

$$\langle A, C, E, D, E \rangle \quad (174)$$

a) Kaydedilen bu dizide hangi ardışık sayfa çiftleri arasında doğrudan bir bağlantı yoktur? Bu eksik bağlantıların en olası nedeni nedir?

b) Eksik bağlantıların bulunduğu her bir geçiş için, site bağlantı grafını kullanarak araya eklenmesi gereken eksik sayfaları belirleyiniz. En kısa yol prensibini kullanarak, her eksik geçiş için olası tamamlama yolunu (veya yollarını) yazınız.

c) Yol tamamlama sonrası oluşan tam (complete) gezinme dizisini yazınız ve bu tamamlanmış dizinin önbellek (caching) etkisiyle nasıl ilişkili olduğunu açıklayınız.

Çözüm

Adım 1: Eksik bağlantıların tespiti.

Kaydedilen dizideki ardışık çiftleri bağlantı grafiyle karşılaştıralım.

$A \rightarrow C$ geçişine bakalım. Bu geçiş için A'dan C'ye doğrudan bir bağlantı yoktur; A yalnızca B ve F sayfalarına bağlanmaktadır. Dolayısıyla bu geçişte bir eksik bağlantı söz konusudur.

$C \rightarrow E$ geçişine bakalım. C'den E'ye doğrudan bir bağlantı mevcuttur ($C \rightarrow E$ graf tanımında verilmiştir). Bu geçiş geçerlidir.

$E \rightarrow D$ geçişine bakalım. E'den D'ye doğrudan bir bağlantı yoktur; E yalnızca B sayfasına bağlanmaktadır. Bu geçişte de bir eksik bağlantı bulunur.

$D \rightarrow E$ geçişine bakalım. D'den E'ye doğrudan bir bağlantı mevcuttur ($D \rightarrow E$). Bu geçiş geçerlidir.

Dolayısıyla $A \rightarrow C$ ve $E \rightarrow D$ geçişlerinde doğrudan bağlantı bulunmamaktadır.

Bu eksik bağlantıların en olası nedeni, kullanıcının tarayıcısının geri düğmesini (back buton) kullanması ve aradaki sayfaları tarayıcı önbelleğinden (cache) yüklemesidir. Önbellekten yüklenen sayfalar için sunucuya yeni bir HTTP isteği gönderilmediğinden, bu sayfalara yapılan erişimler sunucu günlüklerinde görünmez. Sonuç olarak, günlüklerdeki ardışık iki istek arasında gerçekte ziyaret edilmiş ancak kaydedilmemiş sayfalar bulunur.

Adım 2: Yol tamamlama ile eksik sayfaların belirlenmesi.

$A \rightarrow C$ geçişi için, A'dan C'ye ulaşan olası en kısa yolları graf üzerinde arayalım. A'dan çıkan bağlantılar B ve F'dir.

$$A \rightarrow B \rightarrow C \quad (2 \text{ adım}) \quad (175)$$

$$A \rightarrow F \rightarrow C \quad (2 \text{ adım}) \quad (176)$$

Her iki yol da eşit uzunluktadır. Bu noktada belirsizliği gidermek için referrer bilgisine ihtiyaç duyulur; referrer yoksa her iki yol da olasıdır. Tipik bir e-ticaret senaryosunda, ana sayfadan ürün kategorilerine (B) ve oradan ürün detayına (C) gitmek daha yaygın bir davranıştır. En kısa yol prensibiyle $A \rightarrow B \rightarrow C$ tamamlaması uygundur.

$E \rightarrow D$ geçişi için, E'den çıkan tek bağlantı B'dir.

$$E \rightarrow B \rightarrow D \quad (2 \text{ adım}) \quad (177)$$

B, D'ye doğrudan bağlandığı için bu yol hem en kısa hem de tek olası yoldur. Dolayısıyla $E \rightarrow B \rightarrow D$ tamamlaması kesindir.

Adım 3: Tamamlanmış gezinme dizisi ve önbellek etkisi.

Yol tamamlama sonrası oluşan tam gezinme dizisi şu şekildedir:

$$\langle A, B, C, E, B, D, E \rangle \quad (178)$$

Bu tamamlanmış dizi incelendiğinde, kullanıcının gerçek davranışı netleşmektedir: Kullanıcı ana sayfadan (A) ürün kategorilerine (B) gitmiş, buradan Ürün X detay sayfasına (C) geçmiş, ardından Ürün X'i sepete (E) eklemiştir. Daha sonra tarayıcının geri düğmesiyle ürün kategorilerine (B) dönmüş — bu geri dönüş önbellekten yüklendiği için günlükte kaydedilmemiştir — ve buradan Ürün Y detay sayfasına (D) geçmiş, son olarak Ürün Y'yi de sepete (E) eklemiştir.

Önbellek etkisi bu senaryoda kritik bir rol oynamaktadır. Kullanıcı daha önce ziyaret ettiği B (Ürün Kategorileri) sayfasına geri döndüğünde, tarayıcı sayfanın güncel bir kopyasını önbelleğinde tuttuğu için sunucuya yeni bir istek göndermemiştir. Bu nedenle B sayfasına ikinci erişim sunucu günlüklerine yansımamıştır.

Yol tamamlama algoritması, site bağlantı grafini kullanarak eksik B referansını yeniden oluşturmuş ve kullanıcının asıl gezinme yolunu ortaya çıkarmıştır. Yol tamamlama yapılmadan

önce günlüklerde $A \rightarrow C \rightarrow E \rightarrow D \rightarrow E$ şeklinde yanıltıcı bir dizi görünürken, tamamlama sonrası kullanıcının iki ayrı ürünü sırayla inceleyip sepete eklediği anlamlı davranış örüntüsü ortaya çıkmaktadır.

Soru 4: Sayfa Türü Sınıflandırması ve Geçiş Analizi

Bir e-ticaret web sitesinde sayfalar dört işlevsel türe ayrılmaktadır: Başlık (Head – H), Medya (Media – M), Navigasyon (Navigation – N) ve Arama/Veri Girişi (Look-up/Data Entry – L). Başlık sayfaları siteye girişi sağlayan ana sayfa ve kategori ana sayfalarıdır. Medya sayfaları ürün detayı, makale veya blog yazısı gibi asıl içeriği barındırır. Navigasyon sayfaları kullanıcıyı diğer sayfalara yönlendirir; site haritası, kategori listeleri ve arama sonuçları bu türdendir. Arama/Veri Girişi sayfaları ise kullanıcının form doldurduğu, arama yaptığı veya hesap bilgilerini girdiği sayfalardır.

Aşağıda bir web sitesindeki sayfalar ve işlevleri listelenmiştir:

Sayfa	İşlev
/index.html	Site ana sayfası, öne çıkan ürünlerin listelendiği giriş sayfası
/categories/electronics	Elektronik kategorisinin ana sayfası, alt kategorilere bağlantılar içerir
/products/phone123	Bir akıllı telefon modelinin detaylı ürün sayfası
/products/laptop456	Bir dizüstü bilgisayarın detaylı ürün sayfası
/search.php	Arama kutusu ve sonuçların listelendiği sayfa
/sitemap.html	Tüm site yapısını gösteren bağlantı haritası
/cart.html	Alışveriş sepeti içeriğinin görüntülendiği sayfa
/account/register.html	Yeni kullanıcı kayıt formu
/blog/how-to-choose	Blog yazısı: “Nasıl Telefon Seçilir”

a) Yukarıdaki dokuz sayfayı dört sayfa türüne (H, M, N, L) sınıflandırınız. Her sınıflandırmanın gerekçesini birer cümleyle açıklayınız.

b) Bir kullanıcının oturumu şu sayfa geçişlerinden oluşmaktadır: /index.html → /categories/electronics → /products/phone123 → /cart.html → /account/register.html. Bu oturumdaki her bir geçişi, kaynak ve hedef sayfanın türlerini kullanarak (örneğin $H \rightarrow N$ gibi) yorumlayınız. Her geçiş, kullanıcının hangi amacını veya davranışını yansıtmaktadır?

c) Bir e-ticaret analisti, “satın alma hunisi” (purchase funnel) dönüşüm oranlarını ölçmek istemektedir. Sayfa türü sınıflandırmasını kullanarak, $H \rightarrow N \rightarrow M \rightarrow L$ şeklindeki tipik bir dönüşüm yolunun ne anlama geldiğini ve bu tür geçişlerin neden iş değeri açısından önemli olduğunu açıklayınız.

Çözüm

Adım 1: Dokuz sayfanın sayfa türlerine sınıflandırılması.

/index.html, site ana sayfasıdır ve kullanıcıyı siteye yönlendiren bir giriş noktasıdır; dolayısıyla Başlık (H) türündedir.

/categories/electronics, bir kategorinin ana sayfasıdır ve bu kategori altındaki alt sayfalara bağlantılar içerir; bu da Başlık (H) türüne girer, çünkü kategori ana sayfaları da kullanıcıyı daha derin sayfalara yönlendiren giriş noktalarıdır.

/products/phone123 bir ürün detay sayfasıdır, asıl içeriği barındırdığı için Medya (M) türündedir.

/products/laptop456 bir ürün detay sayfasıdır, asıl içeriği barındırdığı için Medya (M) türündedir.

/search.php, arama kutusu ve sonuçları barındırdığı için Arama/Veri Girişi (L) türündedir; ancak arama sonuçlarını da listelediği için aynı zamanda Navigasyon işlevi de taşır — baskın işlev form tabanlı etkileşim olduğu için L kategorisine yerleştirilir.

/sitemap.html tüm site yapısını gösteren bir bağlantı haritasıdır, kullanıcıyı diğer sayfalara yönlendirir; Navigasyon (N) türündedir.

/cart.html alışveriş sepetinin içeriğini görüntüler; bu sayfa hem içerik sunar hem de kullanıcının sepeti düzenlemesine olanak tanır. Sepetteki ürünlerin görüntülenmesi asıl işlev olduğundan Medya (M) olarak sınıflandırılabilir.

/account/register.html bir kayıt formu içerir, kullanıcının veri girişi yaptığı bir sayfadır; Arama/Veri Girişi (L) türündedir.

/blog/how-to-choose bir blog yazısıdır, asıl içeriği barındırır; Medya (M) türündedir.

Özetle:

H (Başlık) : /index.html, /categories/electronics

M (Medya) : /products/phone123, /products/laptop456, /cart.html, /blog/how-to-choose

N (Navigasyon) : /sitemap.html

L (Arama/Veri Girişi) : /search.php, /account/register.html

(179)

Adım 2: Oturum geçişlerinin sayfa türleri üzerinden yorumlanması.

Geçiş 1: $H \rightarrow H$ (/index.html $\rightarrow$ /categories/electronics). Kullanıcı site ana sayfasından bir kategori ana sayfasına geçmektedir. Bu geçiş, kullanıcının siteyi keşfetmeye başladığını ve belirli bir ürün kategorisine (elektronik) ilgi duyduğunu gösterir. Başlık sayfaları arasındaki geçiş, kullanıcının yönelim (orientation) aşamasında olduğuna işaret eder.

Geçiş 2: $H \rightarrow M$ (/categories/electronics $\rightarrow$ /products/phone123). Kullanıcı kategori sayfasından belirli bir ürünün detay sayfasına geçmektedir. Bu, keşfetme davranışından derinlemesine inceleme davranışına geçişi temsil eder; kullanıcı belirli bir ürüne karar verme aşamasına yaklaşmaktadır. E-ticaret açısından bu geçiş yüksek değerlidir çünkü ürün görüntüleme olayını tetikler.

Geçiş 3: $M \rightarrow M$ (/products/phone123 $\rightarrow$ /cart.html). Kullanıcı ürün detay sayfasından sepet sayfasına geçmektedir. Bu geçiş, kullanıcının ürünü sepete eklediğini ve satın alma niyetinin somutlaştığını gösterir. Sepete ekleme, dönüşüm hunisindeki en kritik adımlardan biridir.

Geçiş 4: $M \rightarrow L$ (/cart.html $\rightarrow$ /account/register.html). Kullanıcı sepet sayfasından kayıt formuna yönlendirilmiştir. Bu geçiş, sitenin satın alma işlemini tamamlamak için kullanıcıdan kayıt olmasını talep ettiğini gösterir. Bu noktada kullanıcının kaydı tamamlaması veya siteyi terk etmesi, dönüşüm başarısını belirleyen son eşiktir.

Adım 3: Satın alma hunisi ve sayfa türü geçişlerinin iş değeri açısından önemi.

$H \rightarrow N \rightarrow M \rightarrow L$ şeklindeki tipik bir dönüşüm yolu şu anlama gelir: Kullanıcı bir başlık sayfasından (örneğin ana sayfa) siteye girer; navigasyon sayfasına (örneğin arama sonuçları veya site haritası) geçerek ilgilendiği içeriği arar; bir medya sayfasına (örneğin ürün detayı) ulaşarak içeriği inceler; son olarak bir arama/veri girişi sayfasına (örneğin ödeme veya kayıt formu) geçerek işlemi tamamlar.

Bu tür geçişlerin iş değeri açısından önemi büyüktür çünkü her bir geçiş, dönüşüm hunisindeki bir aşamayı temsil eder:

$H \rightarrow N$ geçişi, kullanıcının site içinde aktif olarak arama yaptığını gösterir; bu aşamada kullanıcı kaybı (drop-off), sitenin navigasyon yapısının yetersiz olduğuna işaret edebilir.

$N \rightarrow M$ geçişi, kullanıcının aradığını bulduğunu ve içerikle etkileşime geçtiğini gösterir; bu aşamadaki kayıp, içerik kalitesi veya alaka sorunlarına işaret eder.

$M \rightarrow L$ geçişi, kullanıcının harekete geçtiğini (satın alma, kayıt, vb.) gösterir; bu aşamadaki kayıp ise form karmaşıklığı, güven eksikliği veya fiyat sürprizi gibi engellere işaret eder.

Her bir geçişteki dönüşüm oranı ölçülerek, huninin hangi aşamasında kullanıcıların kaybedildiği belirlenir ve site tasarımı buna göre optimize edilir. Sayfa türü sınıflandırması, bu analizin otomatikleştirilmesini ve binlerce sayfalı büyük sitelerde dahi ölçeklenebilir olmasını sağlar.

Soru 5: Web Oturumlarında Sık Öğe Kümeleri ve Birliktelik Kuralları

Bir web sitesinin 10 kullanıcı oturumundan oluşan veri kümesi aşağıda verilmiştir. Her oturum, kullanıcının o oturum boyunca eriştiği sayfaların kümesi olarak gösterilmiştir. Sayfalar kısaltmalarla temsil edilmektedir: A= Ana Sayfa, P= Ürünler, D= Ürün Detayı, S= Sepet, O= Ödeme, B= Blog.

Oturum ID	Erişilen Sayfalar
t_1	{A, P, D, S}
t_2	{A, P, S}
t_3	{A, B}
t_4	{A, P, D, S, O}
t_5	{P, D, S}
t_6	{A, P, D, O}
t_7	{A, B, P}
t_8	{A, P, D, S}
t_9	{P, S}
t_{10}	{A, P, D, S, O}

a) Minimum destek eşiği $\sigma = 0.30$ (yani mutlak destek ≥ 3) için tüm sık öğe kümelerini (frequent itemsets) Apriori yaklaşımıyla adım adım keşfediniz. Her seviyede (1-öğe, 2-öğe, 3-öğe kümeleri) adayları ve sık olanları gösteriniz.

b) $\{A, P\} \Rightarrow \{S\}$ ve $\{D\} \Rightarrow \{S\}$ birliktelik kuralları için destek (support) ve güven (confidence) değerlerini hesaplayınız. Hesaplama formüllerini açıkça yazınız.

c) Bu iki kuralı destek ve güven değerleri açısından karşılaştırmız. Her bir kuralın e-ticaret bağlamında ne anlama geldiğini yorumlayınız. Sitenin çapraz satış (cross-selling) stratejisi için hangi kural daha faydalı olabilir?

Çözüm

Adım 1: Sık 1-öğe kümelerinin keşfi.

Toplam oturum sayısı $|T| = 10$, minimum mutlak destek eşiği $\lceil 0.30 \times 10 \rceil = 3$ 'tür. Her bir tekil sayfanın hangi oturumlarda geçtiğini ve destek değerini hesaplayalım.

A sayfası: $t_1, t_2, t_3, t_4, t_6, t_7, t_8, t_{10}$ olmak üzere 8 oturumda geçer.

$$\text{support}(A) = \frac{8}{10} = 0.80 \quad (180)$$

P sayfası: $t_1, t_2, t_4, t_5, t_6, t_7, t_8, t_9, t_{10}$ olmak üzere 9 oturumda geçer.

$$\text{support}(P) = \frac{9}{10} = 0.90 \quad (181)$$

D sayfası: $t_1, t_4, t_5, t_6, t_8, t_{10}$ olmak üzere 6 oturumda geçer.

$$\text{support}(D) = \frac{6}{10} = 0.60 \quad (182)$$

S sayfası: $t_1, t_2, t_4, t_5, t_8, t_9, t_{10}$ olmak üzere 7 oturumda geçer.

$$\text{support}(S) = \frac{7}{10} = 0.70 \quad (183)$$

O sayfası: t_4, t_6, t_{10} olmak üzere 3 oturumda geçer.

$$\text{support}(O) = \frac{3}{10} = 0.30 \quad (184)$$

B sayfası: t_3, t_7 olmak üzere 2 oturumda geçer.

$$\text{support}(B) = \frac{2}{10} = 0.20 \quad (\text{eşik altı, elenir}) \quad (185)$$

Sık 1-öge kümeleri: $\{A\}$ (0.80), $\{P\}$ (0.90), $\{D\}$ (0.60), $\{S\}$ (0.70), $\{O\}$ (0.30).

Adım 2: Sık 2-öge kümelerinin keşfi.

Sık 1-öge kümelerinden 2-öge adayları oluşturulur ve veri tabanı taranır.

$\{A, P\}$: $t_1, t_2, t_4, t_6, t_7, t_8, t_{10}$ (7 oturum), destek = 0.70, sık.

$\{A, D\}$: $t_1, t_4, t_6, t_8, t_{10}$ (5 oturum), destek = 0.50, sık.

$\{A, S\}$: $t_1, t_2, t_4, t_8, t_{10}$ (5 oturum), destek = 0.50, sık.

$\{A, O\}$: t_4, t_6, t_{10} (3 oturum), destek = 0.30, sık.

$\{P, D\}$: $t_1, t_4, t_5, t_6, t_8, t_{10}$ (6 oturum), destek = 0.60, sık.

$\{P, S\}$: $t_1, t_2, t_4, t_5, t_8, t_9, t_{10}$ (7 oturum), destek = 0.70, sık.

$\{P, O\}$: t_4, t_6, t_{10} (3 oturum), destek = 0.30, sık.

$\{D, S\}$: $t_1, t_4, t_5, t_8, t_{10}$ (5 oturum), destek = 0.50, sık.

$\{D, O\}$: t_4, t_6, t_{10} (3 oturum), destek = 0.30, sık.

$\{S, O\}$: t_4, t_{10} (2 oturum), destek = 0.20, elenir.

Sık 2-öge kümeleri: $\{A, P\}$, $\{A, D\}$, $\{A, S\}$, $\{A, O\}$, $\{P, D\}$, $\{P, S\}$, $\{P, O\}$, $\{D, S\}$, $\{D, O\}$.

Adım 3: Sık 3-öge ve 4-öge kümelerinin keşfi.

Sık 3-öge adayları oluşturulur. Apriori prensibi gereği, bir 3-öge kümesinin tüm 2-öge alt kümeleri sık olmalıdır.

$\{A, P, D\}$ için alt kümeler $\{A, P\}$, $\{A, D\}$, $\{P, D\}$ hepsi sıktır, adaydır. Geçtiği oturumlar: $t_1, t_4, t_6, t_8, t_{10}$ (5 oturum).

$$\text{support}(\{A, P, D\}) = \frac{5}{10} = 0.50 \quad (\text{sık}) \quad (186)$$

$\{A, P, S\}$: $t_1, t_2, t_4, t_8, t_{10}$ (5 oturum), destek = 0.50, sık.

$\{A, P, O\}$: t_4, t_6, t_{10} (3 oturum), destek = 0.30, sık.

$\{A, D, S\}$: t_1, t_4, t_8, t_{10} (4 oturum), destek = 0.40, sık.

$\{A, D, O\}$: t_4, t_6, t_{10} (3 oturum), destek = 0.30, sık.

$\{P, D, S\}$: $t_1, t_4, t_5, t_8, t_{10}$ (5 oturum), destek = 0.50, sık.

$\{P, D, O\}$: t_4, t_6, t_{10} (3 oturum), destek = 0.30, sık.

$\{A, S, O\}$ alt kümelerinden $\{S, O\}$ sık olmadığı için elenir.

$\{P, S, O\}$ alt kümelerinden $\{S, O\}$ sık olmadığı için elenir.

$\{D, S, O\}$ alt kümelerinden $\{S, O\}$ sık olmadığı için elenir.

Sık 3-öge kümeleri: $\{A, P, D\}$, $\{A, P, S\}$, $\{A, P, O\}$, $\{A, D, S\}$, $\{A, D, O\}$, $\{P, D, S\}$, $\{P, D, O\}$.

Şimdi 4-öge adaylarına bakalım. $\{A, P, D, S\}$ aday olabilir çünkü tüm 3-öge alt kümeleri sıktır ve $\{S, O\}$ içermez.

$\{A, P, D, S\}$: t_1, t_4, t_8, t_{10} (4 oturum).

$$\text{support}(\{A, P, D, S\}) = \frac{4}{10} = 0.40 \quad (\text{sık}) \quad (187)$$

$\{A, P, D, O\}$: t_4, t_6, t_{10} (3 oturum).

$$\text{support}(\{A, P, D, O\}) = \frac{3}{10} = 0.30 \quad (\text{sık}) \quad (188)$$

$\{A, P, O\}$ içeren diğer kombinasyonlar $\{S, O\}$ gerektirdiğinden elenir.

Sık 4-öge kümeleri: $\{A, P, D, S\}$ (0.40) ve $\{A, P, D, O\}$ (0.30).

5-öge adayı $\{A, P, D, S, O\}$ için, $\{S, O\}$ alt kümesi sık olmadığından (destek 0.20) aday dahi olamaz. Böylece keşif tamamlanır.

Adım 4: Birliktelik kurallarının destek ve güven değerlerinin hesaplanması.

$\{A, P\} \Rightarrow \{S\}$ kuralı için:

$$\text{support}(\{A, P\} \Rightarrow \{S\}) = \text{support}(\{A, P, S\}) = \frac{|\{t_1, t_2, t_4, t_8, t_{10}\}|}{10} = \frac{5}{10} = 0.50 \quad (189)$$

$$\text{confidence}(\{A, P\} \Rightarrow \{S\}) = \frac{\text{support}(\{A, P, S\})}{\text{support}(\{A, P\})} = \frac{0.50}{0.70} = 0.7143 \quad (190)$$

$\{D\} \Rightarrow \{S\}$ kuralı için:

$$\text{support}(\{D\} \Rightarrow \{S\}) = \text{support}(\{D, S\}) = \frac{|\{t_1, t_4, t_5, t_8, t_{10}\}|}{10} = \frac{5}{10} = 0.50 \quad (191)$$

$$\text{confidence}(\{D\} \Rightarrow \{S\}) = \frac{\text{support}(\{D, S\})}{\text{support}(\{D\})} = \frac{0.50}{0.60} = 0.8333 \quad (192)$$

Adım 5: Kuralların karşılaştırılması ve yorumlanması.

$\{A, P\} \Rightarrow \{S\}$ kuralının desteği 0.50 ve güveni 0.7143'tür. Bu kural, ana sayfa ve ürünler sayfasını ziyaret eden oturumların yaklaşık %71.43'ünde sepet sayfasına da erişildiğini ifade eder. Başka bir deyişle, siteye girip ürünlere göz atan her 10 kullanıcıdan yaklaşık 7'si sepete bir şeyler eklemektedir. Bu, sitenin genel dönüşüm performansı hakkında fikir veren makro düzeyde bir kuraldır.

$\{D\} \Rightarrow \{S\}$ kuralının desteği yine 0.50, ancak güveni 0.8333'tür. Bu kural, bir ürün detay sayfasını görüntüleyen oturumların %83.33'ünde sepet sayfasına da erişildiğini ifade eder. $\{D\} \Rightarrow \{S\}$ kuralının güveni, $\{A, P\} \Rightarrow \{S\}$ kuralından belirgin şekilde daha yüksektir. Bunun nedeni, ürün detay sayfasına ulaşan kullanıcıların satın alma niyetinin daha somut olmasıdır; ana sayfa ve ürünler sayfası daha genel keşif sayfalarıyken, ürün detay sayfası karar aşamasına yaklaşmış kullanıcıları temsil eder.

E-ticaret çapraz satış stratejisi açısından $\{D\} \Rightarrow \{S\}$ kuralı daha faydalı olabilir. Bunun nedeni, bu kuralın daha spesifik bir tetikleyiciye (ürün detay sayfası görüntüleme) dayanması ve daha yüksek güvenle sepet erişimini öngörmesidir. Site, ürün detay sayfasında kullanıcıya sepete ekleme çağrısını daha belirgin hale getirerek veya "bu ürünü alanlar şunları da aldı" türünde çapraz satış önerileri sunarak bu yüksek güvenli kuraldan stratejik olarak yararlanabilir.

Buna karşılık $\{A, P\} \Rightarrow \{S\}$ kuralı daha genel bir kullanıcı kesiti için geçerli olduğundan, site geneli dönüşüm hunisi optimizasyonunda kullanılabilir; ancak bireysel kullanıcıya anlık müdahale için yeterince spesifik değildir.

6 Tavsiye Sistemleri — Sorular

Soru 1: İçerik Tabanlı Filtreleme — TF-IDF Madde Profili ve Kosinüs Benzerliği ile Tahmin

Bir film tavsiye sisteminde filmler, konu özetlerindeki kelimeler kullanılarak temsil edilmektedir. Sistemde üç film ve bu filmlere ait konu özetleri aşağıdaki gibidir. Dördüncü bir film (F_4) ise kullanıcıya tavsiye edilip edilmeyeceği değerlendirilecek hedef filmidir.

F_1 : “bilim kurgu uzay” F_2 : “aşk dram aşk” F_3 : “bilim kurgu bilim” F_4 : “bilim uzay kurgu”

Kullanıcı x ; F_1 'e 4, F_2 'ye 2, F_3 'e 5 puan vermiştir (1–5 yıldız aralığında). Her bir F_j dokümanı için önce TF-IDF madde profil vektörünü oluşturunuz. Ardından kullanıcı x 'in profil vektörünü normalize edilmiş ağırlıklı ortalama ile hesaplayınız. Son olarak F_4 filminin kullanıcı x için tahmini faydasını kosinüs benzerliği ile bulunuz ve sonucu yorumlayınız. (Not: $IDF_i = \ln(N/n_i)$ kullanınız; $N = 3$ alınız.)

Çözüm:

Adım 1: Terim kümesinin (sözlük) belirlenmesi.

Belgelerde geçen tüm benzersiz terimler şunlardır: bilim, kurgu, uzay, aşk, dram. Beş terimli bir vektör uzayında çalışılacaktır. Vektörlerdeki boyut sıralaması: (bilim, kurgu, uzay, aşk, dram) olarak sabitlenecektir.

Adım 2: Her doküman için normalize TF değerlerinin hesaplanması.

Terim frekansı (TF), normalize edilmiş ham frekans olarak hesaplanır; ilgili terimin dokümanda geçme sayısı, dokümandaki toplam terim sayısına bölünür.

F_1 dokümanında üç terim vardır: bilim 1 kez, kurgu 1 kez, uzay 1 kez. Normalize TF değerleri:

$$TF_{F_1} = \left[\frac{1}{3}, \frac{1}{3}, \frac{1}{3}, 0, 0 \right] \quad (193)$$

F_2 dokümanında aşk 2 kez, dram 1 kez geçer; toplam terim sayısı 3'tür:

$$TF_{F_2} = \left[0, 0, 0, \frac{2}{3}, \frac{1}{3} \right] \quad (194)$$

F_3 dokümanında bilim 2 kez, kurgu 1 kez geçer; toplam 3 terim:

$$TF_{F_3} = \left[\frac{2}{3}, \frac{1}{3}, 0, 0, 0 \right] \quad (195)$$

F_4 hedef dokümanında bilim, uzay ve kurgu birer kez geçer:

$$TF_{F_4} = \left[\frac{1}{3}, \frac{1}{3}, \frac{1}{3}, 0, 0 \right] \quad (196)$$

Adım 3: Her terim için IDF değerlerinin hesaplanması.

Eğitim kümemiz F_1 , F_2 ve F_3 'ten oluştuğundan $N = 3$ alınır.

“bilim” terimi F_1 ve F_3 'te olmak üzere $n_i = 2$ dokümanda geçer:

$$IDF_{\text{bilim}} = \ln\left(\frac{3}{2}\right) = \ln(1.5) \approx 0.405 \quad (197)$$

“kurgu” terimi de F_1 ve F_3 'te geçtiğinden:

$$IDF_{\text{kurgu}} = \ln\left(\frac{3}{2}\right) \approx 0.405 \quad (198)$$

“uzay” terimi yalnızca F_1 'de geçer ($n_i = 1$):

$$IDF_{\text{uzay}} = \ln\left(\frac{3}{1}\right) = \ln(3) \approx 1.099 \quad (199)$$

“aşk” terimi yalnızca F_2 ’de geçtiğinden:

$$IDF_{\text{aşk}} = \ln(3) \approx 1.099 \quad (200)$$

“dram” terimi de yalnızca F_2 ’de geçtiğinden:

$$IDF_{\text{dram}} = \ln(3) \approx 1.099 \quad (201)$$

Adım 4: TF-IDF madde profil vektörlerinin oluşturulması.

TF-IDF skoru $w_{ij} = TF_{ij} \times IDF_i$ formülüyle hesaplanır. Her doküman için beş boyutlu profil vektörleri:

$$\begin{aligned} \vec{F}_1 &= [0.135, 0.135, 0.366, 0, 0] \\ \vec{F}_2 &= [0, 0, 0, 0.733, 0.366] \\ \vec{F}_3 &= [0.270, 0.135, 0, 0, 0] \\ \vec{F}_4 &= [0.135, 0.135, 0.366, 0, 0] \end{aligned} \quad (202)$$

Adım 5: Kullanıcı profil vektörünün normalize edilmiş ağırlıklı ortalama ile hesaplanması.

İlk olarak kullanıcının ortalama puanı hesaplanır:

$$\bar{r}_x = \frac{4 + 2 + 5}{3} = \frac{11}{3} = 3.667 \quad (203)$$

Her bir filmin normalize ağırlığı, puanından kullanıcı ortalaması çıkarılarak bulunur:

$$w_1 = 4 - 3.667 = 0.333 \quad (204)$$

$$w_2 = 2 - 3.667 = -1.667 \quad (205)$$

$$w_3 = 5 - 3.667 = 1.333 \quad (206)$$

Kullanıcı profil vektörü $\vec{x}$, derecelendirilen maddelerin profil vektörlerinin bu normalize ağırlıklarla çarpılıp toplanmasıyla elde edilir:

$$\begin{aligned} \vec{x} &= 0.333 \cdot \vec{F}_1 + (-1.667) \cdot \vec{F}_2 + 1.333 \cdot \vec{F}_3 \\ &= [0.333(0.135) + 0 + 1.333(0.270), \\ &\quad 0.333(0.135) + 0 + 1.333(0.135), \\ &\quad 0.333(0.366) + 0 + 0, \\ &\quad 0 + (-1.667)(0.733) + 0, \\ &\quad 0 + (-1.667)(0.366) + 0] \\ &= [0.045 + 0.360, 0.045 + 0.180, 0.122, -1.222, -0.610] \\ &= [0.405, 0.225, 0.122, -1.222, -0.610] \end{aligned} \quad (207)$$

Görüldüğü gibi kullanıcı profilinde “bilim”, “kurgu” ve “uzay” terimleri pozitif ağırlığa sahipken (kullanıcının sevdiği içerikler), “aşk” ve “dram” terimleri negatif ağırlığa sahiptir (sevmediği içerikler). F_2 filminin düşük puanlanmış olması bu terimleri negatife çekmiştir.

Adım 6: Kosinüs benzerliği ile tahminin yapılması.

$\vec{x}$ kullanıcı profili ile $\vec{F}_4$ hedef film profili arasındaki kosinüs benzerliği hesaplanır:

$$\cos(\vec{x}, \vec{F}_4) = \frac{\vec{x} \cdot \vec{F}_4}{|\vec{x}| |\vec{F}_4|} \quad (208)$$

Nokta çarpımı adım adım hesaplanır:

$$\vec{x} \cdot \vec{F}_4 = 0.405 \times 0.135 + 0.225 \times 0.135 + 0.122 \times 0.366 + (-1.222) \times 0 + (-0.610) \times 0 \quad (209)$$

$$\vec{x} \cdot \vec{F}_4 = 0.055 + 0.030 + 0.045 = 0.130 \quad (210)$$

Kullanıcı vektörünün büyüklüğü hesaplanır:

$$|\vec{x}| = \sqrt{0.405^2 + 0.225^2 + 0.122^2 + (-1.222)^2 + (-0.610)^2} \quad (211)$$

$$|\vec{x}| = \sqrt{0.164 + 0.051 + 0.015 + 1.493 + 0.372} = \sqrt{2.095} \approx 1.447 \quad (212)$$

Hedef film vektörünün büyüklüğü hesaplanır:

$$|\vec{F}_4| = \sqrt{0.135^2 + 0.135^2 + 0.366^2 + 0^2 + 0^2} = \sqrt{0.018 + 0.018 + 0.134} = \sqrt{0.170} \approx 0.412 \quad (213)$$

Kosinüs benzerliği:

$$\cos(\vec{x}, \vec{F}_4) = \frac{0.130}{1.447 \times 0.412} = \frac{0.130}{0.596} \approx 0.218 \quad (214)$$

Pozitif bir kosinüs benzerliği (0.218) elde edilmiştir. Bu değer, F_4 filminin kullanıcının beğeni profiliyle kısmen örtüştüğünü, ancak mükemmel bir eşleşme (+1) olmadığını gösterir. F_4 ; “bilim”, “kurgu” ve “uzay” terimlerini içerdiğinden kullanıcının sevdiği içeriklere benzerlik göstermekte, fakat “dram” ve “aşk” terimlerini içermediğinden negatif bileşenlerle çakışma yaşanmamaktadır. Sonuç olarak sistem bu filmi kullanıcıya orta düzeyde bir tavsiye olarak sunabilir.

Soru 2: Kullanıcı-Kullanıcı İşbirlikçi Filtreleme — Pearson Korelasyonu ve Puan Tahmini

Aşağıda 4 kullanıcı ve 5 filmten oluşan fayda matrisi verilmiştir. Kullanıcı U_1 ’in M_5 filmi için tahmini puanını ($\hat{r}_{U_1, M_5}$), kullanıcı-kullanıcı işbirlikçi filtreleme yaklaşımıyla hesaplayınız. Benzerlik metriği olarak Pearson korelasyonunu kullanınız; en benzer $k = 2$ komşuyu seçerek ağırlıklı ortalama ile tahmin yapınız. Tüm ara adımları (ortalama hesapları, sapmalar, benzerlik skorları, komşu seçimi ve nihai tahmin) gösteriniz.

	M_1	M_2	M_3	M_4	M_5
U_1	5	3	4	4	?
U_2	3	1	2	3	3
U_3	4	3	4	3	5
U_4	2	4	1	2	4

Çözüm:**Adım 1: Her kullanıcının ortalama puanının hesaplanması.**

Pearson korelasyonu, her kullanıcının derecelendirmelerinden kendi ortalamasının çıkarılmasıyla elde edilen normalize vektörlerin kosinüs benzerliğidir. Ortalamalar yalnızca bilinen puanlar üzerinden hesaplanır.

U_1 kullanıcısı M_1-M_4 arasındaki dört filmi derecelendirmiştir:

$$\bar{r}_{U_1} = \frac{5 + 3 + 4 + 4}{4} = \frac{16}{4} = 4 \quad (215)$$

U_2 kullanıcısı tüm beş filmi derecelendirmiştir:

$$\bar{r}_{U_2} = \frac{3 + 1 + 2 + 3 + 3}{5} = \frac{12}{5} = 2.4 \quad (216)$$

U_3 kullanıcısı tüm beş filmi derecelendirmiştir:

$$\bar{r}_{U_3} = \frac{4 + 3 + 4 + 3 + 5}{5} = \frac{19}{5} = 3.8 \quad (217)$$

U_4 kullanıcısı tüm beş filmi derecelendirmiştir:

$$\bar{r}_{U_4} = \frac{2 + 4 + 1 + 2 + 4}{5} = \frac{13}{5} = 2.6 \quad (218)$$

Adım 2: U_1 ile U_2 arasındaki Pearson korelasyonunun hesaplanması.

Ortak filmler M_1, M_2, M_3, M_4 'tür. U_1 'in bu filmler için ortalamadan sapmaları:

$$5 - 4 = 1, \quad 3 - 4 = -1, \quad 4 - 4 = 0, \quad 4 - 4 = 0 \quad (219)$$

Sapma vektörü: $[1, -1, 0, 0]$.

U_2 'nin aynı filmler için sapmaları:

$$3 - 2.4 = 0.6, \quad 1 - 2.4 = -1.4, \quad 2 - 2.4 = -0.4, \quad 3 - 2.4 = 0.6 \quad (220)$$

Sapma vektörü: $[0.6, -1.4, -0.4, 0.6]$.

Pearson korelasyon formülü uygulanır:

$$\text{sim}(U_1, U_2) = \frac{1 \times 0.6 + (-1) \times (-1.4) + 0 \times (-0.4) + 0 \times 0.6}{\sqrt{1^2 + (-1)^2 + 0^2 + 0^2} \sqrt{0.6^2 + (-1.4)^2 + (-0.4)^2 + 0.6^2}} \quad (221)$$

Pay:

$$0.6 + 1.4 + 0 + 0 = 2.0 \quad (222)$$

Paydanın ilk karekökü:

$$\sqrt{2} \approx 1.414 \quad (223)$$

Paydanın ikinci karekökü:

$$\sqrt{0.36 + 1.96 + 0.16 + 0.36} = \sqrt{2.84} \approx 1.685 \quad (224)$$

Benzerlik skoru:

$$\text{sim}(U_1, U_2) = \frac{2.0}{1.414 \times 1.685} = \frac{2.0}{2.383} \approx 0.839 \quad (225)$$

Adım 3: U_1 ile U_3 arasındaki Pearson korelasyonunun hesaplanması.

Ortak filmler yine M_1-M_4 'tür. U_3 'ün bu filmlerdeki sapmaları:

$$4 - 3.8 = 0.2, \quad 3 - 3.8 = -0.8, \quad 4 - 3.8 = 0.2, \quad 3 - 3.8 = -0.8 \quad (226)$$

Sapma vektörü: $[0.2, -0.8, 0.2, -0.8]$.

Nokta çarpımı:

$$1 \times 0.2 + (-1) \times (-0.8) + 0 \times 0.2 + 0 \times (-0.8) = 0.2 + 0.8 = 1.0 \quad (227)$$

U_3 sapmalarının kareleri toplamı:

$$0.04 + 0.64 + 0.04 + 0.64 = 1.36 \quad (228)$$

Karekökü:

$$\sqrt{1.36} \approx 1.166 \quad (229)$$

Benzerlik skoru:

$$\text{sim}(U_1, U_3) = \frac{1.0}{1.414 \times 1.166} = \frac{1.0}{1.649} \approx 0.607 \quad (230)$$

Adım 4: U_1 ile U_4 arasındaki Pearson korelasyonunun hesaplanması.

Ortak filmler yine M_1-M_4 'tür. U_4 'ün sapmaları:

$$2 - 2.6 = -0.6, \quad 4 - 2.6 = 1.4, \quad 1 - 2.6 = -1.6, \quad 2 - 2.6 = -0.6 \quad (231)$$

Sapma vektörü: $[-0.6, 1.4, -1.6, -0.6]$.

Nokta çarpımı:

$$1 \times (-0.6) + (-1) \times 1.4 + 0 \times (-1.6) + 0 \times (-0.6) = -0.6 - 1.4 = -2.0 \quad (232)$$

U_4 sapma kareleri toplamı:

$$0.36 + 1.96 + 2.56 + 0.36 = 5.24 \quad (233)$$

Karekökü:

$$\sqrt{5.24} \approx 2.289 \quad (234)$$

Benzerlik skoru:

$$\text{sim}(U_1, U_4) = \frac{-2.0}{1.414 \times 2.289} = \frac{-2.0}{3.237} \approx -0.618 \quad (235)$$

Adım 5: En benzer $k = 2$ komşunun seçilmesi ve tahminin yapılması.

Elde edilen benzerlik skorları:

$$\text{sim}(U_1, U_2) \approx 0.839, \quad \text{sim}(U_1, U_3) \approx 0.607, \quad \text{sim}(U_1, U_4) \approx -0.618 \quad (236)$$

En benzer $k = 2$ komşu pozitif ve en yüksek skorlu olan U_2 ve U_3 'tür. U_4 negatif korelasyona sahip olduğundan komşu kümesine alınmaz.

Tahmin formülü ağırlıklı ortalama ile uygulanır:

$$\hat{r}_{U_1, M_5} = \frac{\text{sim}(U_1, U_2) \cdot r_{U_2, M_5} + \text{sim}(U_1, U_3) \cdot r_{U_3, M_5}}{\text{sim}(U_1, U_2) + \text{sim}(U_1, U_3)} \quad (237)$$

$$\hat{r}_{U_1, M_5} = \frac{0.839 \times 3 + 0.607 \times 5}{0.839 + 0.607} = \frac{2.517 + 3.035}{1.446} = \frac{5.552}{1.446} \approx 3.84 \quad (238)$$

Kullanıcı U_1 'in M_5 filmi için tahmini puanı yaklaşık 3.84 olarak bulunur. Bu değer 1–5 yıldız aralığında orta-üst düzeyde bir beğeniyi işaret etmektedir. U_2 ve U_3 kullanıcıları U_1 ile yüksek pozitif korelasyon gösterdiğinden, onların M_5 hakkındaki görüşleri (3 ve 5) güvenilir bir tahmin zemini oluşturmuştur.

Soru 3: Madde-Madde İşbirlikçi Filtreleme — Tam Sayısal Örnek

Aşağıda 5 kullanıcı ve 5 filmden oluşan fayda matrisi verilmiştir. Madde-madde işbirlikçi filtreleme yöntemiyle Kullanıcı U_3 'ün M_3 filmi için tahmini puanını hesaplayınız. Şu adımları sırasıyla gerçekleştiriniz: (1) her filmin ortalama puanını hesaplayıp puan vektörlerini ortalama ile merkezleyiniz, (2) M_3 ile diğer filmler arasındaki Pearson korelasyonunu (merkezlenmiş kosinüs benzerliğini) hesaplayınız, (3) pozitif benzerliğe sahip komşu filmleri belirleyip ağırlıklı ortalama ile U_3 için M_3 tahminini yapınız.

	M_1	M_2	M_3	M_4	M_5
U_1	5	4	5	2	1
U_2	4	5	4	1	2
U_3	5	4	?	2	1
U_4	3	5	4	1	2
U_5	4	3	4	2	1

Çözüm:

Adım 1: Her filmin ortalama puanının hesaplanması.

Ortalamalar yalnızca bilinen puanlar üzerinden hesaplanır.

M_1 filmini beş kullanıcının tamamı derecelendirmiştir:

$$\bar{r}_1 = \frac{5 + 4 + 5 + 3 + 4}{5} = \frac{21}{5} = 4.2 \quad (239)$$

M_2 filmini beş kullanıcının tamamı derecelendirmiştir:

$$\bar{r}_2 = \frac{4 + 5 + 4 + 5 + 3}{5} = \frac{21}{5} = 4.2 \quad (240)$$

M_3 filmini yalnızca U_1, U_2, U_4, U_5 derecelendirmiştir:

$$\bar{r}_3 = \frac{5 + 4 + 4 + 4}{4} = \frac{17}{4} = 4.25 \quad (241)$$

M_4 filmini beş kullanıcının tamamı derecelendirmiştir:

$$\bar{r}_4 = \frac{2 + 1 + 2 + 1 + 2}{5} = \frac{8}{5} = 1.6 \quad (242)$$

M_5 filmini beş kullanıcının tamamı derecelendirmiştir:

$$\bar{r}_5 = \frac{1 + 2 + 1 + 2 + 1}{5} = \frac{7}{5} = 1.4 \quad (243)$$

Adım 2: Normalize edilmiş (ortalama ile merkezlenmiş) puan vektörlerinin oluşturulması.

Bilinmeyen değerlere 0 atanır. Beş kullanıcı sıralamasıyla (U_1 – U_5) normalize vektörler:

$$\begin{aligned}
\vec{M}_1^{norm} &= [5 - 4.2, 4 - 4.2, 5 - 4.2, 3 - 4.2, 4 - 4.2] = [0.8, -0.2, 0.8, -1.2, -0.2] \\
\vec{M}_2^{norm} &= [4 - 4.2, 5 - 4.2, 4 - 4.2, 5 - 4.2, 3 - 4.2] = [-0.2, 0.8, -0.2, 0.8, -1.2] \\
\vec{M}_3^{norm} &= [5 - 4.25, 4 - 4.25, 0, 4 - 4.25, 4 - 4.25] = [0.75, -0.25, 0, -0.25, -0.25] \\
\vec{M}_4^{norm} &= [2 - 1.6, 1 - 1.6, 2 - 1.6, 1 - 1.6, 2 - 1.6] = [0.4, -0.6, 0.4, -0.6, 0.4] \\
\vec{M}_5^{norm} &= [1 - 1.4, 2 - 1.4, 1 - 1.4, 2 - 1.4, 1 - 1.4] = [-0.4, 0.6, -0.4, 0.6, -0.4] \quad (244)
\end{aligned}$$

Adım 3: M_3 ile M_1 arasındaki benzerliğin hesaplanması.

Nokta çarpımı:

$$\vec{M}_3^{norm} \cdot \vec{M}_1^{norm} = 0.75 \times 0.8 + (-0.25) \times (-0.2) + 0 \times 0.8 + (-0.25) \times (-1.2) + (-0.25) \times (-0.2) \quad (245)$$

$$= 0.60 + 0.05 + 0 + 0.30 + 0.05 = 1.00 \quad (246)$$

M_3 normalize vektörünün büyüklüğü:

$$|\vec{M}_3^{norm}| = \sqrt{0.75^2 + (-0.25)^2 + 0^2 + (-0.25)^2 + (-0.25)^2} = \sqrt{0.5625 + 0.0625 + 0 + 0.0625 + 0.0625} = \sqrt{0.75} \quad (247)$$

M_1 normalize vektörünün büyüklüğü:

$$|\vec{M}_1^{norm}| = \sqrt{0.8^2 + (-0.2)^2 + 0.8^2 + (-1.2)^2 + (-0.2)^2} = \sqrt{0.64 + 0.04 + 0.64 + 1.44 + 0.04} = \sqrt{2.80} \approx 1.673 \quad (248)$$

Benzerlik skoru:

$$sim(M_3, M_1) = \frac{1.00}{0.866 \times 1.673} = \frac{1.00}{1.449} \approx 0.690 \quad (249)$$

Adım 4: M_3 ile M_2 arasındaki benzerliğin hesaplanması.

Nokta çarpımı:

$$0.75 \times (-0.2) + (-0.25) \times 0.8 + 0 \times (-0.2) + (-0.25) \times 0.8 + (-0.25) \times (-1.2) \quad (250)$$

$$= -0.15 - 0.20 + 0 - 0.20 + 0.30 = -0.25 \quad (251)$$

M_2 'nin büyüklüğü M_1 ile aynıdır ($\sqrt{2.80} \approx 1.673$).

$$sim(M_3, M_2) = \frac{-0.25}{0.866 \times 1.673} = \frac{-0.25}{1.449} \approx -0.173 \quad (252)$$

Adım 5: M_3 ile M_4 arasındaki benzerliğin hesaplanması.

Nokta çarpımı:

$$0.75 \times 0.4 + (-0.25) \times (-0.6) + 0 \times 0.4 + (-0.25) \times (-0.6) + (-0.25) \times 0.4 \quad (253)$$

$$= 0.30 + 0.15 + 0 + 0.15 - 0.10 = 0.50 \quad (254)$$

M_4 'ün büyüklüğü:

$$|\vec{M}_4^{norm}| = \sqrt{0.4^2 + (-0.6)^2 + 0.4^2 + (-0.6)^2 + 0.4^2} = \sqrt{0.16 + 0.36 + 0.16 + 0.36 + 0.16} = \sqrt{1.20} \approx 1.095 \quad (255)$$

$$sim(M_3, M_4) = \frac{0.50}{0.866 \times 1.095} = \frac{0.50}{0.948} \approx 0.527 \quad (256)$$

Adım 6: M_3 ile M_5 arasındaki benzerliğin hesaplanması.

Nokta çarpımı:

$$0.75 \times (-0.4) + (-0.25) \times 0.6 + 0 \times (-0.4) + (-0.25) \times 0.6 + (-0.25) \times (-0.4) \quad (257)$$

$$= -0.30 - 0.15 + 0 - 0.15 + 0.10 = -0.50 \quad (258)$$

M_5 'in büyüklüğü M_4 ile aynıdır ($\sqrt{1.20} \approx 1.095$).

$$sim(M_3, M_5) = \frac{-0.50}{0.866 \times 1.095} = \frac{-0.50}{0.948} \approx -0.527 \quad (259)$$

Adım 7: Komşu filmlerin belirlenmesi ve tahminin yapılması.

Benzerlik skorları özetlenirse:

$$sim(M_3, M_1) \approx 0.690, \quad sim(M_3, M_2) \approx -0.173, \quad sim(M_3, M_4) \approx 0.527, \quad sim(M_3, M_5) \approx -0.527 \quad (260)$$

Pozitif benzerliğe sahip komşu filmler M_1 ve M_4 'tür. Kullanıcı U_3 bu filmlerin her ikisini de derecelendirmiştir: $r_{U_3, M_1} = 5$ ve $r_{U_3, M_4} = 2$.

Tahmin, pozitif benzerlikli komşuların ağırlıklı ortalaması ile yapılır:

$$\hat{r}_{U_3, M_3} = \frac{sim(M_3, M_1) \cdot r_{U_3, M_1} + sim(M_3, M_4) \cdot r_{U_3, M_4}}{sim(M_3, M_1) + sim(M_3, M_4)} \quad (261)$$

$$\hat{r}_{U_3, M_3} = \frac{0.690 \times 5 + 0.527 \times 2}{0.690 + 0.527} = \frac{3.450 + 1.054}{1.217} = \frac{4.504}{1.217} \approx 3.70 \quad (262)$$

Kullanıcı U_3 'ün M_3 filmi için tahmini puanı yaklaşık 3.70 olarak hesaplanır. Bu sonuç anlamlıdır: M_3 en çok M_1 'e benzemektedir (0.690) ve U_3 , M_1 'e yüksek puan (5) vermiştir. İkincil olarak M_4 ile de pozitif benzerlik (0.527) göstermekte, ancak U_3 'ün M_4 puanı daha düşüktür (2). Ağırlıklı ortalama bu iki etkiyi dengeleyerek 3.70 gibi orta-üst bir tahmin üretmiştir. M_3 ile M_2 ve M_5 arasındaki negatif korelasyonlar ise tahmine dahil edilmemiştir; bu filmler M_3 'ün derecelendirme örüntüsüne ters düşmektedir.

Soru 4: RMSE (Kök Ortalama Kare Hatası) Hesaplama

Bir tavsiye sisteminin test kümesi üzerinde yaptığı tahminler ile gerçek kullanıcı puanları aşağıdaki tabloda verilmiştir. Bu tahminler için RMSE (Root Mean Square Error) değerini hesaplayınız. Her bir test örneği için hatayı (tahmin edilen – gerçek) ayrı ayrı gösteriniz ve nihai RMSE değerini bulunuz. RMSE değerinin ne anlama geldiğini kısaca yorumlayınız.

Test No	Kullanıcı-Film	Tahmin Edilen ($\hat{r}$)	Gerçek (r^*)
1	(U_1, M_3)	4.2	5
2	(U_1, M_7)	3.1	3
3	(U_2, M_1)	2.5	2
4	(U_2, M_5)	4.8	4
5	(U_3, M_2)	3.0	4
6	(U_3, M_9)	1.7	1
7	(U_4, M_4)	4.5	5
8	(U_4, M_8)	2.2	2

Çözüm:

Adım 1: RMSE formülünün tanımlanması.

RMSE, tahmin edilen puanlarla gerçek puanlar arasındaki farkların karelerinin ortalamasının karekökü olarak tanımlanır:

$$RMSE = \sqrt{\frac{1}{N} \sum_{(x,i) \in T} (\hat{r}_{xi} - r_{xi}^*)^2} \quad (263)$$

Burada $N = 8$ test örneği bulunmaktadır.

Adım 2: Her bir test örneği için hata ve kare hatanın hesaplanması.

Test 1 için:

$$e_1 = \hat{r} - r^* = 4.2 - 5 = -0.8 \quad (264)$$

$$e_1^2 = (-0.8)^2 = 0.64 \quad (265)$$

Test 2 için:

$$e_2 = 3.1 - 3 = 0.1 \quad (266)$$

$$e_2^2 = (0.1)^2 = 0.01 \quad (267)$$

Test 3 için:

$$e_3 = 2.5 - 2 = 0.5 \quad (268)$$

$$e_3^2 = (0.5)^2 = 0.25 \quad (269)$$

Test 4 için:

$$e_4 = 4.8 - 4 = 0.8 \quad (270)$$

$$e_4^2 = (0.8)^2 = 0.64 \quad (271)$$

Test 5 için:

$$e_5 = 3.0 - 4 = -1.0 \quad (272)$$

$$e_5^2 = (-1.0)^2 = 1.00 \quad (273)$$

Test 6 için:

$$e_6 = 1.7 - 1 = 0.7 \quad (274)$$

$$e_6^2 = (0.7)^2 = 0.49 \quad (275)$$

Test 7 için:

$$e_7 = 4.5 - 5 = -0.5 \quad (276)$$

$$e_7^2 = (-0.5)^2 = 0.25 \quad (277)$$

Test 8 için:

$$e_8 = 2.2 - 2 = 0.2 \quad (278)$$

$$e_8^2 = (0.2)^2 = 0.04 \quad (279)$$

Adım 3: Kare hataların toplamı ve ortalamasının hesaplanması.

Kare hataların toplamı:

$$\sum e^2 = 0.64 + 0.01 + 0.25 + 0.64 + 1.00 + 0.49 + 0.25 + 0.04 = 3.32 \quad (280)$$

Ortalama kare hata:

$$\frac{1}{N} \sum e^2 = \frac{3.32}{8} = 0.415 \quad (281)$$

Adım 4: RMSE'nin hesaplanması ve yorumlanması.

$$RMSE = \sqrt{0.415} \approx 0.644 \quad (282)$$

Elde edilen RMSE değeri yaklaşık 0.644'tür. Bu değer, sistemin tahminlerinin gerçek puanlardan ortalama olarak yaklaşık 0.64 puan saptığını gösterir. 1-5 yıldız aralığında çalışan bir sistem için bu orta düzeyde bir hatadır.

En büyük bireysel hata Test 5'te görülmektedir (1.0 puanlık sapma); RMSE'nin kare alma işlemi nedeniyle bu büyük hata ($1.0^2 = 1.00$) toplam kare hataya önemli ölçüde katkıda bulunmuştur. Sistem genel olarak kabul edilebilir bir doğruluk sergilemekle birlikte, özellikle U_3 kullanıcısının M_2 filmi için yaptığı 1 puanlık hata gibi büyük sapmaların azaltılması RMSE'yi önemli ölçüde iyileştirecektir.

Soru 5: Precision@K (İlk K'da Hassasiyet) Hesaplama

Bir tavsiye sistemi, test kümesindeki bir kullanıcı için en yüksek tahmin ettiği $K = 5$ filmi aşağıdaki gibi sıralamıştır. Aynı kullanıcının test kümesinde gizlenmiş gerçek puanları da tabloda verilmiştir. Yüksek puan eşiği 4 ve üzeri olarak belirlenmiştir (yani gerçek puanı ≥ 4 olan filmler "ilgili" kabul edilmektedir). Bu kullanıcı için Precision@5 değerini hesaplayınız. Ayrıca eğer sistem yalnızca ilk $K = 3$ filmi önerseydi Precision@3 ne olurdu? Her iki durumu da gösteriniz.

Sıra ($K = 5$)	Tahmin Edilen Film	Kullanıcının Gerçek Puanı (r^*)
1	M_7	5
2	M_2	4
3	M_{11}	3
4	M_5	5
5	M_9	2

Çözüm:

Adım 1: Precision@K metriğinin tanımlanması.

Precision@K metriği, sistemin kullanıcıya önerdiği en iyi K madde arasında, kullanıcının gerçekten yüksek puan verdiği (eşik değerin üzerindeki) maddelerin oranını ölçer. Formel olarak:

$$Precision@K = \frac{|TopK \cap High|}{K} \quad (283)$$

Burada TopK, sistemin en yüksek tahmin ettiği K maddenin kümesi; High ise kullanıcının test kümesinde eşik değerin (≥ 4) üzerinde puan verdiği maddelerin kümesidir.

Adım 2: İlgili (yüksek puanlı) filmlerin belirlenmesi.

Tabloya göre gerçek puanlar: M_7 için 5, M_2 için 4, M_{11} için 3, M_5 için 5, M_9 için 2'dir.

Eşik ≥ 4 olduğuna göre yüksek puanlı (ilgili) filmler M_7 , M_2 ve M_5 olmak üzere üç tanedir:

$$High = \{M_7, M_2, M_5\} \quad (284)$$

Adım 3: Precision@5'in hesaplanması.

Sistemin önerdiği ilk 5 filmin tamamı değerlendirilir:

$$Top5 = \{M_7, M_2, M_{11}, M_5, M_9\} \quad (285)$$

Kesişim:

$$Top5 \cap High = \{M_7, M_2, M_5\} \quad (286)$$

Kesişimde 3 adet film bulunur:

$$Precision@5 = \frac{3}{5} = 0.60 \quad (287)$$

Sistem ilk 5 önerisinin %60'ında kullanıcının gerçekten beğeneceği filmleri yakalamıştır. M_{11} (3 puan) ve M_9 (2 puan) ise eşik altına kaldıklarından hassasiyete katkıda bulunmamıştır.

Adım 4: Precision@3'ün hesaplanması ve karşılaştırma.

Sistem yalnızca ilk $K = 3$ filmi önerseydi:

$$Top3 = \{M_7, M_2, M_{11}\} \quad (288)$$

High kümesiyle kesişim:

$$Top3 \cap High = \{M_7, M_2\} \quad (289)$$

Kesişimde 2 film bulunur:

$$Precision@3 = \frac{2}{3} \approx 0.667 \quad (290)$$

Precision@3 değeri (0.667), Precision@5 değerinden (0.60) daha yüksektir. Bunun nedeni, ilk 3 öneride M_{11} haricindeki iki filmin de ilgili (yüksek puanlı) olmasıdır. İlk 5'e inildiğinde M_5

ilgili olmasına rağmen M_9 ilgisiz olduğundan, K büyüdükçe ilgisiz maddelerin de listeye girme olasılığı artar ve hassasiyet tipik olarak düşer.

Bu örnek, Precision@K metriğinin K değerine duyarlılığını ve tavsiye listesinin üst sıralarındaki başarının alt sıralara göre daha yüksek olabileceğini göstermektedir.